1/1 Силлабус

Текущий урок

Дисциплина: Операционные системы семейства Unix

Общая информация

Длительность

6 недель

Семестр

2 семестр

Аттестация

Экзамен

Авторы и преподаватели

Ефанов Дмитрий Валерьевич

Автор дисциплины

- Кандидат технических наук, доцент.
- Лауреат премии Правительства Российской Федерации в области науки и техники, призёр премии "Новатор Москвы" Правительства Москвы.
- Red Hat Certified Engineer, SCO Master Advanced Certified Engineer.
- Лектор HHSU МИФИ, РЭУ, МГУ, РТУ МИРЭА, Университета «Сириус».
- Эксперт трека «Информационные технологии» конкурса управленцев «Лидеры России».
- Член Российского общества «Знание».

О дисциплине

Аннотация

Учебная дисциплина «Операционные системы семейства Unix» посвящена изучению основных концепций GNU/Linux и формированию практических навыков по её установке, настройке и поддержке в актуальном состоянии. Программа охватывает работу в командной строке, управление файлами и процессами, подключение к сети, а также основы контейнеризации. Освоение дисциплины позволит студентам самостоятельно развернуть рабочее место инженера данных на основе ОС GNU/Linux для решения профессиональных задач.

Цель дисциплины

Познакомить студентов с основными концепциями ОС GNU/Linux и научить организовывать безопасное рабочее место инженера данных, что позволит отказаться от использования проприетарных ОС, и начать использовать всю мощь UNIX при решении своих прикладных задач.

В рамках дисциплины вы

узнаете:

- как устанавливать, настраивать и поддерживать в рабочем состоянии ОС GNU/Linux.

поймете:

- как устроена, как возникла, и для каких задач предназначена ОС GNU/Linux.

применете:

- для организации безопасного рабочего места инженера данных на основе ОС GNU/Linux, а также для отказа от проприетарных ОС на своём компьютере.

Пререквизиты дисциплины

Что нужно знать для прохождения дисциплины: начальные знания ОС GNU/Linux не требуются. Нужно быть уверенным пользователем компьютера с любой ОС и иметь опыт программирования на любом процедурном языке (желательно на языке C).

Технические требования для освоения

Желательно наличие собственного ноутбука с возможностью развёртывания системы виртуализации (VirtualBox, UTM для macOS) для установки ОС GNU/Linux, или свободного раздела на жёстком диске для установки ОС GNU/Linux в качестве рабочей ОС.

Силлабус дисциплины

Умение устанавливать, настраивать и поддерживать в актуальном состоянии рабочую станцию под управлением ОС GNU/Linux.

Тематический план

Неделя	Название модуля	Содержание	Оценивание
1	1. Введение в GNU/Linux	1. Понятие операционной системы 2. Работа в командной строке 3. Файлы и файловые системы 4. Основные операции над файлами	Тест - 5 баллов
2	2. Дискреционное управление доступом (DAC)	1. Угрозы безопасности и формальные модели 2. Управление пользователями 3. Управление паролями пользователей 4. Владение файлом 5. Права доступа к файлу 6. Флаги файла 7. Расширенные атрибуты файла 8. Списки прав доступа ACL 9. Особенности работы с ACL 10. Повышение прав, выполнение команд от имени другого пользователя	Тест - 5 баллов
3	3. Процессы	1. Понятие процесса 2. Жизненный цикл процесса 3. Состояния процесса 4. Алгоритм set-user-ID 5. Повышение прав, выполнение команд от имени другого пользователя 6. Привилегии процесса 7. Средства межпроцессного взаимодействия (IPC) 8. Сигналы 9. Управление заданиями	Тест - 5 баллов
4	4. Установка и настройка GNU/Linux	1. Блочные устройства и файловые системы 2. Блочные устройства и файловые системы 3. Блочные устройства и файловые системы 4. Понятие дистрибутива 5. RPM 6. DNF 7. Начальная загрузка. UEFI 8. Начальная загрузка. Загрузчик GRUB 9. Начальная загрузка. Ядро ОС	Тест - 5 баллов
4	Домашнее задание	Установка ОС GNU/Linux и настройка заданной конфигурации.	Домашнее задание - 20 баллов
5	5. Управление сервисами	1. Менеджер сервисов systemd. Сервисы 2. Менеджер сервисов systemd. Таргеты 3. Менеджер сервисов systemd. Управление питанием. 4. Менеджер сервисов systemd. Другие типы юнитов 5. Журналирование. События 6. Журналирование. Поиск событий 7. Многопользовательский режим 8. PAM 9. Модули PAM	Тест - 5 баллов
6	6. Контейнеры	1. Сетевые средства 2. Контрольные группы 3. Пространства имён 4. Виртуализация устройств 5. Платформа Docker	Тест - 5 баллов
7	Овсянниный проект	Итоговый проект	Проект - 50 баллов

Практика

Курс состоит из синхронных занятий онлайн и асинхронных занятий, которые студент проходит на платформе самостоятельно.

Форматы асинхронных заданий и упражнений на платформе: тестовые задания, выбор одного или нескольких правильных ответов, ввод ответа, написание кода с автоматической проверкой.

Синхронные занятия онлайн представляют собой практико-ориентированные семинарские занятия.

Какие активные задания будут встречаться на семинарских занятиях с преподавателем: разбор и обсуждение практических деталей работа в специализированном программном обеспечении.

В дисциплине заложено 1 домашнее задание и 1 проект. Вы будете решать задачи, сдадите их на проверку ментору и получите оценку и обратную связь.

Система оценивания

Успешное освоение дисциплины предполагает:

- выполнение учебных заданий (текущая аттестация);
- сдачу экзамена (итоговая аттестация).

Оценивание индивидуальной работы студента на платформе производится следующим образом:

Текущая аттестация	Итоговая аттестация
Прогресс в тем: тестирование в модулях и практические задания	Экзаменационное задание

Итоговый балл = текущая + итоговая аттестации.

Максимальный балл, который можно получить в течение семестра — 50 баллов.

Максимальный балл за экзамен — 50 баллов.

90-100	A	отлично	5
85-89	B	хорошо	4
75-84	C		
70-74	D	удовлетворительно	3
65-69	E		
60-64	E	неудовлетворительно	2
0-59	F		

Максимальный суммарный балл за всю дисциплину: 100 баллов. Оценка выставляется по шкале МИФИ.

О пересдаче

При получении оценки «неудовлетворительно» по итогам семестра вы отправляетесь на пересдачу. Тем вам будет предложено выполнить дополнительные задания.

Более подробные условия пересдач будут объявлены ближе к концу семестра.

Обращаем внимание, что получить оценку выше «4» на пересдаче невозможно. Рекомендуем выполнять все задания в срок и не посещать дополнительные аттестации.

‹ Предыдущий урок Следующий урок ›

SkillFactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX Разработка TUNE-IT

1/9 Введение

текущий урок

Модуль посвящён рассмотрению базовых концепций ОС GNU/Linux. Особое внимание уделяется изучению концепции файла и командного интерфейса работы с файлами.

Как полученные знания пригодятся в профессиональном развитии

Операционная система — это программное обеспечение (ПО), без которого невозможно функционирование компьютера и работа прикладных программ. Поэтому и обычный пользователь и тем более разработчик обязательно взаимодействуют с ОС. Владение средствами ОС позволит эффективно и безопасно решать прикладные задачи.

Цели и результат модуля

Студент, освоивший программу первого модуля, должен:

1. Понимать назначение и основные функции ОС.
2. Знать основные компоненты ОС GNU/Linux.
3. Понимать концепцию файла и знать атрибуты файла.
4. Уметь работать в командной строке.
5. Владеть базовыми командами для работы с файлами и директориями.

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX Разработка TUNE-IT

Назначение и задачи операционной системы

Существует много разных определений операционной системы (ОС). Давайте рассмотрим ГОСТовское определение.

> Операционная система (operating system) – это совокупность системных программ, предназначенная для обеспечения определенного уровня эффективности системы обработки информации за счет автоматизированного управления ей работой и предоставляемого пользователю определенного набора услуг [ГОСТ 15971-90].

Там же даётся определение системы обработки информации.

> Система обработки информации – это совокупность технических средств и программного обеспечения, а также методов обработки информации и действий персонала, обеспечивающая выполнение автоматизированной обработки информации [ГОСТ 15971-90].

Систему обработки информации также называют автоматизированной системой или информационной системой.

Из этих определений можно выделить следующие свойства операционной системы:

- Операционная система является основой построения системы автоматизированной обработки информации, называемой также автоматизированной системой или информационной системой (ИС).
- Операционная система предоставляет собой набор программ, которые относятся к классу системного программного обеспечения, и вместе с техническими средствами формируют вычислительную систему (computer system).
- Операционная система управляет техническими средствами вычислительной системы, также называемыми аппаратными средствами или платформами (hardware), которые включают центральный процессор (ЦП), оперативную память (ОП), устройства ввода-вывода и устройства хранения.
- Операционная система предоставляет пользователям набор услуг, также называемых сервисами. Пользователь обращается к сервисам операционной системы для выполнения своих задач и запускает программы в среде операционной системы.

От того, насколько эффективно операционная система управляет аппаратными средствами и предоставляет сервисы пользователю, зависит эффективность системы в целом. Так как существует множество разнообразных задач обработки информации, невозможно с помощью одной операционной системы обеспечить приемлемую эффективность их выполнения во всех случаях.

Если за точку отсчёта истории операционных систем взять создание в компании General Motors операционной системы для мейнфрейма IBM 704 в 1956 году, то за более чем шестидесятилетнюю историю разработано множество самых разных операционных систем, предназначенных для функционирования на разных аппаратных платформах и выполнения разных задач обработки информации.

> Операционная система, которая позволяет одновременно и независимо выполнять несколько программ, принадлежащих разным пользователям, называется многопользовательской (multiuser).

Рассмотрим основные функции, которые должна выполнять современная многопользовательская операционная система:

- Обеспечивать загрузку пользовательских программ в оперативную память и их исполнение.
- Обеспечивать управление памятью.
- Обеспечивать работу с устройствами долговременной памяти.
- Предоставлять более или менее стандартизованный доступ к различным периферийным устройствам.
- Предоставлять некоторый пользовательский интерфейс.
- Предоставлять механизм аутентификации для проверки личности пользователя.
- Предоставлять механизм управления доступом пользователей к ресурсам друг друга.
- Предоставлять механизм защиты от ошибочных пользовательских программ, которые могут блокировать другие программы, запущенные в системе.
- Предоставлять механизм защиты от вредоносных пользовательских программ, которые могут шпионить за деятельностью других пользователей или мешать им работать.
- Предоставлять механизм учета, ограничивающий количество ресурсов, выделенных каждому пользователю.
- Предоставлять механизм регистрации действий пользователей.

Данный курс посвящён изучению многопользовательской операционной системы общего назначения GNU/Linux, которая относится к POSIX-совместимым операционным системам. Иногда такие ОС объединяют понятием семейство UNIX.

Основные понятия ОС GNU/Linux

В ОС GNU/Linux, как и в других UNIX-подобных ОС, двумя основными системными сущностями являются процесс и файл. Чтобы связать внутренние системные сущности с реальным миром, а также обеспечить многопользовательскую работу, вводятся понятие пользователя системы. Самой важной программой ОС является ядро.

Подробно пользователи будут рассматриваться в 2 модуле, а процессы – в 3 модуле. Здесь же дадим самое простое определение процесса:

> Процесс – это программа во время исполнения.

Файл – это сложный механизм, который позволяет сначала сохранить на диске нужную информацию, а потом найти её и прочитать или изменить, при этом обеспечивая контроль доступа к информации со стороны других пользователей. Самым простым определением файла будет следующее:

> Файл – это способ работы с информацией на устройстве хранения.

Подробно файлы будут рассматриваться далее в этом модуле.

Ядро операционной системы

Основная программа операционной системы называется ядром (kernel). Из всех программ операционной системы, ядро самым первым загружается в ОП во время начальной загрузки системы (system boots). Ядро содержит базовую функциональность для обеспечения определенного уровня эффективности системы в целом. Поэтому часто, говоря об операционной системе, подразумевают её ядро, и наоборот. В нашем случае, программа linux является ядром операционной системы, а остальные программы разрабатываются в рамках проекта GNU и других многочисленных проектах.

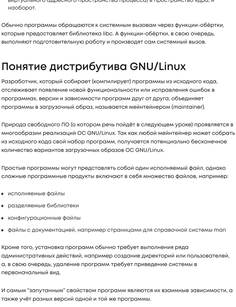
Ядро выполняет две основные задачи:

- Управление аппаратными средствами.
- Предоставление среды исполнения для программ (создание «слепка идеального компьютера»).

В современных операционных системах пользовательская программа не может напрямую взаимодействовать с аппаратными средствами. Когда программа хочет обратиться к аппаратному ресурсу, она должна выполнить запрос к ядру операционной системы. Ядро выполнит различные проверки такого запроса и, в случае его легитимности, будет взаимодействовать с соответствующими аппаратными средствами от имени пользовательской программы.

Чтобы реализовать такой механизм, современные операционные системы полагаются на наличие определенных аппаратных функций, которые запрещают пользовательским программам напрямую взаимодействовать с низкоуровневыми компонентами оборудования или получать доступ к произвольным участкам ОП. В частности, аппаратное обеспечение предоставляет по крайней мере два различных режима выполнения для ЦП: непривилегированный режим для пользовательских программ и привилегированный режим для ядра. Эти режимы называются режим пользователя (user mode) и режим ядра (kernel mode) соответственно.

Таким образом, чтобы гарантировать невозможность обхода механизмов защиты, операционная система должна изолировать их аппаратные механизмы защиты, предоставляемые ЦП.



На рисунке представлена архитектура операционной системы GNU/Linux. Показаны основные подсистемы ядра, а также соответствующие им аппаратные средства. В дистрибутиве GNU/Linux могут быть включены самые разнообразные программы. Иногда трудно провести чёткую грань между программами операционной системы и программами пользователя. Все программы выполняются в виде процессов и с точки зрения управления процессами, системные и пользовательские программы ничем не отличаются. Тем не менее, на рисунке выделены основные программы операционной системы, без которых работа пользователя в системе была бы невозможна.

Обратите внимание на то, что процессы выполняются в непривилегированном режиме пользователя, а ядро – в привилегированном режиме ядра. Для взаимодействия процессов с ядром используется интерфейс системных вызовов.

Системные вызовы

Прикладным программным интерфейсом (application programming interface, API) ядра является интерфейс системных вызовов. С помощью системных вызовов ядро предоставляет программам пользователей (процессам) доступ к своим сервисам.

> Системный вызов – это точка входа в ядро для выполнения функций ядра по запросу процесса.

Рассмотрим общие характеристики системных вызовов:

- Системный вызов изменяет состояние ЦП, переводя его из пользовательского режима в режим ядра, позволяя таким образом ЦП получить доступ к защищенной памяти ядра.
- Набор системных вызовов жёстко зафиксирован. Каждый системный вызов идентифицируется по уникальному номеру. Пользовательские программы обращаются к вызовам по их именам.
- У системного вызова может быть набор аргументов, определяющих информацию, которой должно быть передано из пространства пользователя (то есть из виртуального адресного пространства процесса) в пространство ядра, и наоборот.

Обычно программы обращаются к системным вызовам через функции-обёртки, которые предоставляет библиотека libc. А функции-обёртки, в свою очередь, выполняют подготовительную работу и производят сам системный вызов.

Понятие дистрибутива GNU/Linux

Разработчик, который собирает (компилирует) программы из исходного кода, отслеживает появление новой функциональности или исправление ошибок в программах, версии и зависимости программ друг от друга, объединяет программы в загрузочный образ, называется мейнтейнером (maintainer).

Природа свободного ПО (о котором речь пойдёт в следующем уроке) проявляется в многообразии реализаций ОС GNU/Linux. Так как любой мейнтейнер может собрать из исходного кода свой набор программ, получается потенциально бесконечное количество вариантов загрузочных образов ОС GNU/Linux.

Простые программы могут представлять собой один исполняемый файл, однако сложные программные продукты включают в себя множество файлов, например:

- исполняемые файлы
- разделяемые библиотеки
- конфигурационные файлы
- файлы с документацией, например страницами для справочной системы man

Кроме того, установка программ обычно требует выполнения ряда административных действий, например создание директорий или пользователей, а, в свою очередь, удаление программ требует приведения системы в первоначальный вид.

И самым "запутанным" свойством программ являются их взаимные зависимости, а также учёт разных версий одной и той же программы.

Чтобы программами было легче управлять, их файлы вместе с инструкциями по сборке, установке и удалению, упаковывают в специальные архивы, называемые пакетами.

Существует несколько систем пакетирования, самыми популярными из которых являются RPM и DEB.

> Функционально-полный набор пакетов называют репозиторием пакетов.

Загрузочный образ, включающий программу установки и минимально-необходимую часть репозитория, называют дистрибутивом GNU/Linux.

Существует превеликое множество дистрибутивов GNU/Linux, которые различаются:

- Аппаратной платформой, для которой собраны пакеты.
- Возможностями программы установки.
- Составом пакетов (например, используемой графической оболочкой).
- Условиями технической поддержки.

В данном курсе для выполнения практических заданий будет использоваться дистрибутив, основанный на пакетах RPM, Fedora или РЕД ОС. Вы можете установить дистрибутив GNU/Linux непосредственно на жёсткий диск компьютера и настроить так называемую двойную загрузку, или установить систему виртуализации (например, VirtualBox), и установить дистрибутив GNU/Linux в качестве гостевой ОС.

Так как процесс установки сильно привязан к конкретному дистрибутиву, прошу обратиться к соответствующему руководству по установке, например, к Руководству по установке РЕД ОС.

Политика конфиденциальности | Пользовательское соглашение | Обратиться к поддержку
Функциональные характеристики ПО | Руководство пользователя | Стоимость ПО

SkillFactory – онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX | Разработка TUNE-IT

Командная оболочка BASH

Командная строка - это текстовый интерфейс, который предназначен для ввода инструкций в компьютерную систему.

Командная оболочка (shell) - это программа, которая умеет читать команды, выполнять их интерпретацию, запускать на исполнение и возвращать результат их работы пользователю.

Оболочкой по умолчанию во многих дистрибутивах GNU/Linux является GNU Bourne-Again Shell (bash). Bash - это улучшенная версия одной из самых успешных оболочек, используемых в UNIX-подобных системах, Bourne Shell (sh).

В интерактивном режиме bash отображает строку, ожидая команду от пользователя. Это называется *приглашением* командной строки. Когда обычный пользователь запускает оболочку, *приглашение* по умолчанию заканчивается символом $. Символ $ заменяется символом #, если оболочка запущена от имени суперпользователя root. Это предупреждает пользователя о том, что он работает от имени суперпользователя, что помогает избежать случайных ошибок, которые могут повлиять на всю систему.

Использование bash для выполнения команд может быть очень эффективным. Оболочка bash предоставляет язык сценариев, с помощью которого можно автоматизировать задачи. Оболочка обладает дополнительными возможностями, которые могут упростить или сделать возможными операции, которые не получится эффективно выполнить с помощью графических инструментов.

Команды, вводимые в командной строке, состоят из трёх основных частей:

- Имя команды для выполнения
- Опции, также называемые ключами или *параметрами*, для настройки поведения команды
- Аргументы, над которыми выполняется команда

Формат командной строки следующий:

```
$ <имя_команды> [опции] [аргументы]
```

Команда — это имя программы для выполнения. За ней может следовать один или несколько параметров, которые настраивают поведение команды или то, что она будет делать. Параметры обычно начинаются с одного или двух дефисов (например, -a или --all), чтобы отличать их от аргументов. За командами также может следовать один или несколько аргументов, которые обрабатываются командой.

Каждое слово, введенное в оболочку, разделяется пробелами. Команды - это имена программ, установленных в системе. Каждая команда имеет свои параметры и аргументы.

Когда вы будете готовы выполнить команду, нажмите клавишу Enter. Вводите каждую команду на отдельной строке. Выводы команды отображаются перед появлением следующего приглашения командной строки.

Если вы хотите ввести более одной команды на одной строке, используйте точку с запятой (;) в качестве разделителя команд. Точка с запятой относится к классу символов, называемых метасимволами, которые имеют особое значение для bash. В этом случае вывод обеих команд будет отображен перед появлением следующей *приглашения* командной строки.

Вход пользователя в систему

Для запуска командной оболочки необходимо войти в систему компьютера через терминал. Терминал — это текстовый интерфейс, используемый для ввода команд и вывода информации из компьютерной системы. Существует несколько способов сделать это.

К компьютеру может быть подключена аппаратная клавиатура и дисплей для ввода и вывода, называемые вместе физическая консоль. Физическая консоль поддерживает несколько виртуальных консолей, которые могут запускать отдельные терминалы. Каждая виртуальная консоль поддерживает независимую сессию входа в систему. Переключаться между ними можно, одновременно нажимая Ctrl+Alt и функциональную клавишу (F1–F6). Большинство виртуальных консолей запускают терминал, предоставляющий текстовое *приглашение* для входа, и если вы правильно введёте имя пользователя и пароль, то войдете в систему и увидите *приглашение* командной оболочки.

Система может предоставлять графическое *приглашение* для входа на одной из виртуальных консолей. Вы можете использовать его для входа в графическую среду. Графическая среда также работает на виртуальной консоли.

Чтобы получить приглашение командной строки, необходимо запустить терминальную программу в графической среде. Приглашение командной строки отображается в окне приложения графической терминальной программы.

Многие системные администраторы предпочитают не запускать графическую среду на своих серверах. Это позволяет использовать ресурсы, которые в противном случае использовались бы графической средой, для работы сервера.

Например, в Fedora, если графическая среда доступна, экран входа в систему будет запускаться на первой виртуальной консоли, называемой tty1. Пять дополнительных текстовых приглашений для входа в систему доступны на виртуальных консолях со второй по шестую.

Если вы входите в систему, используя графический экран входа в систему, ваша графическая среда запустится на первой виртуальной консоли, которая в данный момент не используется сеансом входа в систему. Обычно ваш графический сеанс заменяет *приглашение* для входа в систему на второй виртуальной консоли (tty2). Однако, если эта консоль используется активным текстовым сеансом входа в систему (а не просто приглашением для входа в систему), используется следующая свободная виртуальная консоль.

Графический экран входа в систему продолжает работать на первой виртуальной консоли (tty1). Если вы уже вошли в графическую сессию и вошли в систему как другой пользователь на графическом экране входа в систему или использовали пункт меню «Сменить пользователя» для смены пользователя в графической среде без выхода из системы, для этого пользователя будет запущена другая графическая среда на следующей свободной виртуальной консоли.

При выходе из графической среды она завершит работу, и физическая консоль автоматически переключится обратно на графический экран входа в систему на первой виртуальной консоли.

Выход пользователя из системы

Когда вы закончите работу с оболочкой и захотите выйти, вы можете выбрать один из нескольких способов завершения сеанса. Вы можете ввести команду exit, чтобы завершить текущий сеанс оболочки. В качестве альтернативы, завершить сеанс можно, нажав Ctrl+D.

Примеры простых команд

Команда date отображает текущую дату и время. Она также может использоваться суперпользователем для установки системных часов. Аргумент, начинающийся со знака плюс (+), задает строку формата для команды date.

```
$ date
Mon Jan 26 01:44:56 AM MSK 2026
$ date +%d.%m.%Y
26.01.2026
```

Команда passwd изменяет пароль пользователя. Текущий пароль для учетной записи должен быть указан до того, как будет разрешено изменение. По умолчанию команда passwd настроена так, чтобы требовать от пользователя надежный пароль, состоящий из строчных букв, заглавных букв, цифр и символов, и не основанный на словарных словах. Суперпользователь может использовать команду passwd для изменения паролей других пользователей.

```
$ passwd
Текущий пароль: old_password
Новый пароль: new_password
Повторите новый пароль: new_password
passwd: все токены аутентификации успешно обновлены.
```

В UNIX-подобных ОС не используется расширение файлов для их классификации по функциональности. Команда file сканирует начало содержимого файла и отображает его назначение. Файлы для определения передаются в качестве аргументов команде.

```
$ file /etc/passwd
/etc/passwd: ASCII text
$ file /bin/passwd
/bin/passwd: setuid ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically
linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=855a6c27759a90b72e1d115db4d719a1f637d, for GNU/Linux 3.2.0, stripped
$ file /etc
/etc: directory
```

Автодополнение по клавише Tab

Автодополнение по клавише Tab позволяет пользователю быстро завершать команды или имена файлов после того, как он ввел в командной строке достаточно символов, чтобы сделать имя уникальным. Если введенные символы не уникальны, двойное нажатие клавиши Tab отобразит все команды, начинающиеся с уже введенных символов.

Автодополнение по клавише Tab можно использовать для завершения имен файлов при их вводе в качестве аргументов команд.

При нажатии клавиши Tab завершается максимально возможная часть имени файла. Повторное нажатие клавиши Tab приводит к тому, что bash выводит список всех файлов, соответствующих текущему шаблону. Поэтому вводите дополнительные символы, пока имя не станет уникальным, а затем используйте автодополнение по клавише Tab для завершения команды.

Также для многих команд можно использовать автодополнение по клавише Tab для поиска нужных опций или аргументов.

История команд

Команда history отображает список ранее выполненных команд, перед которыми стоит номер команды. Восклицательный знак (!) — это метасимвол, используемый для выполнения предыдущих команд без необходимости их повторного ввода. Команда !номер подставляет команду, соответствующую указанному номеру. Команда !строка находит самую последнюю команду, начинающуюся с указанной строки.

Клавиши со стрелками можно использовать для навигации по предыдущим командам в истории оболочки. Клавиша UpArrow редактирует предыдущую команду в списке истории. Клавиша DownArrow редактирует следующую команду в списке истории. Клавиши LeftArrow и RightArrow перемещают курсор влево и вправо в текущей команде из списка истории, чтобы отредактировать её перед запуском.

Очень удобно использовать комбинацию клавиш Esc+. или Alt+. для вставки последнего слова предыдущей команды в текущее положение курсора. Повторное использование комбинации клавиш зажмет этот текст последним словом еще более ранних команд в истории. Сочетание клавиш Alt+. более предпочтительно, так как можно удерживать клавишу Alt и многократно нажимать клавишу . для удобного перемещения по предыдущим командам.

При интерактивном использовании bash предоставляет функцию редактирования командной строки. Это позволяет пользователю использовать команды текстового редактора для перемещения внутри командной строки и изменения текущей вводимой команды. Хоткеи представлены в таблице.

Хоткей	Описание
Ctrl+A	Переход в начало командной строки.
Ctrl+E	Переход в конец командной строки.
Ctrl+U	Очистить от курсора до начала командной строки.
Ctrl+K	Очистить от курсора до конца командной строки.
Ctrl+LeftArrow	Переход к началу предыдущего слова в командной строке.
Ctrl+RightArrow	Переход к концу следующего слова в командной строке.
Ctrl+R	Поиск шаблона в списке команд истории.

Политика конфиденциальности Пользовательское соглашение Обратиться в поддержку
Функциональные характеристики ПО Руководство пользователя Стоимость ПО
Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026
Дизайн MAX Разработка TUNE-IT

Урок 4. Файлы и файловые системы

Как было сказано, одной из базовых задач ОС является файл. В этом уроке мы рассмотрим, что такое файлы, какие они бывают и как устроена файловая система.

Понятие файловой системы

Файловая система — это часть операционной системы, которая предоставляет приложениям и пользователям интерфейс для работы с данными, хранящимися на устройствах хранения.

Файловая система по своему функциональному назначению можно разделить на три уровня:

- Дисковая файловая система, предназначена для организации хранения данных на устройствах хранения, например, на жестких дисках. Чтобы создать дисковую файловую систему, необходимо блочное устройство. Существует множество дисковых файловых систем, например, ext4, NTFS, APFS.
- Сетевая файловая система, обеспечивает доступ к файлам, находящимся на удаленных компьютерах (сервисах) сквозь интерфейс локальной файловой системы. Примеры: NFS, SMB/CIFS.
- Специальная (или псевдо-) файловая система, предназначена для доступа к данным ядра операционной системы. Например, в Linux есть файловая система procfs, которая предоставляет доступ к информации о процессах, а sysfs — к информации об устройствах.

В данном уроке мы включили системные будет подразумевается дисковая файловая система.

Файловая система — это способ хранения информации на устройстве хранения.

Рассмотрим в упрощенном виде структуру файловой системы ОС UNIX:

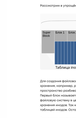


Таблица inodes

Для создания файловой системы необходимо выделить часть ее устройства хранения, например, раздел на диске. При создании файловой системы, помимо блоков с данными, создаются специальные блоки, которые называются inode. Inode — это структура данных, которая описывает файл в файловой системе и дает ссылки на блоки с данными. Подробнее о структуре inode мы поговорим в следующей главе. Все inode хранятся в специальной таблице inodes. Количество inode ограничено, поэтому в файловой системе может быть ограниченное количество файлов.

Существуют некоторые операционные файловые системы, поддерживающие работу без inode, например, FAT. Они используются для решения специальных задач, например [хранения данных на флеш-накопителях](https://ru.wikipedia.org/wiki/FAT) и [для обмена данными](https://ru.wikipedia.org/wiki/FAT) между различными системами, например, в [загрузочном разделе](https://ru.wikipedia.org/wiki/FAT) UEFI.

Индексный дескриптор файла

В операционных системах современного типа, таких как Linux, Solaris, и в информационных системах отдельных фирм информация о файле хранится в inode, в которых находятся номера блоков, в информации, которая называет файл. Будем называть данные файла, в информации, которая называет файл. Будем называть данные файла, в информации, которая называет файл.

Для описания inode, для описания inode, для предоставления структуры данных, которая называется inode. Inode, inode, inode,

5/9 Урок 4. Основные операции над файлами

текущий урок

Рассмотрим наиболее часто используемые команды для работы с файлами и директориями.

Синтаксис многих команд подчиняется следующей логике:

```
$ <команда> <источник> <приёмник>
```

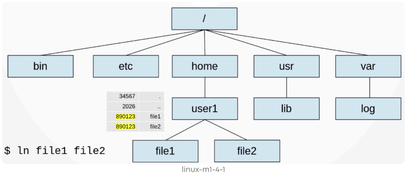
Если у команды два аргумента, то они оба интерпретируются, как имена файлов. Первый файл - это существующий файл, а второй файл - это файл, который появится после выполнения команды.

Если у команды несколько аргументов, то последний аргумент интерпретируется, как имя директории, в которую нужно скопировать или переместить файлы, заданные предыдущими аргументами.

```
$ cp f1 f2 # скопирует существующий файл f1 в новый файл f2
$ cp f1 f2 f3 d1 # скопирует существующие файлы f1, f2, f3 в директорию d1 с теми же
именами
```

Создание жёсткой ссылки на файл

Для создания жёсткой ссылки на файл используется команда ln.



linux-ml-4-1

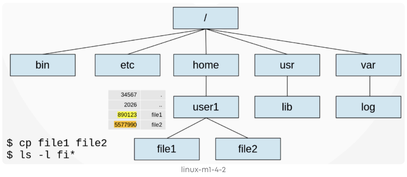
В результате создания новой жёсткой ссылки на файл, в иноде меняется:

- количество жёстких ссылок
- время изменения (модификации) инода

А в директории появляется новая запись с тем же номером инода.

Копирование файла

Для копирование файла используется команда cp.

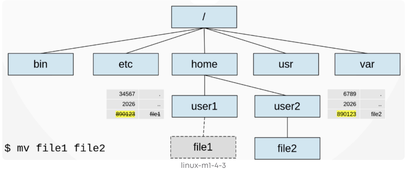


linux-ml-4-2

В результате копирования файла, создаётся новый файл, т.е. выделяется новый инод. В исходном иноде меняется время чтения файла. А в директории появляется новая запись с номером нового инода.

Перемещение файла

Для перемещения файла используется команда mv.



linux-ml-4-3

Результаты перемещения файла зависят от того, происходит оно внутри одной файловой системы, или файл перемещается в другую файловую систему. Если внутри одной файловой системы, то жёсткая ссылка переносится из одной директории в другую, возможно с новым именем. Если перемещение выполняется внутри одной директории, то происходит переименование файла.

Базовые команды

Рассмотрим список команд, которые будут использоваться в данном курсе.

Базовые команды для ориентации в системе

uname - вывести информацию о системе

id - вывести информацию о пользователе

who - показать пользователей, работающих в системе

pwd - вывести текущую (также называемую рабочей) директорию

date - вывести или установить дату и время

ps - посмотреть информацию о выполняющихся процессах

cal - вывести календарь

echo - вывести строку текста (по сути - вывести свои аргументы)

clear - очистить экран

exit или ^d - завершить ввод

Команды управления файлами

touch - установка меток времени на файл; если файл не существует, то он создаётся

ln - создание жёсткой или символьной ссылки на файл

cp - копирование файла

mv - перемещение или переименование файла

rm - удаление файла

find - поиск файла по атрибутам

Команды управления директориями

mkdir - создание директории

cd - изменение текущей (рабочей) директории

ls - просмотр содержимого директории

rmdir - удаление директории

rm -r - удаление иерархии директорий

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026
Дизайн MAX Разработка TUNE-IT

6/9 Лабораторная работа

текущий урок

В данной лабораторной работе применим на практике все знания, полученные в модуле, а именно: работу в командной строке и управление файлами.

Цель лабораторной работы - увидеть собственными глазами, как различные операции над файлом влияют на значения полей структуры ext4_inode.

Сначала выведем текущее время, определим блочное устройство (раздел), содержащее файловую систему, выведем идентификаторы пользователя, маску создания файлов и контекст безопасности текущей директории.

```
$ date
Fri Jan 23 01:37:32 AM MSK 2026
$ df .
Filesystem     1K-blocks     Used Available Use% Mounted on
/dev/nvme0n1p6  32765712 27192296   3879312  88% /home
$ id
uid=1000(user0) gid=1000(user0) groups=1000(user0)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
$ umask
0022
$ ls -dZ .
unconfined_u:object_r:user_home_t:s0 .
```

Теперь создадим файл file1 и определим номер его инода.

```
$ touch file1
$ ls -i file1
394359 file1
```

Дальнейшие действия будем выполнять в режиме суперпользователя. Выведем содержимое инода с номером 394359.

```
# debugfs -R 'stat <394359>' /dev/nvme0n1p6
debugfs 1.47.1 (20-May-2024)
Inode: 394359   Type: regular    Mode:  0644   Flags: 0x80000
Generation: 3717866247    Version: 0x00000000:00000003
User:  1000   Group:  1000   Project:     0   Size: 0
File ACL: 0
Links: 1   Blockcount: 0
Fragment:  Address: 0    Number: 0    Size: 0
 ctime: 0x6972a6ba:d2feebfc -- Fri Jan 23 01:37:46 2026
 atime: 0x6972a6ba:d2feebfc -- Fri Jan 23 01:37:46 2026
 mtime: 0x6972a6ba:d2feebfc -- Fri Jan 23 01:37:46 2026
crtime: 0x6972a6ba:d2feebfc -- Fri Jan 23 01:37:46 2026
Size of extra inode fields: 32
Extended attributes:
  security.selinux (37) = "unconfined_u:object_r:user_home_t:s0\000"
Inode checksum: 0x463be0f0
EXTENTS:
```

Видно, что права доступа к файлу установлены в соответствии с пользовательской маской, идентификаторы владельца и группы унаследованы из соответствующих идентификаторов процесса, который создал файл. Все временные метки (timestamps) установлены в одинаковое значение со временем создания файла crtime. Контекст безопасности файла унаследован от контекста безопасности текущей директории. Цифра 37 - это длина строки, содержащей контекст безопасности, включая 0-ой байт. Так как файл пустой (Size: 0), то и для хранения файла не выделены ни блоки (Blockcount: 0) ни экстент (EXTENTS).

Теперь будем выполнять действия над файлом file1 и наблюдать, как меняется содержимое полей структуры ext4_inode. Запишем данные в файл.

```
# cat > file1
Hello, МЕРнI!
# debugfs -R 'stat <394359>' /dev/nvme0n1p6
debugfs 1.47.1 (20-May-2024)
Inode: 394359   Type: regular    Mode:  0644   Flags: 0x80000
Generation: 3717866247    Version: 0x00000000:00000004
User:  1000   Group:  1000   Project:     0   Size: 14
File ACL: 0
Links: 1   Blockcount: 8
Fragment:  Address: 0    Number: 0    Size: 0
 ctime: 0x6972a7cf:95324794 -- Fri Jan 23 01:42:23 2026
 atime: 0x6972a6ba:d2feebfc -- Fri Jan 23 01:37:46 2026
 mtime: 0x6972a7cf:95324794 -- Fri Jan 23 01:42:23 2026
crtime: 0x6972a6ba:d2feebfc -- Fri Jan 23 01:37:46 2026
Size of extra inode fields: 32
Extended attributes:
  security.selinux (37) = "unconfined_u:object_r:user_home_t:s0\000"
Inode checksum: 0x83732db5
EXTENTS:
(0):363530
```

В файл записано 14 байтов, включая символ перевода строки, для чего выделены блоки (Blockcount: 8) и экстент (EXTENTS:(0):7788915). Изменились время модификации самого инода ctime и время модификации содержимого файла mtime.

Выведем данные из файла.

```
# cat file1
Hello, МЕРнI!
# debugfs -R 'stat <394359>' /dev/nvme0n1p6
debugfs 1.47.1 (20-May-2024)
Inode: 394359   Type: regular    Mode:  0644   Flags: 0x80000
Generation: 3717866247    Version: 0x00000000:00000004
User:  1000   Group:  1000   Project:     0   Size: 14
File ACL: 0
Links: 1   Blockcount: 8
Fragment:  Address: 0    Number: 0    Size: 0
 ctime: 0x6972a7cf:95324794 -- Fri Jan 23 01:42:23 2026
 atime: 0x6972a83f:c91b52f8 -- Fri Jan 23 01:44:15 2026
 mtime: 0x6972a7cf:95324794 -- Fri Jan 23 01:42:23 2026
crtime: 0x6972a6ba:d2feebfc -- Fri Jan 23 01:37:46 2026
Size of extra inode fields: 32
Extended attributes:
  security.selinux (37) = "unconfined_u:object_r:u
ser_home_t:s0\000"
Inode checksum: 0xe1511981
EXTENTS:
(0):363530
```

Изменилось время последнего доступа к файлу (т.е. чтения из файла) atime.

Создадим жёсткую ссылку на файл.

```
# ln file1 file12
# debugfs -R 'stat <394359>' /dev/nvme0n1p6
debugfs 1.47.1 (20-May-2024)
Inode: 394359   Type: regular    Mode:  0644   Flags: 0x80000
Generation: 3717866247    Version: 0x00000000:00000004
User:  1000   Group:  1000   Project:     0   Size: 14
File ACL: 0
Links: 2   Blockcount: 8
Fragment:  Address: 0    Number: 0    Size: 0
 ctime: 0x6972a95f:37e097b4 -- Fri Jan 23 01:49:03 2026
 atime: 0x6972a83f:c91b52f8 -- Fri Jan 23 01:44:15 2026
 mtime: 0x6972a7cf:95324794 -- Fri Jan 23 01:42:23 2026
crtime: 0x6972a6ba:d2feebfc -- Fri Jan 23 01:37:46 2026
Size of extra inode fields: 32
Extended attributes:
  security.selinux (37) = "unconfined_u:object_r:user_home_t:s0\000"
Inode checksum: 0x0fc2b072
EXTENTS:
(0):363530
```

Увеличилось количество жёстких ссылок Links и изменилось время модификации самого инода ctime.

Удалим файл. Для этого нужно удалить обе жёсткие ссылки.

```
# \rm file1 file12
```

Так как rm - это алиас 'rm -i', а потому rm будет задавать дополнительные вопросы, с помощью обратного слэша можно отключить алиас и выполнить исходную команду.

Файл удаляется после того, как удаляется последняя жёсткая ссылка на этот файл. Т.е. уже нет имени, по которому можно получить доступ к файлу.

```
# debugfs -R 'stat <394359>' /dev/nvme0n1p6
debugfs 1.47.1 (20-May-2024)
Inode: 394359   Type: regular    Mode:  0644   Flags: 0x80000
Generation: 3717866247    Version: 0x00000000:00000004
User:  1000   Group:  1000   Project:     0   Size: 0
File ACL: 0
Links: 0   Blockcount: 0
Fragment:  Address: 0    Number: 0    Size: 0
 ctime: 0x6972ad35:93bfeb4c -- Fri Jan 23 02:05:25 2026
 atime: 0x6972a83f:c91b52f8 -- Fri Jan 23 01:44:15 2026
 mtime: 0x6972ad35:93bfeb4c -- Fri Jan 23 02:05:25 2026
crtime: 0x6972a6ba:d2feebfc -- Fri Jan 23 01:37:46 2026
 dtime: 0x6972ad35:(93bfeb4c) -- Fri Jan 23 02:05:25 2026
Size of extra inode fields: 32
Extended attributes:
  security.selinux (37) = "unconfined_u:object_r:user_home_t:s0\000"
Inode checksum: 0x29c677c5
EXTENTS:
```

Файл удалён из файловой системы, о чём свидетельствуют нулевые счётчики жёстких ссылок Links и блоков Blockcount, а также пустой экстент EXTENTS. Хотя сам файл удалён (т.е. освобождены его блоки данных и индексный дескриптор), индексный дескриптор, который он занимал, продолжает существовать в файловой системе, и теперь в нём появилось время удаления файла dtime (оно, например, используется утилитами восстановления файлов).

Политика конфиденциальности Пользовательское соглашение Обратиться в поддержку
Функциональные характеристики ПО Руководство пользователя Стоимость ПО
Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026
Дизайн MAX Разработка TUNE-IT

7/9 Тестирование. Введение в GNU/Linux

текущий урок

Тестирование — до 13 апр. 2026 г. 23:59 MSK Завершено

Задание 1.1

0.5/0.5 points (graded)

Для чего предназначено ядро операционной системы?

- [x] Для создания среды выполнения программ
- [x] Для управления аппаратными средствами компьютера
- [ ] Для проверки пароля пользователя при входе в систему
- [ ] Для предоставление пользователю графического интерфейса

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.2

0.0/0.5 points (graded)

Какие типы файлов хранят свои данные на жёстком диске?

- [x] Обычные (регулярные) файлы
- [ ] Директории
- [ ] Символьные ссылки
- [ ] Файлы блочных устройств
- [ ] Файлы символьных устройств
- [ ] FIFO
- [ ] UNIX-сокеты

✖

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.3

0.5/0.5 points (graded)

Где хранятся метаданные (атрибуты) файла?

- (•) В специальной структуре данных, называемой инодом
- ( ) В специальной структуре данных, называемой жёсткой ссылкой
- ( ) В суперблоке файловой системы

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.4

0.5/0.5 points (graded)

Где хранится имя файла?

- ( ) В специальной структуре данных, называемой инодом
- (•) В специальной структуре данных, называемой жёсткой ссылкой
- ( ) В блоках с данными файла

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.5

0.5/0.5 points (graded)

Какой командой можно посмотреть время удаления файла?

- ( ) ls
- ( ) stat
- (•) debugfs
- ( ) file

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.6

0.5/0.5 points (graded)

На один файл (инод) имеется две жёсткие ссылки. Как определить, какая ссылка была создана раньше, а какая позже?

- (•) Нельзя определить
- ( ) По метке времени (timestamp) в иноде
- ( ) По информации в самой жёсткой ссылке

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.7

0.5/0.5 points (graded)

Какой номер у инода корневой директории?

- ( ) 0
- (•) 2
- ( ) 255

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.8

0.5/0.5 points (graded)

Сколько будет создано жёстких ссылок на только что созданную директорию?

- ( ) 1
- (•) 2
- ( ) 3

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.9

0.5/0.5 points (graded)

Каким образом команда file определяет назначение файла?

- (•) По первым байтам данных файла
- ( ) По расширению файла
- ( ) По информации из жёсткой ссылки на данный файл

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.10

0.5/0.5 points (graded)

При каких условиях файл считается удалённым?

- (•) Количество жёстких ссылок на файл стало равным нулю
- ( ) Размер файла стал равным нулю
- ( ) Пользователь выполнил команду rm для данного файла

✔

Show answer

Отправить You have used 1 of 1 attempt

‹ Предыдущий урок Следующий урок ›

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026.
Дизайн MAX Разработка TUNE-IT

8/9 Заключение

текущий урок

В данном модуле мы познакомились с ОС GNU/Linux, научились работать в командной строке, посмотрели структуру файловой системы, подробно изучили что такое файл, какие бывают файлы, какие атрибуты есть у файла, какие операции можно выполнять над файлами и директориями.

В следующем модуле мы продолжим говорить про файлы, но будем их рассматривать с точки зрения многопользовательского режима работы и информационной безопасности, научимся устанавливать права доступа на файлы, а также так настраивать атрибуты файлов, что их не сможет удалить даже суперпользователь (root).

Источники

- Ефанов, Д. В. Базовые механизмы защиты ядра Linux : учебное пособие / Д. В. Ефанов. — Москва : НИЯУ МИФИ, 2022. — 192 с. — ISBN 978-5-7262-2924-9. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/355520 (дата обращения: 22.01.2026). — Режим доступа: для авториз. пользователей.
- ГОСТ 15971-90. Системы обработки информации. Термины и определения. М.: Издательство стандартов, 1991
- Pro Bash Programming: Scripting the GNU/Linux Shell. Chris Johnson. Apress, 2009.
- The Intel 64 and IA-32 Architectures Software Developer's Manual. Volume 3 (3A, 3B, 3C & 3D): System Programming Guide. 2016.
- Керриск Майкл. Linux API. Исчерпывающее руководство. — СПб.: Питер, 2019.
- Бовет Д., Чезати М. Ядро Linux, 3-е изд. — СПб.: БХВ-Петербург, 2007.

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX Разработка TUNE-IT

1/13 Введение

текущий урок

Модуль посвящён рассмотрению концепции дискреционного управления доступом (DAC) в ОС GNU/Linux. DAC реализован во всех современных ОС и является базовым механизмом управления доступом процессов к файлам.

Как полученные знания пригодятся в профессиональном развитии

ОС GNU/Linux — это многопользовательская ОС. Поэтому управление доступом к файлам является краеугольным камнем для организации безопасного рабочего места. Владение средствами управления доступом позволит удобно и эффективно назначать права для участников одной команды/проекта, обеспечивая безопасность и гибкость работы.

Цели и результат модуля

Студент, освоивший программу второго модуля, должен:

1. Понимать общую концепцию DAC.
2. Уметь назначать владельца и группу-владельца файла.
3. Уметь устанавливать базовые права доступа на файл.
4. Уметь формировать ACL на файл.
5. Уметь переключаться между обычным пользователем и суперпользователем root.

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX Разработка TUNE-IT

Триада CIA

В связи с тем, что информация является предметом собственности (государства, коммерческой компании, отдельного лица), то неизбежно возникает проблема угрозы безопасности этой информации, заключающейся в неконтролируемом ее распространении, в хищении, несанкционированном уничтожении, искажении, передаче, копировании, блокировании доступа к информации. Следовательно, возникает проблема защиты информации от утечки и несанкционированных воздействий на информацию и её носители, а также предотвращения других форм незаконного вмешательства в информационные ресурсы и информационные системы.

> Угроза информационной безопасности — совокупность условий и факторов, создающих опасность нарушения информационной безопасности.

Существует множество классификаций угроз безопасности. Самой классической считается триада CIA, которая является аббревиатурой от слов Confidentiality (конфиденциальность), Integrity (целостность) и Availability (доступность).

Тогда можно определить угрозу информационной безопасности, как потенциальное событие или действие, которое может нанести ущерб данным, нарушив их конфиденциальность, целостность или доступность.

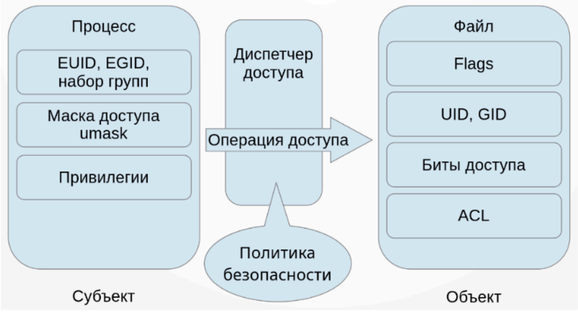
Посмотреть расширенный перечень угроз можно на сайте ФСТЭК РФ.

Как только сформулированы угрозы, можно разрабатывать механизмы защиты, предотвращающие реализацию угроз. Базовым механизмом защиты в ОС является механизм управления доступом.

Понятие DAC

Двумя основными механизмами управления доступом в операционных системах являются дискреционное управление доступом (discretionary access control, DAC) и мандатное управление доступом (mandatory access control, MAC). Далее в курсе будем называть эти механизмы DAC и MAC соответственно. С основами MAC мы познакомимся в конце курса, а в данном модуле мы подробно рассмотрим DAC.

С точки зрения информационной безопасности при осуществлении доступа всегда есть две стороны взаимодействия: субъект, осуществляющий доступ и объект, к которому осуществляется доступ. Другими словами, субъект пытается выполнить какие-то операции над объектом. Между субъектом и объектом находится диспетчер доступа (также называемый монитором взаимодействия), который на основании некой политики безопасности решает: разрешить субъекту доступ к объекту или нет.



linux-m1-1-1

Особенностью DAC является то, что у каждого объекта доступа есть владелец (пользователь), который самостоятельно устанавливает права на свой объект. Программы также выполняются от имени определённых пользователей. Диспетчер доступа должен решить, можно ли одному пользователю, который запустил программу, получить доступ к файлу другого пользователя.

В таблице показаны особенности принятия решения о доступе в DAC. Хотя мы будем рассматривать процессы в следующем модуле, перечислим здесь атрибуты всех участников взаимодействия.

Свойства	Реализация
Атрибуты безопасности процесса	UID, EUID, FSUID, GID, EGID, FSGID, дополнительные группы, привилегии, маска доступа
Атрибуты безопасности файла	UID, GID, биты доступа, ACL, флаги файла
Решение о доступе	Сравнение идентификаторов FSUID, FSGID и дополнительных групп процесса с идентификаторами UID, GID, битов доступа и ACL файла. Анализ привилегий процесса.

В четвёртом модуле мы вернёмся к этой таблице и заполним её аналогично для SELinux.

‹ Предыдущий урок
Следующий урок ›

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026
Дизайн MAX Разработка TUNE-IT

Концепция пользователя

В первом модуле мы говорили, что GNU/Linux является многопользовательской операционной системой, в которой одновременно могут работать несколько пользователей. Для реализации этой концепции вводится понятие учетной записи пользователя (user account), или просто пользователя. С точки зрения DAC каждый файл (объект доступа) принадлежит определённому пользователю, и каждый процесс (субъект доступа) выполняется от имени определённого пользователя. Таким образом, концепция пользователя позволяет определить владельцев системных ресурсов, право доступа к этим ресурсам, и связать внутренние сущности системы с внешним миром.

Для примера выведем идентификаторы пользователя, затем создадим новый файл и выведем владельцев нового файла и файла /etc/passwd, затем посмотрим, от имени кого выполнялись эти команды (обычного пользователя sarah (home UID)

```
$ id
uid=1001(user1) gid=1001(user1) groups=1001(user1)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
$ touch file1
$ ls -l file1
-rw-rw-r--. 1 user1 user1 0 Feb  2 23:38 file1
$ ls -l /etc/passwd
-rw-r--r--. 1 root root 2748 Nov 06 09:00 /etc/passwd
$ ps -o user,uid
USER        UID
bash       1000
```

В UNIX конфигурационные файлы, как правило, представляют собой обычные текстовые файлы, что позволяет производить настройку системы с помощью обычного текстового редактора, и даже средствами командной строки.

Для управления пользователями используются следующие файлы: /etc/passwd, /etc/shadow, /etc/group, /etc/default/useradd, /etc/login.defs. Рассмотрим эти файлы.

Файл /etc/passwd

Список пользователей системы хранится в текстовом файле /etc/passwd. Каждая строчка этого файла описывает одного пользователя и представляет собой запись, состоящую из семи полей, разделённых двоеточием. В таблице представлены порядок и назначение полей.

Формат записей файла /etc/passwd:

Номер поля	Назначение
1.	Логическое (входное) имя пользователя. Имя может состоять из маленьких букв, цифр, символов подчёркивания или тире. Имя должно начинаться с маленькой буквы или символа подчёркивания и может заканчиваться символом доллара. Регулярное выражение для имени пользователя: `[a-z_][a-z0-9_-]*$?` Имя может быть длиной до 32 символов.
2.	Хэш пароля пользователя. Однако, если используется файл /etc/shadow, то хэш пароля хранится в нём.
3.	UID (user id) – идентификатор пользователя.
4.	GID (group id) – идентификатор основной группы пользователя.
5.	Комментарий, например, настоящее имя пользователя. Иногда это поле называют GECOS (or General Electric Comprehensive Operating System), так как в первых версиях UNIX в этом поле хранили идентификаторы для доступа к заданию печати, которые работали под управлением GECOS.
6.	Путь к домашнему директорию пользователя, в которую попадает пользователь после входа в систему и в переменную которой он является.
7.	Путь к программе, которая будет запускаться после входа пользователя в систему. Обычно это путь к интерпретатору команд bash.

Причём, что домашним директорием пользователя root - это /root, а для обычных пользователей домашние директории располагаются в директории /home.

Во время входа пользователя в систему процесс входа login становится процессом bash, в котором в качестве рабочей директории устанавливается домашняя директория пользователя.

В качестве примера работы с файлом /etc/passwd рассмотрим shell-скрипт, который выводит на экран информацию о пользователях, у которых в качестве оболочки стоит bash.

```
grep 'bash$' /etc/passwd |
while IFS=: read user passwd uid gid name homedir shell
do
    printf "%16s: %s\n" \
           User "$user" \
           Password "$passwd" \
           "User ID" "$uid" \
           "Group ID" "$gid" \
           Name "$name" \
           "Home directory" "$homedir" \
           Shell "$shell"
    read < /dev/tty
done
```

Для разбиения записей на отдельные поля используется тот факт, что поля разделяются символом ':'. Поэтому если переопределить переменную IFS (Internal Field Separator), то BASH будет читать отдельные поля, как отдельные слова, разделённые символом-разделителем.

Файл /etc/group

Пользователи объединяются в группы для более гибкого управления доступом к файлам. Пользователь должен входить как минимум в одну группу - такую группу называют основной (primary). Группа пользователь может входить в несколько дополнительных (supplementary) групп. Список групп хранится в текстовом файле /etc/group. Каждая строка этого файла описывает одну группу и представляет собой запись, состоящую из четырёх полей, разделённых двоеточием. В таблице представлены порядок и назначение полей.

Формат записей файла /etc/group.

Номер поля	Назначение
1.	Имя группы. Имя может состоять из маленьких букв, цифр, символов подчёркивания или тире. Имя должно начинаться с маленькой буквы или символа подчёркивания и может заканчиваться символом доллара. Регулярное выражение для имени группы следующее: `[a-z_][a-z0-9_-]*$?` Имя группы может быть длиной до 16 символов.
2.	Хэш пароля. Встречается редко.
3.	GID (group id) – идентификатор группы.
4.	Список имён пользователей, входящих в группу, через запятую.

Командный интерфейс

Командный интерфейс управления учётными записями пользователей и групп представлен следующими командами:

- Команды управления пользователями: useradd, userdel, usermod.
- Команды управления группами: groupadd, groupdel, groupmod.

Рассмотрим только наиболее часто используемые команды.

Для создания пользователя предназначена команда useradd. Синтаксис команды:

```
useradd [опции] имя_пользователя
```

С помощью опций можно явно задать каждую характеристику пользователя. Однако, если опции не заданы, берутся значения по умолчанию из файлов /etc/default/useradd и /etc/login.defs. Кроме того, после создания домашних директорий пользователя, в неё копируются содержимое директории /etc/skel/, в которую системный администратор может поместить файлы, которые по-умолчанию должны присутствовать у каждого пользователя.

Рассмотрим содержимое файла /etc/default/useradd.

```
$ cat /etc/default/useradd
GROUP=100
HOME=/home
INACTIVE=-1
EXPIRE=
SHELL=/bin/bash
SKEL=/etc/skel
CREATE_MAIL_SPOOL=yes
```

Эту же информацию можно получить с помощью опции -D.

```
$ useradd -D
GROUP=100
HOME=/home
INACTIVE=-1
EXPIRE=
SHELL=/bin/bash
SKEL=/etc/skel
CREATE_MAIL_SPOOL=yes
```

Приведём пример содержимого директории /etc/skel/.

```
$ ls -al /etc/skel/
total 12
drwxr-xr-x.  3 root root  4096 Apr 23  2020 .
drwxr-xr-x. 134 root root 12288 Oct  9 08:42 ..
-rw-r--r--.  1 root root   18 Jun  2  2020 .bash_logout
-rw-r--r--.  1 root root  141 Jun  2  2020 .bash_profile
-rw-r--r--.  1 root root  376 Jun  2  2020 .bashrc
```

Рассмотрим фрагмент файла /etc/login.defs.

```
$ cat /etc/login.defs
UMASK 022
PASS_MAX_DAYS 9768
PASS_MAX_DAYS 99999
PASS_MIN_DAYS 0
PASS_MIN_LEN 5
UID_MIN                  1000
UID_MAX                 60000
SYS_UID_MIN               201
SYS_UID_MAX               999
USERGROUPS_ENAB yes
CREATE_HOME     yes
```

Всё пространство идентификаторов пользователя (UID) можно разделить на следующие диапазоны:

0 – идентификатор пользователя root, который в UNIX считается полноправным администратором системы, также называемого суперпользователем.

1-999 - идентификаторы пользователей, от имени которых выполняются системные процессы.

1000-60000 - идентификаторы обычных пользователей.

Рассмотрим пример создания пользователя:

```
# useradd user1
$ grep user1 /etc/passwd
user1:x:1001:1001::/home/user1:/bin/bash
# grep user1 /etc/shadow
user1:!!:18163:0:99999:7:::
$ grep user1 /etc/group
user1:x:1001:
```

Видно, что команда useradd по-умолчанию создаёт записи для нового пользователя в трёх основных файлах. Для пользователя создано новая индивидуальная группа, в которую входит только данный пользователь. Пароль пользователю не установлен, поэтому его нужно установить явно командой passwd.

Для модификации пользователя используется команда usermod. Её опции в основном повторяют опции команды useradd. Синтаксис команды:

```
usermod [опции] имя_пользователя
```

Для добавления пользователя в дополнительные группы используется опция -G. Однако, если пользователь входит в несколько в ещё одну дополнительную группу, то не надо удалять из других дополнительных групп, то нужно использовать опцию -a. Рассмотрим пример:

```
$ grep user1 /etc/group
user1:x:1001:
# usermod -G users user1
$ grep user1 /etc/group
users:x:100:user1
user1:x:1001:
# usermod -a -G wheel user1
$ grep user1 /etc/group
wheel:x:10:user1
users:x:100:user1
user1:x:1001:
```

Иногда пользователю требуется возможность аутентификации по паролю, но не требуется работать в командной оболочке. Например, на почтовых серверах пользователи должны иметь учётные записи, чтобы подключаться из почтовых клиентов. Тогда в качестве входной оболочки можно установить программу /sbin/nologin.

```
# usermod -s /sbin/nologin user1
# grep user1 /etc/passwd
user1:x:1001:1001::/home/user1:/sbin/nologin
```

Если пользователь попытается войти в систему, программа /sbin/nologin просто закроет соединение.

Для удаления пользователя используется команда userdel. Синтаксис команды:

```
userdel [опции] имя_пользователя
```

При удалении пользователя возникают ряд вопросов, например, что делать с его файлами как в домашней директории, так и в других системных каталогах, нужно ли отдавать освободившийся UID новым пользователям, или нет и т.д. По умолчанию команда userdel только удаляет учётную запись пользователя, но не затрагивает его файлы. Для удаления домашней директории используется опция -r.

Рассмотрим пример, когда удаляется только учётная запись пользователя, но файлы пользователя остаются, поэтому новый пользователь получает доступ к файлам пользователя, который был удалён.

```
# useradd user1
$ id user1
uid=1001(user1) gid=1001(user1) groups=1001(user1)
$ ls -l /home/
drwx------. 3 user1  user1  4096 Oct  9 04:47 user1
# userdel user1
$ ls -l /home/
drwx------. 3 1001  1001  4096 Oct  9 04:47 user1
# useradd user2
$ tail -2 /etc/passwd
user2:x:1001:1001::/home/user2:/bin/bash
# id user1
uid=1001(user1) gid=1001(user1) groups=1001(user1)
$ ls -l /home/
drwx------. 3 user1  user1  4096 Oct  9 04:47 user1
# ls -l /home/
drwx------. 3 user2  user2  4096 Oct  9 04:48 user2
```

Чтобы предотвратить подобные ситуации рекомендуется не удалять пользователя, а блокировать его учётную запись. Дополнительно можно придерживаться следующих правил:

При создании нового пользователя явно указывать его UID. Например:

```
# useradd -u 1002 user2
```

При удалении пользователя, перемещать все его файлы в специально выделенную директорию, а его домашнюю директорию удалять. Например:

```
# find / -user user1 -exec mv {} /mybk/mjpeney; 2>/dev/null
# userdel -r user1
```

Чтобы найти файлы, которые не принадлежат никакому пользователю, можно выполнить следующую команду:

```
# find / -nouser -o -nogroup 2>/dev/null
```


SkillFactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту. © 2016

Хэш-функция

Для вычисления хэша пароля должна использоваться криптографическая хэш-функция (hash-function). Определение и свойства криптографической хэш-функции приведены в отечественном стандарте ГОСТ Р 34.11-2012:

> Хэш-функция – это функция, отображающая строки бит в строки бит фиксированной длины.

Криптографическая хэш-функция должна обладать следующими свойствами:

- Стойкостью к вычислению прообраза, т. е. по данному значению функции сложно вычислить исходные данные, отображаемые в это значение;
- Стойкостью к вычислению второго прообраза, т. е. для заданных исходных данных сложно вычислить другие исходные данные, отображаемые в то же значение функции;
- Стойкостью к поиску коллизий, т.е. сложно вычислить какую-либо пару исходных данных, отображаемых в одно и то же значение.

Файл /etc/shadow

Для управления паролями пользователей используется текстовый файл /etc/shadow. Этот файл предназначен пользователю root и не доступен на чтение и запись обычным пользователям. Каждая строка этого файла соответствует одному пользователю системы и представляет собой запись, состоящую из девяти полей, разделённых двоеточием. В таблице представлено назначение полей и назначение полей файла /etc/shadow.

Основная функция файла /etc/shadow – хранить хэши паролей пользователей системы.

Рассмотрим пример содержимого второго поля:

```
:$6$ZjdGqIxH$lhjqUdkS7aR247kobwEIyaRNUYScdvaLzM2uqYmnYpVxnvaOOtuaeEOTyeeEdBdkLCBkBUY9?P
rrcRvA7NyV//0Rw.:
```

В поле, содержащем хэш пароля, можно выделить три части, разделённые знаком $: алгоритм хэширования (цифра 6 означает SHA-512), так называемую соль – случайное число, которое добавляется к паролю, чтобы вычислить хэш (таким образом даже если у двух пользователей будут одинаковые пароли, их хэши будут разными), сам хэш.

Формат записей файла /etc/shadow.

Номер поля	Назначение
1.	Логическое (входное) имя существующего пользователя
2.	Хэш пароля. Если поле пустое, то аутентификация для пользователя не требуется. Если содержимое поля начинается с восклицательного знака, то считается, что пользователь заблокирован, т.ф. он не сможет войти в систему с помощью пароля. Однако пользователь может войти с помощью других средств (например, ssh).
3.	Дата последнего изменения пароля, в количестве дней, прошедших с 1 января 1970 года. Значение 0 означает, что пользователь должен изменить свой пароль при следующем входе в систему. Пустое поле означает, что пароль никогда не устаревает.
4.	Минимальный срок действия пароля - количество дней, которые должны пройти, чтобы пользователь смог снова изменить свой пароль. Пустое поле или значение 0 означают, что минимальный срок не установлен.
5.	Максимальный срок действия пароля – количество дней, по прошествии которых пользователь должен изменить свой пароль. По прошествии этого количества дней у пользователя будет произведено изменение пароля при следующем входе в систему. Пустое поле означает, что не установлены максимальный срок, период предупреждения, период бездействия. Если максимальный срок действия пароля меньше минимального срока действия пароля, пользователь не сможет изменить свой пароль.
6.	Период предупреждения - количество дней до истечения максимального срока действия пароля, в течение которого пользователю выводится предупреждение о необходимости поменять пароль. Пустое поле или значение 0 означают, что период предупреждения не установлен.
7.	Период бездействия пароля - количество дней после истечения максимального срока действия пароля, в течение которого учётная запись продолжает быть активной и пользователь может войти в систему, чтобы изменить свой пароль. После истечения периода бездействия, вход пользователя в систему заблокируется, учётная запись становится неактивной. Пользователь должен обратиться к администратору. Пустое поле означает, что период бездействия не установлен.
8.	Срок действия учетной записи в количестве дней, прошедших с 1 января 1970 года. Пустое поле означает, что срок действия учетной записи никогда не истечет. Значение 0 использовать не следует, так как оно может интерпретироваться как учетная запись без срока действия, или с истечением срока действия 1 января 1970 года.
9.	Зарезервировано для будущего использования.

Соотношение временных характеристик пароля показано на рисунке.

linux-m2-3-1

Обратите внимание, что срок действия учетной записи и срок действия пароля - это две разных настройки. В случае истечения срока действия учетной записи пользователю полностью запрещен вход в систему. В случае истечения срока действия пароля, пользователю разрешено входить в систему, используя свой пароль.

Командный интерфейс

Для изменения временных атрибутов пароля пользователя используется команда chage. Синтаксис команды:

```
chage [опции] имя_пользователя
```

Опция -l выводит на экран информацию об учётной записи пользователя. Опция -d устанавливает дату последнего изменения пароля.

Для изменения пароля пользователя, а также временных атрибутов пароля, используется команда passwd. Синтаксис команды:

```
passwd [опции] имя_пользователя
```

Опция -d полностью удаляет пароль пользователя.

Чтобы заставить пользователя сменить пароль при следующем входе в систему можно выполнить команду passwd -e или chage -d 0. В результате, в поле даты последнего изменения пароля будет записан 0.

Форсирование изменения пароля

Рассмотрим пример. Предположим, что сегодня 2 февраля и в системе создан пользователь, который будет работать над проектом в течение 6 месяцев (180 дней). Команда date позволяет вычислить дату, которая наступит через определённое количество дней после текущего дня. Срок действия учетной записи для нового пользователя можно ограничить 180 днями. Пароль требуется менять не реже, чем каждые три месяца (90 дней). Минимальный срок действия пароля не установлен. За пять дней до обязательной смены пароля, пользователю начнут выдаваться предупреждения, что пароль необходимо поменять. Если пароль не поменяется, то в течение 10 дней он сможет войти в систему со старым паролем и установить новый пароль. Команда chage -l выведет текущие настройки пароля пользователя.

```
# date
Mon Feb  2 11:46:27 PM MSK 2026
# date -d +180days +%Y-%m-%d
2026-08-01
# chage -E $(date -d +180days +%Y-%m-%d) user1
# chage -m 0 -M 90 -W 5 -I 10 user1
# chage -l user1
Last password change                                    : Feb 02, 2026
Password expires                                        : May 03, 2026
Password inactive                                       : May 13, 2026
Account expires                                         : Jul 31, 2026
Minimum number of days between password change          : 0
Maximum number of days between password change          : 90
Number of days of warning before password expires       : 5
# grep user1 /etc/shadow
user1:$y$j9T$1LGWC.s7YGJTGFzgraIVi5$.vFvnHZ1n3D2Rxzst39s/bP33ITn1./n3hRoMI/Qe4:20456:0:90
:5:10:20651:
```

Видно, что в записи для пользователя в файле /etc/shadow установлены соответствующие поля. Например, число 20666 означает количество дней от 1 января 1970 года до 31 июля 2026 года. Если нужно принудить пользователя сменить свой пароль при следующем входе в систему, то можно воспользоваться командой chage -d 0.

```
# chage -d 0 user1
# chage -l user1
Last password change                                    : password must be changed
Password expires                                        : password must be changed
Password inactive                                       : password must be changed
Account expires                                         : Jul 31, 2026
Minimum number of days between password change          : 0
Maximum number of days between password change          : 90
Number of days of warning before password expires       : 5
# grep user1 /etc/shadow
user1:$y$j9T$1LGWC.s7YGJTGFzgraIVi5$.vFvnHZ1n3D2Rxzst39s/bP33ITn1./n3hRoMI/Qe4:0:0:90:5:1
0:20651:
```

Видно, что в записи для пользователя в файле /etc/shadow в поле даты последнего изменения пароля установлен 0.

Блокировка пользователя

Чтобы заблокировать вход пользователя в систему по паролю, можно использовать команды usermod с опцией -L или passwd с опцией -l.

После создания учётной записи пользователя, в поле, где хранится хэш пароля, записан восклицательный знак, что не позволит пользователю войти в систему без пароля. При блокировке пользователя хэш пароля не изменяется, но в начало записывается восклицательный знак. При разблокировании восклицательный знак просто удаляется. Рассмотрим пример:

```
# useradd user3
# grep user3 /etc/shadow
user3:!!:19909:0:90:7:::
# passwd user3
ввод пароля
# grep user3 /etc/shadow
user3:!$6$4wqZJ7v.wQnQ8fSpbLkAPuLi7vEQvYXKC/2ycmaaVkJHFv2BeKLzzpatY2Hdj8X2ot1q1/GeOOIMq
sR7OYoVyPWc1tdJmLQ=0:19909:0:90:7:::
# passwd -l user3
# grep user3 /etc/shadow
user3:!!$6$4wqZJ7v.wQnQ8fSpbLkAPuLi7vEQvYXKC/2ycmaaVkJHFv2BeKLzzpatY2Hdj8X2ot1q1/GeOOIMq
sR7OYoVyPWc1tdJmLQ=0:19909:0:90:7:::
# passwd -u user3
# grep user3 /etc/shadow
user3:$6$4wqZJ7v.wQnQ8fSpbLkAPuLi7vEQvYXKC/2ycmaaVkJHFv2BeKLzzpatY2Hdj8X2ot1q1/GeOOIMq
sR7OYoVyPWc1tdJmLQ=0:19909:0:90:7:::
# passwd -l user3
# grep user3 /etc/shadow
user3:!!$6$4wqZJ7v.wQnQ8fSpbLkAPuLi7vEQvYXKC/2ycmaaVkJHFv2BeKLzzpatY2Hdj8X2ot1q1/GeOOIMq
sR7OYoVyPWc1tdJmLQ=0:19909:0:90:7:::
# grep user3 /etc/shadow
user3:$6$4wqZJ7v.wQnQ8fSpbLkAPuLi7vEQvYXKC/2ycmaaVkJHFv2BeKLzzpatY2Hdj8X2ot1q1/GeOOIMq
sR7OYoVyPWc1tdJmLQ=0:19909:0:90:7:::
```

Как видно, usermod вставляет один восклицательный знак, а passwd два. Возможности команд usermod, passwd и chage пересекаются, что позволяет системному администратору выбрать свой стиль общения с системой.

Работа с учётной записью пользователя вручную

Для создания, модификации и удаления учётных записей пользователей система предоставляет как командный интерфейс, так и набор библиотечных функций. Но так как информация о пользователях хранится в простых текстовых файлах, управлять учётными записями можно и вручную.

Рассмотрим последовательность шагов для создания учётной записи пользователя вручную:

1. Выбрать отличительные характеристики пользователя: логическое имя, идентификатор пользователя UID, идентификатор группы пользователя GID.
2. Создать запись в /etc/passwd.
3. Создать запись в /etc/group.
4. Создать домашнюю директорию пользователя по шаблону: /home/имя_пользователя.
5. Скопировать в домашнюю директорию файлы, которые должны присутствовать у пользователя и, в том числе, выполняться при входе пользователя в систему.
6. Изменить владельца и группу домашней директории и всего её содержимого на вновь созданного пользователя и его группу. При необходимости изменить права к домашней директории.
7. Создать запись в /etc/shadow в соответствии с политикой безопасности организации по управлению паролями (например, как часто надо пользователю менять свой пароль).
8. Установить пользователю пароль.

‹ Предыдущий урок | Следующий урок ›

SkillFactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн - MAX | Разработка TUNE-IT

5/13 Урок 4. Владение файлом

текущий урок

С точки зрения DAC каждый файл принадлежит одному определённому пользователю и одной определённой группе пользователей. Поэтому в иноде файла хранятся два идентификатора: идентификатор владельца (UID) и идентификатор группы-владельца (GID).

При создании нового файла (например, с помощью вызовов open() или mkdir()) его владелец (UID) устанавливается равным FSUID создающего процесса.

Группа-владелец (GID) нового файла зависит от типа файловой системы, опций монтирования и наличия бита set-group-ID у родительской директории.

Если файловая система ext4 смонтирована с опцией -o grpid, то GID нового файла устанавливается равным GID родительской директории.

По умолчанию файловая система ext4 монтируется с опцией -o nogrpid. В этом случае GID нового файла устанавливается равным FSGID создающего процесса, но если у родительской директории установлен бит set-group-ID, то GID нового файла устанавливается равным GID родительской директории.

Только процесс с привилегией CAP_CHOWN может изменить владельца файла, а также изменить группу-владельца на произвольную другую группу.

Владелец файла не может изменить владельца, т.е. передать свой файл другому пользователю, а может изменить только группу-владельца и только на ту группу, в состав которой он входит сам.

Если у исполняемого файла меняются владелец или группа-владелец, то биты set-user-ID и set-group-ID сбрасываются. Это повышает безопасность и гарантирует, что обычный пользователь не сможет на свой собственный исполняемый файл установить биты set-user-ID или set-group-ID, а затем передать данный файл пользователю или группе root и, тем самым, повысить свои привилегии.

Если у исполняемого файла меняются владелец или группа-владелец, то все наборы привилегий очищаются.

Для изменения владельца и/или группы-владельца файла предназначена команда chown.

```
chown [опции] [владелец][:[группа]] список_файлов
chown [опции] --reference=файл-образец список_файлов
```

В одной команде можно указать новых владельца и группу-владельца через двоеточие, а можно указать только владельца или только группу-владельца. Владелец и группа-владельца могут быть заданы как своими именами, так и числовыми значениями. Различные сочетания и результаты выполнения команды представлены в таблице.

Формат команды chown.

Формат	Результат
владелец	Меняется владелец каждого файла. Группа-владелец не меняется.
владелец:группа	Меняется владелец и группа-владелец каждого файла.
владелец:	Меняется владелец каждого файла. Группа-владелец каждого файла меняется на основную группу нового владельца.
:группа	Владелец не меняется. Группа-владелец каждого файла меняется на новую группу.
: или аргумент опущен	Ничего не меняется.

С опцией --reference меняются владелец и группа-владелец на владельца и группу файла-образца. С опцией -R (в длинном виде --recursive) команда выполняется рекурсивно.

Для изменения группы-владельца файла предназначена отдельная команда chgrp.

```
chgrp [опции] группа список_файлов
chgrp [опции] --reference=файл-образец список_файлов
```

У каждого файла меняется группа-владелец на новое значение. С опцией -R (в длинном виде --recursive) команда выполняется рекурсивно. С опцией --reference меняется группа-владелец на группу файла-образца.

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX Разработка TUNE-IT

Биты прав доступа файла

С точки зрения DAC с помощью битов прав доступа можно разграничить доступ к файлу для трёх категорий пользователей:

- владелец (owner или user) файла;
- группа владельца (group) файла (т.е. пользователи, входящие в эту группу);
- все остальные пользователи (other).

Для каждой категории пользователей задаются три права доступа к файлу:

- чтение (read);
- запись (write);
- исполнение (execute) для обычных файлов и поиск (search) для директорий.

Кроме того, файлу назначаются три специальных права:

- бит set-user-ID;
- бит set-group-ID;
- sticky-бит.

Такие обозначения, для каждого файла формируется вектор из 12 бит доступа, которые хранятся в inodeдане. О битах поэт, inode иногда файла.

Если значение бита равно 1, то говорят, что бит установлен (set). В примере не предоставлено для обычного файла и директории некоторые права интерпретируются по-разному. В таблице показаны название и назначение битов для различных файлов.

Биты прав доступа

Биты	Обычный файл	Директория
r (read)	Читать из файла	Выводить содержимое директории в коротком формате (только имена файлов)
w (write)	Писать в файл	Создавать, удалять, переименовать и перемещать файлы в директории. Также требуется право на поиск.
x (exec)	Запускать файл на исполнение (если является программой или сценарием)	Читать метаданные файлов в директории (владельца, группу, права доступа, время редактирования и размер файлов). Также требуется право на поиск в директории (cd), выводить содержимое директории в длинном формате (ls -l).
s (set-user-ID)	Установление эффективного идентификатора пользователя EUID процесса при исполнении (если установлен атрибут владельца файла) x0.	Не используется
s (set-group-ID)	Установление эффективного идентификатора группы EGID процесса при исполнении (если установлен идентификатор группы файла) x0.	Установление идентификатор группы GID для всплыв, создаваемых в директории. Также требуется право на запись.
t (sticky)	Не используется	Удалять или переименовывать файлы в директории может только владелец директории, или владелец файла, или суперпользователь (root). Также требуется право на запись.

Чтобы доступны двоичный файл достаточно только права на исполнение, а для запуска сценариев необходимо наличие двух прав на чтение и на исполнение.

При доступе к файлу разрешения на поиск необходимы для всех директорий, начиная с корневой директории.

Право доступа на чтение директории позволяет только вывести список имен файлов. Чтобы получить доступ к самим файлам (т. к. содержимому их inodeов) нужно право на поиск в директории. И наоборот, если только право на поиск, но отсутствуют права на чтение, можно получить доступ к файлу в директории, если известно его имя. Одним из вариантов создания данной директории. Такой способ используется для управления доступом к содержимому общедоступных директорий.

Установка бита set-group-ID на директорию используется для организации директорий, предназначенных для работы над совместным проектом несколько пользователей, включённых в одну общую группу.

Установка sticky-бита используется для организации общедоступных директорий, таких как /tmp. В таких директориях стандартные пользователи имеют разрешение и удалять собственные файлы, но не имеют удалять файлы, принадлежащие другим пользователям. Подробнее о sticky-бите вы узнаете в курсе «Операционные приложения от азов» (ссылка появится позже).

Таким образом, простая смена битов прав доступа позволяет контролировать следующие виды доступа процесса к файлу:

- создание, переименование, перемещение и удаление файла;
- чтение метаданных файла (содержимого inode);
- чтение содержимого файла;
- изменение содержимого файла;
- запуск файла на исполнение.

Рассмотрим всевозможные комбинации битов доступа в поле r_mode inode файла, показанный в виде таблицы. Поле r_mode имеет размер 16 бит. Четыре левых разряда не используются для хранения этих битов. Остальные 12 бит имеют размер 1 бит. Для удобства, вместо 12 бит удобнее их читать по три бита, каждый отвечает за свою группу пользователей. Для чтения и записи битов доступа к файлу используется системный вызов chmod (сокращение от англ. change mode).

	suid	sgid	sticky	r	w	x	r	w	x	r	w	x	Ос-таль-ные
	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	1	0	0	1	0	0	1	0	0	1	1
	0	1	0	0	1	0	0	1	0	0	1	0	2
	0	1	1	0	1	1	0	1	1	0	1	1	3
	1	0	0	1	0	0	1	0	0	1	0	0	4
	1	0	1	1	0	1	1	0	1	1	0	1	5
	1	1	0	1	1	0	1	1	0	1	1	0	6
	1	1	1	1	1	1	1	1	1	1	1	1	7
	Тип файла	Специальные		Владелец (UID)		Группа (GID)		Остальные (other)					

Так как здесь битов достаточно, чтобы закодировать восьмеричное число, то во многих языках они для использования восьмеричных констант.

Алгоритм проверки прав доступа

Ядро проверяет права доступа к файлу каждый раз при указании его имени в системном вызове. Если путь к файлу содержит директорию, то помимо проверки прав доступа к самому файлу, ядро проверяет также права доступа для каждой директории в полном имени. Проверки прав доступа выполняются благодаря сравнению эффективных идентификаторов пользователя и группы, о таком дополнительном сухом процессе.

Алгоритм проверки прав доступа:

1. Если процесс является привилегированным, то предоставляется полный доступ.
2. Если EUID для процесса совпадает с идентификатором владельца файла UID, то предоставляется доступ в соответствии с правами доступа для владельца файла.
3. Если EGID или идентификатор любой дополнительной группы для процесса совпадают с идентификатором группы владельца файла GID, то доступ предоставляется в соответствии с правами доступа для группы.
4. В остальных случаях доступ к файлу предоставляется в соответствии с правами доступа для всех остальных пользователей.

Проверки прав доступа для владельца, группы и остальных выполняются по порядку и прекращаются, если только обнаруживается подходящий пункт. Это может привести к несоответствию последствиях. если, например, право доступа для группы превышает право владельца. То владелец будет фактически иметь меньше права доступа к файлу, чем участники группы.

Если у файла установлен ACL, то применяется изменения версия алгоритма.

Для создания привилегированного процесса используются две привилегии: CAP_DAC_READ_SEARCH и CAP_DAC_OVERRIDE.

Командный интерфейс

Вывод прав доступа

Права доступа к файлу и выводе команды ls показаны в виде строки из 10 символов (-rwxrwxrwx). Первый символ показывает тип файла (- – обычный файл, d – директория). Следующие три символа (rwx) показывают права доступа для владельца, следующие три символа (rwx) – для группы, последние три символа – для остальных пользователей. Если вместо символа выводится прочерк (-), то это право не предоставлено, то вместо символа выводится знак -.

Право доступа к принадлежность файла можно просмотреть с помощью команд ls -l или stat. Например:

```
$ ls -l /etc/passwd
-rw-r--r-- 1 root root 2699 Dec 30 04:25 /etc/passwd
$ stat /etc/passwd
  File: /etc/passwd
  Size: 2699      Blocks: 8          IO Block: 4096   regular file
Device: 801h/2049d  Inode: 536243    Links: 1
Access: (0644/-rw-r--r--)  Uid: (    0/    root)   Gid: (    0/    root)
Access: 2021-12-30 04:25:22.797999952 +0300
Modify: 2021-12-30 04:25:22.796999952 +0300
Change: 2021-12-30 04:25:22.797999952 +0300
 Birth: -
```

Изменение битов прав доступа к файлу

Для изменения права доступа к файлу используется chmod. Чтобы процесс обладал привилегией CAP_FOWNER, или чтобы его FSUID совпадал с идентификатором владельца файла UID.

Для повышения безопасности вместо символьных констант.

Если процесс не обладает привилегией CAP_FSETID, и группа файла не совпадает с FSUID процесса или с одной из его дополнительных групп. Бит set-group-ID будет сброшен, при этом это не считается ошибкой.

Если процесс не обладает привилегией CAP_FSETID, и он пытается писать в файл, у которого установлены биты set-user-ID и/или set-group-ID, то эти биты будут сброшены.

Для изменения битов прав доступа к файлу предназначена команда chmod.

```
chmod [опции] права[,права] имена_файлов
chmod [опции] режим_доступа имена_файлов
chmod [опции] --reference=файл имена_файлов
```

Команда chmod может применять права, которые нужно установить на файл, в трёх видах: в виде символического, в виде восьмеричного числа, и через имена файл-объекта.

Рассмотрим пример. Создадим новую директорию и один файл в ней. Затем будем менять права доступа к директории и проверять возможность доступа к файлу при различных комбинациях разрешений к ней файлов.

Обратите внимание на то, что команда ls может быть алиасом на команду ls. В Linux-системах по умолчанию команда ls алиасом на команду ls, которая выводит всю ей имеется новое значок: можно, указав первую команда chmod. ?

```
$ mkdir dir1
$ ls -ld dir1
drwxrwxr-x. 2 user1 user1 4096 Feb  1 04:11 dir1
$ touch dir1/f1
$ ls -l dir1
total 0
-rw-rw-r-- 1 user1 user1 0 Feb  1 04:11 f1
```

Установим только для владельца директории только право на поиск.

```
$ chmod 100 dir1
$ ls -ld dir1
d--x------ 2 user1 user1 4096 Feb  1 04:11 dir1
$ ls -l dir1
ls: cannot open directory 'dir1': Permission denied
$ cat dir1/f1
cat: cannot open directory 'dir1': Permission denied
$ ls dir1/f1
dir1/f1
$ ls -l dir1/f1
ls: cannot open directory 'dir1': Permission denied
$ touch dir1/f1
touch: cannot touch 'dir1/f1': Permission denied
$ rm dir1/f1
rm: cannot remove 'dir1/f1': Permission denied
$ mv dir1/f1 dir1/f2
mv: cannot move 'dir1/f1' to 'dir1/f2': Permission denied
```

Видно, что можно получить доступ к самому файлу, но имени либо директории в правах на записи. Установим только для владельца директории только право на запись.

```
$ chmod 200 dir1
$ ls -ld dir1
d-w------- 2 user1 user1 4096 Feb  1 04:11 dir1
$ touch dir1/f1
$ touch dir1/f2
$ ls -l dir1
ls: cannot open directory 'dir1': Permission denied
$ rm dir1/f1
rm: cannot remove 'dir1/f1': Permission denied
$ chmod 000 dir1
$ ls -ld dir1
d--------- 2 user1 user1 4096 Feb  1 04:11 dir1
$ touch dir1/f1
touch: cannot touch 'dir1/f1': Permission denied
$ ls dir1/f1
ls: cannot access 'dir1/f1': Permission denied
$ rm dir1/f1
rm: cannot remove 'dir1/f1': Permission denied
```

Видно, что можно создавать или удалять файлы в директории, необходимо иметь разрешение и на поиск и на запись для данной директории.

Обратите внимание: не обязательно иметь какие-либо права доступа к самому файлу, чтобы удалить его!

Установим только для владельца директории только право на чтение.

```
$ chmod 400 dir1
$ ls -ld dir1
dr-------- 2 user1 user1 4096 Feb  1 04:17 dir1
$ touch dir1/f1
touch: cannot touch 'dir1/f1': Permission denied
$ ls dir1
ls: cannot access 'dir1/f1': Permission denied
total 0
? ? ? ? ? ? ? ? ? f1
```

Видно, что в этом случае можно только вывести имя файла, но нельзя получить доступ к нему (недоступны его inode).

Для поиска файлов используется команда find. Если нужно найти файлы с определенными правами доступа, нужно использовать выражение -perm. Существует три формы записи выражения -perm.

```
-perm [права]
```

Биты прав доступа файла точно совпадают с аргументом права.

```
-perm -[права]
```

Все из заданных прав доступа файла должны присутствовать у файла.

```
-perm /[права]
```

Любой из заданных битов прав доступа должен присутствовать у файла.

Права можно задавать в восьмеричном или символьном виде. Символьный вид по синтаксису аналогичен chmod.

Например, следующая команда найдёт все файлы в директории с установленным битом set-user-ID и скорректирует информацию о них в файле /root/suid.txt и директив "режим_доступа владелец:путь_к_файлу".

```
# find / -perm -4000 -fprintf /root/suid.txt '%#m %u %p\n'
# cat /root/suid.txt
4755 root /usr/bin/chage
4755 root /usr/bin/chage
4777 root /usr/bin/umount
4777 root /usr/bin/umount
...
```

Найти файлы в текущей директории, у которых установлены права на чтение и запись для владельца и группы-владельца, а остальные пользователи могут только читать.

```
$ find . -perm 664
```

Пользовательская маска процесса

Для новых файлов ядро используют права, указанные в вызовах (в том системного вызова open). Для новых директорий это права устанавливается в соответствии с аргументом права команды mkdir. Однако указанные параметры изменяются с помощью пользовательской маски процесса, которая является атрибутом процесса. Она задаёт права доступа, которые будут сброшены при создании процессом (обладающим данной маской) новых файлов или директорий.

Обычно процесс наследует атрибуты umask, которые он наследует от своей родительской оболочки. Поэтому пользователь может управлять данным атрибутом в программах, выполняемых в оболочке, используя встроенную в оболочку команду umask.

Маска umask используется в большинстве оболочек по умолчанию устанавливает для атрибуты права восьмеричное значение 022 (---w--w-). Это право на запись должно быть всегда отключено для группы и для остальных пользователей.

Следовательно, если учесть, что обычно группит пользователей создаёт приватную группу (только по есть членом и запись разрешены для всех пользователей), то новые файлы, создаваемые другими другие не членами этой группы владельца, а для группы и остальных будут доступны только с правом на чтение (команда ls -l отобразит это как rw-r--r--).

Аналогично для директорий, если умасть, что аргументы mode системного вызова mkdir() обычно устанавливаю значение 0777 (rwxrwxrwx), то новые директории предоставлены всем пользователей, новые директории создаются с предоставлением всем доступа владельцу, группам и остальным пользователям предоставляется только права доступа на чтение и выполнение (rwxr-xr-x).

Увеличивается использования в большинстве обычно по умолчанию устанавливают без одновременно значение 022 (---w--w-). Пользователь второй раз вызывать umask(), чтобы восстановить предыдущее (всевозможное) значение маски. Так как такой способ не очень удобен, в большинстве оболочек предусмотрена команда с многопотоковыми программами.

Маску процесса можно также увидеть в поле umask файла /proc/self/status. Также обратим, процесс может изменить текущее значение своей маски, не изменяя ее значения. Например:

```
$ cat /proc/self/status | grep Umask
Umask: 0002
```

В BASH имеется встроенная команда umask.

```
umask [-p] [-S] [режим]
```

Если режим начинается с цифры, то он интерпретируется как восьмеричное число. В противном случае он интерпретируется как символьная маска. Символьная маска аналогична той, которая используется в команде chmod. Если задан режим, то маска устанавливается в соответствии с ним. Опция -S выводит маску в символьном виде, противной для использования в скриптах.

Политика конфиденциальности Пользовательское соглашение Обратиться к поддержку
Функциональные характеристики ПО Руководство пользователя Стоимость ПО

SkillFactory, Школа онлайн-профессий
ул. Льва Толстого, 16, корп. 1, Москва, Российская Федерация, 119021
Все права защищены. Копирование, распространение и использование запрещено.

Сделано MAX PROJECTS by TUNE IT

7/13 Урок 6. Флаги файла

текущий урок

Флаги файла (инода) позволяют связать с инодом дополнительные атрибуты, которые влияют на разные аспекты поведения файла, в т.ч. и на управление доступом к файлу.

Имена флагов и их числовые значения определяются в заголовочном файле ядра fs/ext4/ext4.h. Рассмотрим для примера несколько флагов, представленных в таблице.

Некоторые флаги файла.

Имя флага	Опция команды chattr	Описание
EXT4_IMMUTABLE_FL	i	Файл защищён от изменений. Файл нельзя удалить или переименовать. Нельзя сделать новую ссылку на файл. Установить данный флаг могут только процессы с привилегией CAP_LINUX_IMMUTABLE. Когда флаг установлен, даже привилегированный процесс не может изменить содержимое файла или его метаданные.
EXT4_APPEND_FL	a	Файл можно открыть только для записи в режиме добавления в конец файла (должен быть установлен флаг O_APPEND). Этот флаг можно использовать, например, для файла журнала. Установить данный флаг могут только процессы с привилегией CAP_LINUX_IMMUTABLE.
EXT4_NOATIME_FL	A	У файла не обновляется время последнего доступа (поле atime инода). Это позволяет избежать обновления инода при каждом доступе к файлу и таким образом повышает эффективность ввода-вывода.
EXT4_EXTENTS_FL	e	Файл использует экстенты для хранения данных на диске.

В основном, если флаги файла установлены для директории, их автоматически наследуют новые файлы и директории, создаваемые в данной директории.

Флаги может поменять либо владелец файла, либо пользователь root. В третьем модуле мы узнаем, что для изменения флагов инода необходимо, чтобы FSUID процесса соответствовал идентификатору владельца файла UID, либо чтобы процесс обладал привилегией CAP_FOWNER.

Для изменения флагов инода предназначена команда chattr.

```
chattr [опции] [режим] список_файлов
```

Формат режима следующий: +-=[aAcCdDeFijPsStTu]. Оператор '+' добавляет указанные флаги к существующим; оператор '-' удаляет указанные флаги; оператор '=' устанавливает указанные флаги в качестве единственных флагов инода.

Для просмотра флагов файла предназначена команда lsattr.

```
lsattr [опции] список_файлов
```

Рассмотрим пример с файлом, который нельзя изменить, в т.ч. удалить. Обратите внимание, что установить флаг “immutable” можно только в режиме суперпользователя (с помощью команды sudo).

```
$ touch f1
$ touch f2
$ ls -l
-rw-rw-r--. 1 user1 user1 0 Feb  3 03:02 f1
-rw-rw-r--. 1 user1 user1 0 Feb  3 03:02 f2
$ sudo chattr +i f1
$ lsattr
----i---------e----- ./f1
--------------e----- ./f2
$ rm f*
rm: cannot remove 'f1': Operation not permitted
$ ls -l
-rw-rw-r--. 1 user1 user1 0 Feb  3 03:02 f1
$ sudo rm f1
rm: cannot remove 'f1': Operation not permitted
```

Видно, что после установки флага +i, файл не может удалить не только владелец файла, но и суперпользователь root (с помощью команды sudo). Для удаления такого файла нужно сначала снять флаг immutable.

Рассмотрим ещё один пример с файлом, у которого установлен флаг “append only”, и в который можно только добавлять данные.

```
# echo abc > f_append_only
# echo 123456 > f_append_only
# ls -l f_append_only
-rw-r--r--. 1 root root 7 Feb  3 03:02 f_append_only
# cat f_append_only
123456
# chattr +a f_append_only
# lsattr f_append_only
-----a--------e----- f_append_only
# echo abc > f_append_only
bash: f_append_only: Operation not permitted
# echo abc >> f_append_only
# cat f_append_only
123456
abc
```

Видно, что после установки флага +a, информацию в файл можно только добавлять. В BASH для этого нужно воспользоваться перенаправлением вывода '>>'.

‹ Предыдущий урок Следующий урок ›

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026
Дизайн MAX Разработка TUNE-IT

8/13 Урок 7. Расширенные атрибуты файла

текущий урок

В прошлом модуле мы рассмотрели атрибуты файла, которые хранятся в иноде. Так как инод - это структура данных с фиксированным размером, в нём невозможно хранить метаданные файла с произвольным размером. Поэтому был разработан механизм расширенных атрибутов (extended attributes, EA). Идея заключается в выделении для файла дополнительного блока, в котором сохраняются произвольные данные о файле, а в иноде прописывается ссылка на этот блок. Нужно обратить внимание, что расширенные атрибуты реализуются на уровне файловой системы, т.е. ими можно воспользоваться, если файлы хранятся в файловой системе, которая поддерживает механизм расширенных атрибутов.

> Расширенные атрибуты - это пары «имя-значение», которые можно связать с инодом файла. Они предназначены для расширения стандартных атрибутов дополнительными произвольными метаданными.

Расширенные атрибуты используются для хранения ACL, контекста безопасности SELinux и привилегий файла. Однако, концепция расширенных атрибутов является настолько общей, что позволяет задействовать их и для других целей.

Имена расширенных атрибутов представлены в виде пары namespace.name, где namespace предназначен для разграничения расширенных атрибутов по функционально различным пространствам имён, а компонент name уникальным образом идентифицирует расширенный атрибут внутри данного пространства.

Существует четыре пространства имён:

- user
- trusted
- system
- security

Расширенными атрибутами пространства user могут управлять непривилегированные процессы при определенных условиях: для извлечения значения данного атрибута необходимо иметь разрешение на чтение файла; для изменения значения данного атрибута необходимо иметь разрешение на запись. Расширенные атрибуты пространства user можно назначать только файлам и директориям. При этом, непривилегированному процессу не разрешено назначать расширенный атрибут user для директории, владельцем которой является другой пользователь, если для данной директории установлен sticky-бит.

В данном курсе мы работаем с файловой системой ext4, в которой реализованы расширенные атрибуты. Для включения механизма расширенных атрибутов необходимо смонтировать файловую систему с ключом user_xattr:

```
$ mount -o user_xattr [устройство] [точка_монтирования]
```

Расширенными атрибутами пространства trusted могут управлять процессы, обладающие привилегией CAP_SYS_ADMIN.

Расширенные атрибуты пространства system используются ядром для хранения списка контроля доступа (ACL) файла.

Расширенные атрибуты пространства security используются ядром для хранения контекстов безопасности SELinux, а также привилегий исполняемых файлов.

Для установки расширенных атрибутов файла используется команда setfattr.

```
setfattr [опции] -n имя [-v значение] список_файлов
setfattr [опции] -x имя список_файлов
```

Опция -n устанавливает значение атрибута, опция -x полностью удаляет атрибут. Атрибут может быть не определён; значение расширенного атрибута может быть пустой строкой.

Для просмотра расширенных атрибутов файла используется команда getfattr.

```
getfattr [опции] -n имя список_файлов
getfattr [опции] -d [-m шаблон] список_файлов
```

Опция -n выводит значение атрибута по его имени, опция -d (--dump) выводит значение атрибутов, имена которых удовлетворяют шаблону. Опция -m задаёт шаблон в виде регулярного выражения. Шаблон по умолчанию соответствует всем атрибутам в пространстве user. Шаблон "-" включает все атрибуты (это похоже на метасимвол ‘*’ в bash).

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX Разработка TUNE-IT

Списки прав доступа файла

Список прав доступа (Access Control Lists, ACL) – представляет собой набор записей, каждая из которых задает права доступа к файлу для конкретного пользователя или конкретной группы пользователей. ACL расширяет дискреционную модель доступа к файлам, позволяя задавать индивидуальные права достаточно большому количеству пользователей и групп.

Механизм ACL реализован на уровне файловой системы ext4. Для включения механизма ACL необходимо смонтировать файловую систему с ключом acl:

```
$ mount -o acl [устройство] [точка_монтирования]
```

Существует два типа ACL:

- ACL, который используется для определения прав доступа процесса к файлу. Такие ACL связываются с обычными файлами, директориями и другими типами файлов. Для их хранения используется расширенный атрибут system.posix_acl_access. Будем в тексте таких ACL называть просто ACL.
- ACL по умолчанию, который связывается только с директориями, и определяет начальный ACL, который связывается с файлами и директориями, создаваемыми в данной директории. Для их хранения используется расширенный атрибут system.posix_acl_default. Будем такие ACL называть ACL по умолчанию.

Процесс может прочитать ACL файла только если у него есть право поиска в директории, содержащей файл.

Формат записей ACL

Каждая запись состоит из трёх частей:

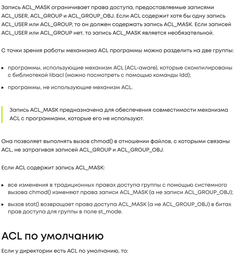
- тип записи;
- идентификатор пользователя или группы (UID или GID) в записях типа ACL_USER или ACL_GROUP;
- права доступа, как комбинация чтения, записи и исполнения/поиска.

Возможные типы записей представлены в таблице.

Типы записей ACL

Тип записи	Назначение
ACL_USER_OBJ	Запись определяет права доступа для владельца файла. ACL должен содержать ровно одну такую запись. Она задаёт традиционные права доступа для владельца (пользователя) файла.
ACL_USER	Запись определяет права доступа для пользователя, идентификатор которого содержит запись. Таких записей может быть от нуля и больше, однако для конкретного пользователя не может быть более одной такой записи.
ACL_GROUP_OBJ	Запись определяет права доступа для группы-владельца файла. ACL должен содержать ровно одну такую запись. Она задаёт традиционные права доступа для группы-владельца файла, если только ACL не содержит запись ACL_MASK.
ACL_GROUP	Запись определяет права доступа для группы, идентификатор которой содержит запись. Таких записей может быть от нуля и больше, однако для конкретной группы не может быть более одной такой записи.
ACL_MASK	Запись определяет максимальные права доступа, которые могут быть предоставлены записями ACL_USER, ACL_GROUP_OBJ, ACL_GROUP. ACL содержит не более одной записи ACL_MASK. Если ACL содержит записи ACL_USER или ACL_GROUP, то наличие записи ACL_MASK является обязательным. Она задаёт традиционные права доступа для группы-владельца файла.
ACL_OTHER	Запись определяет права доступа для процессов, которые не соответствуют другим записям в ACL. ACL должен содержать ровно одну такую запись. Она задаёт традиционные права доступа для остальных пользователей.

На рисунке показана, как соотносятся категории пользователей, используемые битами прав доступа, и типы записей ACL.



Linux ACL (1)

Пользователь может создать простой ACL, который фактически дублирует биты доступа. Как только пользователь создаст хотя бы одну индивидуальную запись для пользователя или группы - будет создан полноценный ACL, в котором обязательно появится запись ACL_MASK.

Алгоритм проверки прав доступа с помощью ACL

Проверка производится в следующем порядке до первого совпадения:

1. Если процесс является привилегированным, то предоставляются все права. За исключением случая, когда привилегированный процесс запускает исполняемый файл, такому процессу предоставляется право на исполнение, только если оно предоставлено хотя бы одной записью ACL.

2. Если FSUID процесса совпадает с идентификатором владельца файла UID, то процессу предоставляются права доступа, указанные в записи ACL_USER_OBJ.

3. Если FSUID процесса совпадает с идентификатором какой-либо записи ACL_USER, то данному процессу предоставляются права, указанные в этой записи, логически умноженные (операция И) со значением ACL_MASK.

4. Если FSGID или любой идентификатор дополнительной группы процесса совпадают с идентификатором группы-владельца файла GID (запись ACL_GROUP_OBJ) или с идентификатором какой-либо записи ACL_GROUP, и найденная запись предоставляет требуемые права, данному процессу предоставляются права, указанные в этой записи, логически умноженные (операция И) со значением ACL_MASK. Если такая запись найдена, но она не предоставляет требуемые права, то доступ запрещается.

5. В остальных случаях процессу предоставляются права доступа, указанные в записи ACL_OTHER.

Назначение записи ACL_MASK

Запись ACL_MASK ограничивает права доступа, предоставляемые записями ACL_USER, ACL_GROUP и ACL_GROUP_OBJ. Если ACL содержит хотя бы одну запись ACL_USER или ACL_GROUP, то он должен содержать запись ACL_MASK. Если записей ACL_USER или ACL_GROUP нет, то запись ACL_MASK является необязательной.

С точки зрения работы механизма ACL программы можно разделить на две группы:

- программы, использующие механизм ACL (ACL-aware), которые скомпилированы с библиотекой libacl (можно посмотреть с помощью команды ldd);
- программы, не использующие механизм ACL.

> Запись ACL_MASK предназначена для обеспечения совместимости механизма ACL с программами, которые его не используют.

Она позволяет выполнять вызов chmod() в отношении файлов, с которыми связаны ACL, не затрагивая записей ACL_GROUP и ACL_GROUP_OBJ.

Если ACL содержит запись ACL_MASK:

- все изменения в традиционных правах доступа группы с помощью системного вызова chmod() изменяют права записи ACL_MASK (а не записи ACL_GROUP_OBJ);
- вызов stat() возвращает права доступа ACL_MASK (а не ACL_GROUP_OBJ) в битах прав доступа для группы в поле st_mode.

ACL по умолчанию

Если у директории есть ACL по умолчанию, то:

- директория, создаваемая в данной директории, наследует ACL по умолчанию в качестве своего ACL по умолчанию;
- новый файл или директория, создаваемые в данной директории, устанавливают ACL по умолчанию в качестве своего ACL доступа. Для записей, соответствующих традиционным битам прав доступа (ACL_USER_OBJ, ACL_MASK или, если отсутствует запись ACL_MASK, ACL_GROUP_OBJ, ACL_OTHER), применяется операция "И" с соответствующими битами аргумента mode системного вызова (open(), mkdir()), использованного для создания файла или подкаталога.

Если у директории есть ACL по умолчанию, то маска umask не принимает участия в определении прав доступа для записей ACL доступа нового файла, созданного в данной директории.

Если у директории нет ACL по умолчанию, то:

- новые подкаталоги также не имеют ACL по умолчанию;
- права доступа нового файла или директории устанавливаются по традиционным правилам: права устанавливаются в значение аргумента mode (переданного в open(), mkdir()), исключая биты из маски процесса.

Командный интерфейс

Если на файл установлен ACL, то команда ls -l выведет минимальную информацию об ACL. Знак "+" в конце строки битов доступа говорит о том, что с файлом связан ACL. Права для владельца файла совпадают с правами владельца файла, права для группы-владельца показывают установленную ACL маску, а не права для группы-владельца файла. Права для остальных пользователей совпадают с правами для остальных пользователей.

Если на файл установлен ACL, то с помощью команды chmod нельзя изменить права для группы-владельца файла, вместо этого будет изменена ACL-маска. Чтобы изменить права для группы-владельца файла нужно использовать команду setfacl -m g:[права имя_файла.

Для вывода ACL предназначена команда getfacl.

```
getfacl [опции] список_файлов
getfacl [опции] -
```

По умолчанию getfacl выводит как ACL доступа, так и ACL по умолчанию (если это директория, у которой есть ACL по умолчанию), а также в виде комментариев эффективные права для тех записей, права в которых отличаются от эффективных. Опции меняют это поведение. Опция -d выводит только ACL доступа, а опция -d выводит только ACL по умолчанию. Опция -R выводит ACL рекурсивно для всех файлов и директорий. Если вместо списка файлов напечатать знак "-", то getfacl читает список файлов из стандартного ввода.

Вывод команды getfacl может быть использован как ввод для команды setfacl. Например, можно с помощью команды getfacl -R /путь_к_директории вывести все ACL для директории и её содержимого. Этот вывод можно сохранить в файле, и затем использовать для массового восстановления ACL на нескольких файлах с помощью команды setfacl --set-file=имя_файла.

Для каждого файла выводится следующая информация.

Первые три строки в виде комментариев содержат имена файла, владельца и группы-владельца.

Четвёртая строка содержит дополнительные права доступа: set-user-ID (s), set-group-ID (s), sticky (t) или (-), если права отсутствуют. Если все права отсутствуют, строка не выводится.

Следующие строки выводят ACL доступа, и по порядку содержат права для владельца, права для именованных пользователей, права для группы-владельца, права для именованных групп. Далее выводится маска (mask), которая ограничивает права, назначенные именованным пользователям и именованным группам (права владельца и всех остальных пользователей не затрагивается маской). Затем выводятся права для всех остальных пользователей системы (other).

В отличие от обычных файлов, директории могут иметь ACL по умолчанию, который определяет права на вновь создаваемые файлы и директории.

Следующие строки выводят ACL по умолчанию в том же порядке, что и ACL доступа. Обычно для пользователей и групп присутствуют права 0, которые для обычных файлов будет проигнорировано (так как новые файлы обычно создаются неисполняемыми), а для директорий будет установлено право на поиск.

Для установки ACL предназначена команда setfacl.

```
setfacl [опции] [{-m|-x} acl_спец] [{-M|-X} acl_file] файл
setfacl --restore=файл
```

Рассмотрим несколько примеров.

Предоставление индивидуальному пользователю user1 доступа на чтение:

```
setfacl -m u:user1:r file1
```

Запрет доступа на запись для всех групп и всех индивидуальных пользователей (с помощью маски эффективных прав):

```
setfacl -m m:rx file1
```

Удаление записи индивидуальной группы из ACL файла.

```
setfacl -x g:group1 file1
```

Копирование ACL одного файла в другой:

```
getfacl file1 | setfacl --set-file=- file2
```

Копирование ACL доступа в ACL по умолчанию для одной и той же директории:

```
getfacl --access dir1 | setfacl -d -M- dir1
```

Предыдущий урок
Следующий урок

Суперпользователь root

В ОС GNU/Linux существует специальный пользователь, который имеет полную власть над системой - это суперпользователь с именем root и пользовательским идентификатором UID равным 0. Этот пользователь используется для администрирования системы. Чтобы обычный пользователь мог конфигурировать ПО, настраивать время и сеть, управлять файловыми системами и устройствами, он должен повысить (эскалировать) свои права до пользователя root.

Нужно отметить, что обычный пользователь может управлять USB-устройствами, а именно монтировать их и работать с файлами на этих устройствах.

С неограниченными правами появляется большая ответственность: пользователь root может разрушить всю систему: удалить все файлы, удалить пользователей, установить вредоносное ПО и т.д. Если учётная запись пользователя root скомпрометирована, то злонамеренный пользователь может получить административные права в системе. Хотя существует возможность входить в систему под пользователем root, этого делать категорически не рекомендуется.

Системный администратор должен входить в систему и работать в ней под учётной записью обычного пользователя. И только для выполнения административных задач, требующих повышенных прав, временно переходить в режим root, используя для этого специальные механизмы (su, sudo, PolicyKit).

Если войти в систему под учётной записью пользователя root, то всё графическое окружение будет выполняться с административными правами без всякой на то необходимости. Тогда любая брешь в безопасности, которая в обычной ситуации скомпрометировала только обычного пользователя, потенциально может скомпрометировать всю систему.

Команда su

Для временного переключения одного пользователя на другого используется команда su. Если su выполняет обычный пользователь, он должен ввести пароль того пользователя, на которого он переключается. Если su выполняет root, то пароль не запрашивается. Если имя пользователя не указывается, то по умолчанию происходит переключение на пользователя root.

Команда su без ключей запускает так называемый “не-входной” bash, а команда su - (с ключём минус) “входной” bash. Разница заключается в том, что “su -” устанавливает окружение bash как будто пользователь вошёл в систему, а su использует текущее окружение.

Рекомендуется, чтобы системный администратор выполнял “su -”, чтобы получить окружение bash того пользователя, на которого происходит переключение.

Обычно команда su используется, чтобы получить интерфейс командной строки, которая выполняется от имени другого пользователя, в большинстве случаев root. Однако, с ключём -c можно просто запустить некоторую команду от имени другого пользователя.

Команда sudo

В некоторых случаях учетная запись пользователя root может вообще не иметь установленного пароля по соображениям безопасности. В этом случае пользователи не могут войти в систему как root напрямую с паролем, и команда su не может быть использована для получения интерактивной оболочки. Одним из инструментов, который можно использовать для получения доступа root в этом случае, является команда sudo.

В отличие от su, sudo обычно требует от пользователей ввода собственного пароля для аутентификации, а не пароля учетной записи пользователя, на которого происходит переключение.

То есть пользователям, которые используют sudo для выполнения команд от имени root, не нужно знать пароль root. Вместо этого они используют свои собственные пароли для аутентификации доступа.

Кроме того, sudo можно настроить таким образом, чтобы разрешить определенным пользователям выполнять любую команду от имени другого пользователя или только некоторые команды от имени этого пользователя.

Например, если sudo настроен на разрешение пользователю user1 выполнять команду usermod от имени root, пользователь user1 может выполнить следующую команду для блокировки или разблокировки учетной записи пользователя:

```
[user1@host ~]$ sudo usermod -L user2
[sudo] password for user1:
[user1@host ~]$ su - user2
Password:
su: Authentication failure
[user1@host ~]$
```

Если пользователь попытается выполнить команду от имени другого пользователя, а конфигурация sudo этого не позволяет, команда будет заблокирована, попытка будет зарегистрирована, и по умолчанию электронное письмо будет отправлено пользователю root.

Все члены группы wheel могут использовать sudo для выполнения команд от имени любого пользователя, включая root. Пользователю будет предложено ввести свой собственный пароль.

Если в системе есть пользователь без прав администратора, который может использовать sudo для выполнения команды su, он может запустить “sudo su -”, чтобы получить интерактивную оболочку пользователя root. Это работает, потому что sudo запускает “su -” от имени пользователя root, а пользователю root не нужно вводить пароль для использования su.

Другой способ получить доступ к учетной записи root с помощью sudo - использовать команду “sudo -i”. Эта команда переключит пользователя на учетную запись root и запустит оболочку по умолчанию этого пользователя (обычно bash) и связанные с ней сценарии входа в систему. Если просто нужно запустить оболочку, можно использовать команду “sudo -s”.

Политика конфиденциальности Пользовательское соглашение Обратиться в поддержку
Функциональные характеристики ПО Руководство пользователя Стоимость ПО

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX Разработка TUNE-IT

11/13 **Тестирование. DAC**

текущий урок

Тестирование — до 13 апр. 2026 г. 23:59 MSK Завершено

Задание 1.1

0.0/0.5 points (graded)

Какие условия влияют на назначение группы-владельца нового файла?

- [x] Режим монтирования файловой системы
- [x] Бит Set-GID, установленный на директорию, в которой создаётся файл
- [x] Дополнительные группы, в которые входит пользователь, создающий файл
- [ ] Маска доступа (umask)

✖

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.2

0.5/0.5 points (graded)

Какими дискреционными правами должен обладать процесс, чтобы удалить файл?

- [x] Правом записи на директорию, содержащую файл
- [x] Правом поиска на директорию, содержащую файл
- [ ] Правом записи на файл
- [ ] Пользователь, удаляющий файл, должен быть владельцем файла

✔

Show answer

Отправить

Задание 1.3

0.5/0.5 points (graded)

В каком пространстве расширенных атрибутов хранятся ACL?

- (•) system
- ( ) user
- ( ) trusted
- ( ) security

✔

Show answer

Отправить

Задание 1.4

0.5/0.5 points (graded)

Какие типы файлов могут иметь ACL по умолчанию?

- ( ) Обычные
- (•) Директории
- ( ) Специальные файлы устройств

✔

Show answer

Отправить

Задание 1.5

0.5/0.5 points (graded)

Для чего предназначена запись ACL_MASK?

- (•) Для совместимости с командой chmod
- ( ) Для ограничения прав владельца файла
- ( ) Для ограничения прав группы-владельца файла
- ( ) Это аналог маски доступа (umask) для файлов, на которые установлен ACL

✔

Show answer

Отправить

Задание 1.6

0.5/0.5 points (graded)

Что произойдёт после выполнения команды "getfacl dir1 | setfacl --set-file=- dir2"?

- (•) ACL доступа директории dir2 станет равным ACL доступа директории dir1
- ( ) ACL по умолчанию директории dir1 станет равным её ACL доступа
- ( ) ACL доступа директории dir1 будет добавлено к ACL доступа директории dir2
- ( ) Будет ошибка

✔

Show answer

Отправить

Задание 1.7

0.5/0.5 points (graded)

Для каких типов записей ACL маска (ACL_MASK) ограничивает максимальные права доступа к файлу?

- [ ] ACL_USER_OBJ
- [x] ACL_USER
- [ ] ACL_OTHER
- [x] ACL_GROUP_OBJ
- [x] ACL_GROUP

✔

Show answer

Отправить

Задание 1.8

0.5/0.5 points (graded)

Что произойдёт после выполнения команды "getfacl --access dir1 | setfacl -d -M- dir1"?

- (•) ACL по умолчанию директории dir1 станет равным её ACL доступа
- ( ) ACL доступа директории dir1 станет равным её ACL по умолчанию
- ( ) В ACL доступа директории dir1 будет удалён элемент ACL_MASK
- ( ) Будет ошибка

✔

Show answer

Отправить

Задание 1.9

0.5/0.5 points (graded)

Что произойдёт после выполнения команды "usermod -aG wheel student"?

- (•) Пользователь student будет добавлен в группу wheel; остальные группы останутся без изменений
- ( ) Группа wheel станет основной группой пользователя student
- ( ) Пользователь student будет добавлен в группу wheel и исключён из всех других дополнительных групп
- ( ) Будет ошибка

✔

Show answer

Отправить

Задание 1.10

0.5/0.5 points (graded)

Пароль какого пользователя нужно вводить при выполнении команды 'sudo usermod -L user2'?

- (•) Пользователя, который запустил команду sudo
- ( ) Пользователя, над которым будет выполнена команда usermod
- ( ) Пользователя root

✔

Show answer

Отправить

‹ Предыдущий урок Следующий урок ›

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026.
Дизайн MAX Разработка TUNE-IT

12/13 Заключение

текущий урок

В данном модуле мы познакомились с базовым механизмом управления доступом пользователей к файлам в ОС GNU/Linux, научились назначать права доступа и переключаться между пользователями.

В следующем модуле мы подробно рассмотрим процессы, которые представляют собой вторую основную системную сущность. Таким образом знания, полученные в первых трёх модулях, объединятся в стройную картину, на основании которой можно будет двигаться дальше и рассматривать вопросы системного администрирования.

Резюме

В этом модуле мы изучили следующие команды:

useradd - создание учётной записи пользователя

userdel - удаление учётной записи пользователя

usermod - изменение учётной записи пользователя

chage - вывод изменение атрибутов пароля пользователя

passwd - изменение пароля и атрибутов пароля пользователя

chown - изменение владельца файла

chgrp - изменение группы-владельца файла

chmod - изменение прав доступа к файлу

lsattr - вывод флагов файла

chattr - установка флагов файла

getfacl - вывод ACL

setfacl - установка ACL

su - переключение пользователей

sudo - выполнение команды от имени другого пользователя

В этом модуле мы изучили следующие системные файлы:

/etc/passwd - информация об учетных записях пользователей

/etc/shadow - информация о паролях пользователей

/etc/group - информация об учетных записях пользовательских групп

/etc/default/useradd - значения по умолчанию для создания учетной записи

/etc/login.defs - значения по умолчанию для создания учетной записи и пароля пользователя

/etc/skel/ - директория, содержащая файлы по умолчанию

Источники

- Ефанов, Д. В. Базовые механизмы защиты ядра Linux : учебное пособие / Д. В. Ефанов. — Москва : НИЯУ МИФИ, 2022. — 192 с. — ISBN 978-5-7262-2924-9. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/355520 (дата обращения: 22.01.2026). — Режим доступа: для авториз. пользователей.
- ГОСТ Р 50922-96. Защита информации. Основные термины и определения. М.: Издательство стандартов, 1996.
- ГОСТ 34.11-2018. Информационная технология. Криптографическая защита информации. Функция хэширования. М.: Стандартинформ, 2018.
- Pro Bash Programming: Scripting the GNU/Linux Shell. Chris Johnson. Apress, 2009.

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX Разработка TUNE-IT

1/13 **Введение**

текущий урок

Модуль посвящён рассмотрению концепции процесса, его жизненного цикла, средств взаимодействия процессов. Также изучаются механизмы Set-UID и привилегий, что является продолжением темы DAC, подробно представленной в предыдущем модуле.

Как полученные знания пригодятся в профессиональном развитии

ОС GNU/Linux — это многозадачная многопользовательская ОС. Данные свойства реализуются с помощью специальных системных сущностей - процессов. Поэтому управление процессами является необходимым навыком не только системного администратора, но и разработчика, так как разработка и отладка программ - это непосредственная работа с процессами.

Цели и результат модуля

Студент, освоивший программу третьего модуля, должен:

1. Понимать общую концепцию процессов.
2. Уметь выполнять мониторинг и управление процессами.
3. Уметь посылать сигналы процессам и управлять заданиями bash.
4. Понимать как процесс взаимодействует с файлами.
5. Уметь пользоваться механизмом Set-UID и привилегиями процессов для создания привилегированных программ.

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX Разработка TUNE-IT

Определение процесса

Концепция процесса является фундаментальной для многозадачной операционной системы. Самое простое определение процесса следующее:

> Процесс - это программа во время исполнения.

Давайте посмотрим, что скрывается за этим простым определением. На диске хранится исполняемый файл, который представляет собой скомпилированную программу и содержимым которого является двоичный код. Когда эту программу запускают на исполнение, она становится процессом. Говорят, что программа выполняется в контексте процесса.

Чтобы запустить программу, сначала нужно считать её содержимое в оперативную память, т.е. выделить место для этой программы, затем нужно выделить время ЦП для выполнения инструкций программы. В ходе выполнения программа может открывать файлы и записывать туда информацию, взаимодействовать с другими работающими программами, устанавливать сетевые соединения и т.д. Другими словами, для своего выполнения программа требует выделения системных ресурсов (места в ОП, времени ЦП, пропускной способности системы ввода/вывода и сетевой системы и т.д.). Как уже было сказано, управлением ресурсами занимается ядро операционной системы. Поэтому, с точки зрения ядра процесс - это единица потребления системных ресурсов, такая системная сущность для которой выделяются системные ресурсы. Т.е. задача обеспечения программы необходимыми ресурсами решается как задача выделения ресурсов процессу, а также их учёта и мониторинга. Поэтому давайте дадим более сложное определение процесса:

> Процесс - это единица потребления системных ресурсов.

Процесс можно рассматривать как некий контейнер, в который загружается программа.

Несколько процессов могут выполнять одну и ту же программу одновременно, при этом один процесс может выполнять несколько программ последовательно.

Ядро Linux является многопроцессным (или многозадачным) в том смысле, что одновременно на одном процессоре могут выполняться несколько процессов чередуя своё выполнение во времени. За справедливым распределением времени ЦП между конкурирующими процессами следит планировщик (scheduler), который выбирает для выполнения наиболее приоритетный процесс, а также может прервать (вытеснить) выполнение процесса, если появился более приоритетный процесс. Такой подход к планированию называется вытесняющей (preemptable) многозадачностью.

В понятие процесса включается:

- адресное пространство (address space), выделенное процессу в памяти, в которое загружается программа;
- контекст (состояние) процесса;
- атрибуты безопасности;
- окружение (environment) процесса, включающее глобальные и локальные переменные;
- выделенные системные ресурсы (например, дескрипторы открытых файлов и сетевые порты).

Дескриптор процесса

Чтобы управлять процессами, ядру нужна информация об их состоянии, связях друг с другом, о взаимодействии с файловой системой и другими подсистемами ядра и т.п.

Мы говорили в первом модуле, что для файла разделяется информация, которая хранится в файле (данные), и информация, которая описывает файл (метаданные или атрибуты). Для процесса можно провести такую аналогию с файлами: в процессе можно выделить программу, которую выполняет процесс, и информацию, которая описывает процесс, как системную сущность. Эту информацию будем называть атрибутами процесса.

Для описания процесса ядро использует дескриптор процесса, который представлен структурой task_struct. В таблице показаны основные поля дескриптора процесса.

Основные поля структуры task_struct

Название	Назначение
state	Состояние выполняющегося процесса
flags	Флаги процесса
*sched_class	Класс планирования
*mm	Виртуальная память процесса
exit_state	Состояние завершающегося процесса
exit_code	Код возврата процесса
exit_signal	Сигнал, по которому завершился процесс
pid	Идентификатор процесса - основной атрибут процесса
comm[TASK_COMM_LEN]	Имя программы, которую выполняет процесс
*fs	Информация о файловой системе
*files	Таблица дескрипторов открытых файлов
*nsproxy	Пространство имён
*signal	Обработчики сигналов
*cgroups	Контрольные группы

У каждого процесса есть идентификатор (process ID — PID), положительное целое число, уникальным образом идентифицирующее процесс в системе. Идентификаторы процессов используются и возвращаются различными системными вызовами. PID также используется при необходимости создания идентификатора, который будет уникальным для процесса. Идентификатор вызывающего процесса возвращается системным вызовом getpid().

Посмотреть содержимое дескриптора процесса, зная его PID, можно в файле файле /proc/<pid>/status.

Ядро ограничивает максимальное количество процессов в системе, устанавливая максимально возможный PID. При создании нового процесса ему присваивается следующий по порядку PID. Всякий раз при достижении ограничения идентификаторов ядро перезапускает свой счётчик идентификаторов процессов, чтобы они назначались, начиная с наименьших целочисленных значений.

Верхний порог для идентификаторов процессов можно изменить, задав значение в файле /proc/sys/kernel/pid_max (которое на единицу больше, чем максимально возможное количество идентификаторов процессов). На 32-разрядной платформе максимальным значением для этого файла является 32 768, но на 64-разрядной платформе оно может быть установлено в любое значение вплоть до 2^22 (приблизительно 4 миллиона), позволяя справиться с очень большим количеством процессов. Это можно посмотреть следующим образом:

```
$ uname -m
x86_64
$ cat /proc/sys/kernel/pid_max
4194304
```

У каждого процесса есть процесс-родитель, который его создал с помощью системного вызова fork(). Определить идентификатор своего родительского процесса вызывающий процесс может с помощью системного вызова getppid().

Процесс с PID равным 1 – это процесс systemd, который запускается ядром в конце процедуры начальной загрузки. Этот процесс отвечает за запуск сервисов операционной системы в пространстве пользователя и перевод системы в некоторое состояние (называемое target), например в многопользовательский режим. Процесс systemd никогда не завершается. С точки зрения ядра это обычный пользовательский процесс.

Пространство числовых идентификаторов PID организовано в виде древовидной структуры с процессом systemd в основании дерева. Каждый процесс имеет родительский процесс, и может иметь процессов-потомков. Родитель каждого процесса имеет собственного родителя и т.д., возвращаясь в конечном итоге к процессу systemd - предку всех процессов. Вывести дерево процессов на экран можно с помощью команды pstree.

Если у некоторого процесса завершился родитель, то его родительским процессом становится процесс systemd, поэтому процесс с PID равным 1 называют всеобщим предком.

Так как systemd постоянно развивается, родителем осиротевшего процесса может стать не именно процесс systemd с PID равным 1, а какой-то вспомогательный процесс, но с точки зрения концепции процесса это всё равно некий корневой процесс в дереве процессов.

Родитель любого процесса может быть найден при просмотре поля PPid в файле /proc/<pid>/status.

‹ Предыдущий урок Следующий урок ›

Существует множество системных вызовов, предназначенных для управления процессами. Но самыми главными являются всего четыре: fork() предназначен для создания нового процесса, exec() — для запуска новых программ, exit() - для завершения процесса и wait() - для ожидания завершения процесса-потомка. Вместе эти четыре системных вызова образуют жизненный цикл процесса, показанный на рисунке.

linux-mp3-1-1

Но самом деле, для нормального завершения процесса предназначен системный вызов, который называется _exit(). А функция exit() - это библиотечная функция, которая внутри себя вызывает _exit().

Рассмотрим данные четыре системных вызова.

Создание процесса

Для создания нового процесса предназначен системный вызов fork().

```
#include <sys/types.h>
#include <unistd.h>
pid_t fork(void);
```

Новый процесс называется процессом-потомком. Это функция вызывается один раз, а управление возвращает дважды, с единственным отличием – в процессе-потомке она возвращает 0, а в родительском процессе – идентификатор (PID) созданного процесса-потомка.

Последнее обстоятельство объясняется тем, что у процесса может быть много потомков, а в ядре нет функций, с помощью которых можно было бы получить их идентификаторы. В процессе-потомке функция fork() возвращает 0, поскольку процесс-потомок имеет только одного родителя и всегда может получить его идентификатор с помощью функции getppid(). Идентификатор процесса-потомка с зарежимерован во ядром, поэтому невозможно получить 0 в качестве идентификатора процесса-потомка.

И родительский, и процесс-потомок продолжают выполнение инструкций программы, следующих за вызовом функции fork(). Процесс-потомок предоставляет собой копию родительского процесса. Таким образом, потомок получает в свое распоряжение копии сегментов данных, кучи и стека родителя. Родитель и потомок не используют совместно одну и ту же область памяти. Они совместно используют сегмент кода. Не производится немедленное копирование сегментов данных, стека и кучи, поскольку чаще всего за вызовом fork() следует exec(), который все меняет. Вместо этого используется метод, который получил название копирование при записи (copy-on-write, COW). Указанные выше области памяти используются совместно обоими процессами, но ядро делает их доступными только для чтения. Если один из процессов попытается изменить данные в этих областях, ядро немедленно сделает только копию конкретного участка памяти, обычно это «страница» виртуальной памяти.

Кроме того, есть возможность создания новых процессов с помощью системного вызова clone(). Это более универсальный вариант функции fork(), который позволяет вызывающему процессу определить, что будет совместно использоваться потомком и родителем.

В общем случае нельзя сказать точно, какой из двух процессов первым получит управление после вызова функции fork() – родитель или потомок. Это во многом зависит от алгоритма планирования, используемого ядром. Если необходимо синхронизировать работу процессов, то потребуется организовать некоторое взаимодействие между ними.

Процесс-потомок наследует от процесса-родителя следующие характеристики:

- Реальный идентификатор пользователя, реальный идентификатор группы, эффективный идентификатор пользователя, эффективный идентификатор группы.
- Идентификаторы дополнительных групп.
- Идентификатор группы процессов.
- Идентификатор сессии.
- Управляющий терминал.
- Флаги setuserID и setgroupID.
- Текущую рабочую директорию.
- Корневую директорию.
- Маску режима создания файлов.
- Маску сигналов и их диспозицию.
- Дескрипторы открытых файлов.
- Флаги closeonexec для открытых дескрипторов.
- Среду окружения.
- Присоединенные сегменты разделяемой памяти.
- Отображения в память.
- Ограничения на ресурсы.

Существуют следующие отличия между процессами:

- Функция fork() возвращает различные значения.
- Различные идентификаторы процессов.
- Различные идентификаторы родительских процессов.
- Значения таймеров в процессе-потомке устанавливаются равными 0.
- Блокировки файлов, установленные в родительском процессе, не наследуются.
- Таймеры, ожидающие срабатывания, в процессе-потомке сбрасываются.
- Набор сигналов, ожидающих обработки, в процессе-потомке очищается.

Рассмотрим фрагмент программы process67_fork.c из книги Кирсанова. Программа определяет две переменные - одну в системе данных и одну в стеке, и создаёт процесс-потомок с помощью системного вызова fork(). Программа демонстрирует, что процессы-родитель и потомок получают отдельные копии сегментов данных и стека.

```
/* Глобальная переменная в сегменте инициализированных данных */
static int idata = 111;
int
main(int argc, char *argv[])
{
    int istack = 222; /* Локальная переменная в стеке */
    pid_t childPid;

    switch (childPid = fork()) {
    case -1:
        errExit("fork");

    case 0: /* процесс-потомок */
/* Изменить переменные в сегментах данных и стека */
        idata *= 3;
        istack *= 3;
        break;

    default: /* родительский процесс */
/* Засунуть на три секунды, чтобы выполнялся потомок */
        sleep(3);
        break;
    }

/* Этот код выполняется и родителем и потомком */

    printf("PID=%ld %s idata=%d istack=%d\n", (long) getpid(), (childPid == 0) ? "child"
    : "(parent)", idata, istack);

    exit(EXIT_SUCCESS);
}
```

После запуска программы получим:

```
$ ./t_fork
PID=17892 (child)  idata=333 istack=666
PID=17891 (parent) idata=111 istack=222
```

Видно, что переменные в процессе-потомке были изменены, а в процессе-родителе остались исходные значения.

Завершение процесса

Двумя основными способами завершения работы процесса являются следующие:

- Нормальное завершение процесса с помощью системного вызова _exit().
- Аварийное завершение процесса, вызванное получением процессом сигнала с реакцией по умолчанию.

```
#include <unistd.h>
void _exit(int status);
```

Системный вызов _exit() немедленно завершает процесс. Все дескрипторы открытых файлов закрываются. Все потомки данного процесса переходят в процессу systemd. Процессу-родителю данного процесса посылается сигнал SIGCHLD.

Аргумент status задаёт код возврата процесса. Несмотря на то, что он имеет тип int, процессу-родителю возвращается только младшие 8 бит (status & 0xff) в качестве кода возврата процесса. Этот код процесс-родитель может получить с помощью семейства системных вызовов wait().

Код 0 принято считать признаком успешного завершения, а любые другие значения говорят о том, что процесс завершился аварийно. Определены две константы: EXIT_SUCCESS (0) и EXIT_FAILURE (1), которые могут использоваться в качестве аргумента системного вызова _exit() для повышения читаемости программы.

Системный вызов _exit() никогда не возвращает значения.

Но самом деле, функция «обертка_exit() библиотеки языка C выполняет системный вызов exit_group(). Для того, чтобы завершить все потоки процесса. Сам системный вызов _exit() завершает только вызывающий поток.

Мониторинг процесса-потомка

Процесс-родитель должен отслеживать изменения состояний своих потомков — когда они завершают работу или останавливаются по сигналу. Существует два метода мониторинга процессов-потомков: семейство системных вызовов wait() и перехват сигнала SIGCHLD, которым ядро извещает процесс-родитель о завершении процесса-потомка.

```
#include <sys/wait.h>

pid_t wait(int *wstatus);
pid_t waitpid(pid_t pid, int *wstatus, int options);
```

В случае успеха wait() возвращает PID завершившегося процесса-потомка. В случае ошибки возвращает -1.

В случае успеха waitpid() возвращает PID процесса-потомка, у которого изменилось состояние. Если задан макрос WNOHANG и один или более процессов-потомков соответствующие критерии ожидания, но ещё не изменили свой статус, возвращается 0. В случае ошибки возвращает -1.

В таблице собраны различия между системными вызовами wait() и waitpid().

Сравнение системных вызовов wait() и waitpid()	
wait()	waitpid()
Можно отслеживать только следующего завершившегося потомка (если потомков несколько).	Можно отслеживать конкретного потомка.
Если все потомки выполняются, вызов блокируется до тех пор, пока какой-нибудь из потомков не завершится.	Можно организовать неблокирующее ожидание, чтобы, если все потомки ещё выполняются, немедленно узнать об этом.
Можно отслеживать только завершение потомка.	Можно отслеживать потомков, которые завершились, а также потомков, которые были остановлены (по сигналу SIGSTOP или SIGTTIN) или возобновили свою работу (по сигналу SIGCONT).

Процесс-потомок, который завершился, но не был ожидаем процессом-родителем, переходит в состояние "зомби" ("сущность"). Ядро сохраняет в дескрипторе процесса-зомби минимальный набор информации (PID, код завершения, статистику использования ресурсов) для того, чтобы процесс-родитель смог позже выполнить системный вызов wait() и считать эту информацию. До тех пор, пока процесс-зомби полностью не исчезнет, он будет занимать дескриптор процесса, и если в системе будет достигнута максимальное количество процессов, то будет невозможно создать новый процесс.

Как уже было сказано, максимально возможное количество процессов (и нитей) можно получить из файла /proc/sys/kernel/pid_max, например:

```
$ cat /proc/sys/kernel/pid_max
4194304
```

На платформе x86-64, pid_max может быть установлено в любое значение до 2^22 (PID_MAX_LIMIT, примерно 4 млн.).

Если процесс-родитель завершится, порождённые им процессы-зомби унаследуются процессом systemd, который выполняет системный вызов wait(), чтобы удалить процессы-зомби.

Функции-обертки wait() и waitpid() библиотеки языка C выполняют системный вызов wait4(). Системный вызов wait4() является нестандартным. Для создания портируемых программ его использовать напрямую не рекомендуется.

Запуск новой программы

Функция fork() часто используется для создания нового процесса, который затем запускает другую программу с помощью одной из функций семейства exec().

Существует шесть различных функций семейства exec(), но только функция execve() является системным вызовом. Остальные пять – обычные библиотечные функции, которые внутри себя вызывают execve().

Когда процесс вызывает одну из функций exec(), то его коды полностью замещаются кодами другой программы, и эта новая программа начинает выполнение собственной функции main(). Идентификатор процесса при этом не меняется, поскольку функция exec() не создаёт новый процесс, она просто замещает в текущем процессе его сегмент кода, сегмент данных, динамическую область памяти и сегмент стека другой программой.

```
#include <unistd.h>
int execve(const char *pathname, char *const argv[],
           char *const envp[]);
```

Каждый дескриптор открытого процессом файла имеет флаг close-on-exec (закрыть при вызове exec). Если этот флаг установлен, дескриптор закрывается функцией exec. По умолчанию после вызова функции exec дескриптор остается открытым, если флаг close-on-exec не был специально установлен с помощью функции fcntl().

‹ Предыдущий урок Следующий урок ›

SkillFactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2020.
Дизайн: MAX Разработка: TUNE-IT

4/13 Урок 3. Структура памяти процесса

текущий урок

Память процесса состоит из сегментов. Существует пять типов сегментов:

Сегмент текста — содержит двоичный код исполняемой программы, запущенной процессом. Текстовый сегмент создается только для чтения, чтобы процесс не мог случайно изменить свои собственные инструкции из-за неверного значения указателя. Поскольку многие процессы могут выполнять одну и ту же программу, текстовый сегмент создается с возможностью совместного использования. Таким образом, единственная копия кода программы может быть отображена на виртуальное адресное пространство всех процессов.

Сегмент инициализированных данных — хранит глобальные и статические переменные, инициализированные явным образом. Значения этих переменных считываются из исполняемого файла при загрузке программы в память.

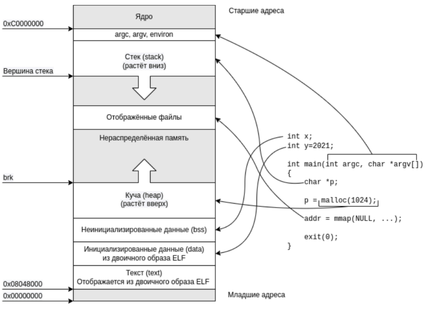
Сегмент неинициализированных данных — содержит глобальные и статические переменные, не инициализированные явным образом. Перед запуском программы система инициализирует всю память в этом сегменте значением 0. Исторически этот сегмент часто называют bss (block started by symbol). Основная причина помещения прошедших инициализацию глобальных и статических переменных в отдельный от неинициализированных переменных сегмент заключается в том, что, когда программа сохраняется на диске, нет никакого смысла выделять пространство под неинициализированные данные. Вместо этого исполняемой программе просто нужно записать местоположение и размер, требуемый для сегмента неинициализированных данных, и это пространство выделяется загрузчиком программы в ходе ее выполнения.

Куча — область, из которой память может динамически выделяться в ходе выполнения программы. Верхнюю границу кучи называют крайней точкой программы (program break).

Стек — содержит стековые фреймы. На платформе x86-32 стек располагается в верхней части памяти и растет вниз (по направлению к куче). Хотя стек растет вниз, край стека называется вершиной. Текущая вершина стека отслеживается в специально предназначенном для этого регистре — указателе стека. Для каждой вызванной на данный момент функции выделяется один стековый фрейм, в котором хранятся локальные (автоматические) переменные функции, аргументы и возвращаемое значение.

Команда size(1) выводит размеры сегмента текста, сегментов инициализированных и неинициализированных (bss) данных исполняемой программы.

На рисунке показана структура памяти процесса для архитектуры x86-32 и пример размещения переменных в разных сегментах памяти.



linux-m3-3-1

Область над стеком содержит аргументы командной строки программы и список переменных среды процесса. Области, закрашенные серым цветом, представляют собой недопустимые диапазоны в виртуальном адресном пространстве процесса, то есть области, для которых не созданы таблицы страниц.

Структура виртуальной памяти подразумевает разбиение памяти, используемой каждой программой, на небольшие блоки фиксированного размера, называемые страницами. Соответственно, оперативная память делится на блоки страничных кадров (фреймов) одинакового размера. В любой отдельно взятый момент времени в страничных кадрах физической памяти требуется наличие только некоторых страниц программы. Эти страницы формируют так называемый резидентный набор. Копии неиспользуемых страниц программы размещаются в области подкачки — зарезервированной области дискового пространства, применяемой для дополнения оперативной памяти компьютера, — и загружаются в оперативную память лишь по мере надобности. Когда процесс ссылается на страницу, которой нет в оперативной памяти, происходит ошибка отсутствия страницы, в результате чего ядро приостанавливает выполнение процесса, пока страница загружается с диска в память [Керриск].

В ЦП за реализацию механизма виртуальной памяти отвечает блок управления памятью (Memory Management Unit, MMU). На платформе x86-32 размер страницы составляет 4096 байт. Программа может определить размер страницы с помощью вызова sysconf(_SC_PAGESIZE). Для каждого процесса ядро поддерживает таблицу страниц. Если процесс пытается получить доступ к адресу, для которого не имеется соответствующей записи в таблице страниц, он получает сигнал SIGSEGV.

Механизм виртуальной памяти отделяет виртуальное адресное пространство процесса от физического адресного пространства оперативной памяти, что дает следующие преимущества:

- Изоляция процессов друг от друга и от ядра. Один процесс не может прочитать или изменить память другого процесса или ядра. Это достигается за счет записей в таблице страниц для каждого процесса, указывающих на различные наборы физических страниц в оперативной памяти (или в области подкачки).
- Совместное использование памяти несколькими процессами. Это возможно благодаря наличию записей в таблице страниц в различных процессах, ссылающихся на одни и те же страницы оперативной памяти. Совместное использование памяти происходит при двух наиболее распространенных обстоятельствах.
- Несколько процессов, выполняющих одну и ту же программу, могут совместно использовать одну и ту же (предназначенную только для чтения) копию программного кода. Эта разновидность совместного применения памяти осуществляется неявно, когда несколько программ выполняют один и тот же программный файл (или загружают одну и ту же совместно используемую библиотеку).
- Для явного запроса областей совместно используемой памяти с другими процессами процессы могут задействовать системные вызовы shmget() и mmap(). Это делается в целях обмена данными между процессами.
- Упрощается реализация схем защиты памяти, то есть записи в таблице страниц могут быть помечены, чтобы показать, что содержимое соответствующей страницы защищено от всего, кроме чтения, записи, выполнения или некоторого сочетания допустимых действий. Когда страницы оперативной памяти совместно используются несколькими процессами, можно указать, что у памяти есть защита от каждого процесса. Например,у одного процесса может быть доступ только к чтению страницы, а у другого — как к чтению, так и к записи.
- При компоновке программы не нужно знать о физическом размещении программы в оперативной памяти.
- Программа загружается и запускается быстрее, поскольку в памяти требуется разместить только ее часть. Кроме того, объем памяти для среды выполнения процесса (то есть виртуальный размер) может превышать ёмкость оперативной памяти.

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026
Дизайн MAX Разработка TUNE-IT

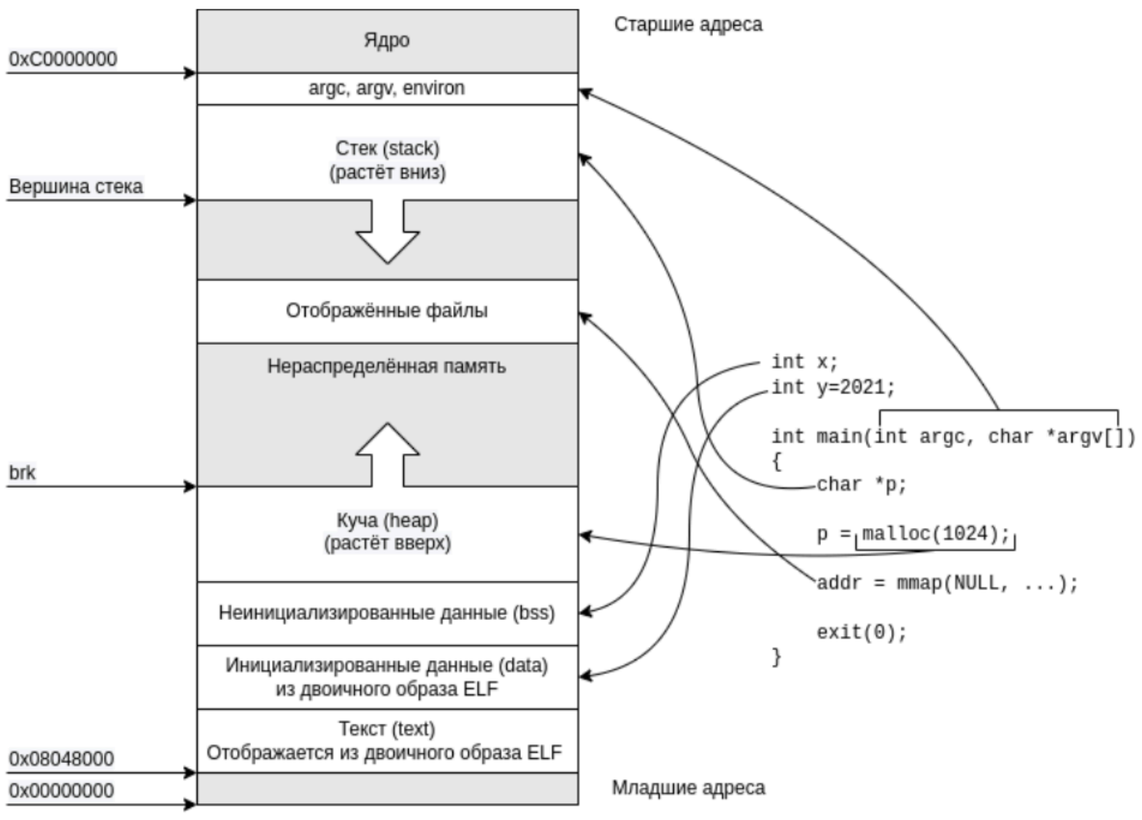
5/13 Урок 4. Алгоритм set-user-ID

текущий урок

При вызове функции exec() реальные идентификаторы пользователя и группы не изменяются, но эффективные идентификаторы могут быть изменены в зависимости от состояния битов set-UID и set-GID файла запускаемой программы.

Если бит set-UID установлен, то в качестве эффективного идентификатора пользователя процесса принимается идентификатор владельца файла программы. В противном случае эффективный идентификатор пользователя не изменяется (он не устанавливается равным реальному идентификатору пользователя).

Рассмотрим работу механизма Set-UID на примере изменения пользователем своего пароля. На рисунке показана последовательность запуска программы passwd с установленным битом set-UID.



linux-m3-4-1

Пользователь в командной оболочке bash запускает программу passwd, которая предназначена для смены пароля. Как мы знаем из предыдущего модуля, пароли пользователей хранятся в файле /etc/shadow, который в свою очередь доступен только пользователю root.

Возникает вопрос: как обычный пользователь может изменить содержимое файла, на который у него нет никаких прав?

Чтобы запустить программу passwd, командная оболочка bash сначала с помощью системного вызова fork() создаёт новый процесс, который начинает выполняться с теми же идентификаторами пользователя, что и сам bash.

Затем происходит системный вызов exec(), который обращается к бинарному файлу (программе) passwd. Почему мы можем запустить программу passwd на исполнение? Потому что у неё установлен бит ‘x’ для всех остальных пользователей.

В процессе запуска программы passwd, ядро видит, что на неё также установлен бит Set-UID, а значит её нужно запускать с изменённым эффективным идентификатором пользователя (EUID). Так как файл программы принадлежит пользователю root, EUID процесса устанавливается равным 0.

В результате, процесс, в контексте которого выполняется программа passwd, получает доступ к файлу /etc/shadow с правами пользователя root.

Работа с эффективным идентификатором группы происходит аналогичным образом.

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX Разработка TUNE-IT

Понятие привилегий

Механизм привилегий (capabilities) позволяет разделить неограниченные права суперпользователя (т.е. пользователя root, у которого EUID равен 0) по административно системы на отдельные группы прав, которые могут быть независимо друг от друга предоставлены обычным пользователям (у которых EUID отличен от 0). Этим достигается реализация принципа наименьших привилегий, а также уменьшается поверхность атаки: если программа, обладающая одной или более привилегиями скомпрометирована, её возможности по нанесению ущерба системе будут меньше по сравнению с такой же программой, выполняющейся с EUID равным 0.

Ядро Linux поддерживает около сорока привилегий. Привилегии хранятся в виде бинарных 64-битных векторов – упорядоченных наборов, где каждый бит соответствует одной определённой привилегии. Если бит установлен (равен 1), то привилегия предоставлена, если бит сброшен (равен 0), то привилегия не предоставлена.

Файл /proc/sys/kernel/cap_last_cap содержит числовое значение самой старшей привилегии, поддерживаемой ядром, т.е. это число показывает старший бит, который хранится в векторах привилегий. Например:

```
$ uname -r
5.15.32-100.fc32.x86_64
$ cat /proc/sys/kernel/cap_last_cap
40
```

В таблице показаны некоторые привилегии.

Название	Назначение
CAP_AUDIT_CONTROL	Включить или выключить аудит ядра. Изменить правила фильтрации. Получить статус и правила фильтрации.
CAP_AUDIT_READ	Читать журнал аудита через групповой netlink-сокет.
CAP_AUDIT_WRITE	Писать в журнал аудита.
CAP_CHOWN	Изменять UID или GID файла.
CAP_DAC_OVERRIDE	Права на чтение и запись любого типа файла. Права на исполнение обычного файла, если у него установлен хотя бы один из трёх битов исполнения. Права на поиск в директории.
CAP_DAC_READ_SEARCH	Права на чтение любого типа файла. Права на поиск в директории.
CAP_FOWNER	Права выступать владельцем любого файла.
CAP_LINUX_IMMUTABLE	Право устанавливать флаги инода FS_APPEND_FL и FS_IMMUTABLE_FL.
CAP_SETFCAP	Запись в расширенный атрибут файла security.capability.
CAP_SYS_ADMIN	Очень широкий набор действий (почти что root).

Привилегии можно назначать процессам и исполняемым файлам.

Привилегии процесса

Каждый процесс обладает пятью 64-битными векторами привилегий. Эти вектора хранятся в структуре cred, на которую ссылается указатель из дескриптора процесса task_struct. Рассмотрим фрагмент структуры cred:

```
struct cred {
...
        kernel_cap_t    cap_inheritable;
        kernel_cap_t    cap_permitted;
        kernel_cap_t    cap_effective;
        kernel_cap_t    cap_bset;
        kernel_cap_t    cap_ambient;
...
}
```

Тип kernel_cap_t является структурой, состоящей из двух 32-битных беззнаковых целых (unsigned int):

```
typedef struct kernel_cap_struct {
        __u32 cap[_KERNEL_CAPABILITY_U32S];
} kernel_cap_t;
```

, где _KERNEL_CAPABILITY_U32S = 2.

Наследуемый вектор (inheritable) - это привилегии, которые сохраняются при выполнении вызова execve(). Наследуемые привилегии остаются наследуемыми при выполнении любой программы. Наследуемые привилегии добавляются к разрешённому набору при выполнении программы, у которой есть соответствующие биты в наследуемом наборе файла.

Разрешённый вектор (permitted) - это максимальный набор привилегий, которые может использовать процесс (является максимальным эффективным набором). Это также максимальный набор привилегий, которые могут быть добавлены в наследуемый набор процесса, не имеющих привилегию CAP_SETPCAP в своём эффективном наборе. Если процесс удаляет привилегии из своего разрешённого набора, он никогда не сможет повторно получить эту возможность. Исключение составляют случаи запуска в процессе программы с установленным битом set-user-ID или программы с установленной привилегией на исполняемый файл.

Эффективный вектор (effective) - это набор привилегий, используемый ядром при выполнении проверок для процесса (permission checks).

Ограничивающий вектор (bounding) — это механизм, который может использоваться для ограничения привилегий, получаемых во время вызова execve().

Внешний вектор (ambient) - это привилегии, которые сохраняются при выполнении вызова execve() непривилегированной программой. Привилегия может быть внешней, если она одновременно разрешённая и наследуемая. Внешние привилегии автоматически сбрасываются, если соответствующая разрешённая или наследуемая привилегия сбрасывается.

Выполнение программы, которая изменяет UID или GID из-за наличия битов set-user-ID или set-group-ID, или выполнение программы, у которой установлены какие-либо файловые привилегии, очистит внешний набор.

Внешние привилегии добавляются в разрешенный набор и присваиваются эффективному набору при вызове execve().

Процесс-потомок наследует копии векторов привилегий процесса-родителя.

Посмотреть привилегии процесса можно в файле /proc/<pid>/status. Рассмотрим на примере обычного пользователя user1 привилегии процесса bash.

```
$ id
uid=1001(user1) gid=1001(user1) groups=1001(user1)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
$ cat /proc/self/status | grep ^Cap
CapInh: 0000000000000000
CapPrm: 0000000000000000
CapEff: 0000000000000000
CapBnd: 000001ffffffffff
CapAmb: 0000000000000000
$ capsh --print
Current: =
Bounding set =cap_chown,cap_dac_override,cap_dac_read_search,cap_fowner,cap_fsetid,cap_kill,cap_setgid,
cap_setuid,cap_setpcap,cap_linux_immutable,cap_net_bind_service,cap_net_broadcast,cap_net_
admin,cap_net_raw,cap_ipc_lock,cap_ipc_owner,cap_sys_module,cap_sys_rawio,cap_sys_chroot,c
ap_sys_ptrace,cap_sys_pacct,cap_sys_admin,cap_sys_boot,cap_sys_nice,cap_sys_resource,cap_s
ys_time,cap_sys_tty_config,cap_mknod,cap_lease,cap_audit_write,cap_audit_control,cap_setfc
,cap_mac_override,cap_mac_admin,cap_syslog,cap_wake_alarm,cap_block_suspend,cap_audit_re
ad,38,39,40
Ambient set =
Securebits: 00/0x0/1'b0
 secure-noroot: no (unlocked)
 secure-no-suid-fixup: no (unlocked)
 secure-keep-caps: no (unlocked)
 secure-no-ambient-raise: no (unlocked)
uid=1001(user1)
gid=1001(user1)
groups=1001(user1)
$ capsh --decode=0000000000000001
0x0000000000000001=cap_chown
```

Команда id выводит информацию о пользователе. Затем из файла /proc/self/status получаем содержимое пяти векторов привилегий процесса bash.

Для исследования и конструирования наборов привилегий предназначена команда capsh. Синтаксис команды capsh следующий:

```
capsh [OPTION]
```

Опция --print выводит привилегии процесса в мнемоническом виде и дополнительную информацию.

Опция --decode=N выводит название привилегии по её номеру.

Привилегии файла

Привилегии также можно связать с исполняемым файлом с помощью команды setcap. Привилегии файла хранятся в его расширенных атрибутах в пространстве security.capability. Запись в этот расширенный атрибут требует наличия привилегии CAP_SETFCAP.

Суперпозиция привилегий исполняемого файла и привилегий вызывающего процесса определяет привилегии процесса после выполнения системного вызова execve().

Существует три набора привилегий файла:

- Разрешённый набор (permitted) - эти привилегии автоматически разрешаются процессу, независимо от наследуемых привилегий процесса.
- Наследуемый набор (inheritable) - этот набор привилегий логически перемножается с наследуемым набором процесса (операция логического И), чтобы определить какие наследуемые привилегии должны быть включены в разрешённом наборе процесса после вызова execve().
- Эффективный бит (effective). Если этот бит установлен, то во время выполнения вызова execve() все новые разрешённые привилегии процесса также поднимаются в векторе эффективных привилегий. Если этот бит не установлен, тогда после execve(), никаких из новых разрешённых привилегий нет в новом эффективном наборе.

Вычисление привилегий во время вызова execve()

Во время выполнения вызова execve(), ядро вычисляет новые привилегии процесса по следующему алгоритму:

P'(ambient) = (файл привилегированный) ? 0 : P(ambient)

P'(permitted) = (P(inheritable) & F(inheritable)) |

(P(permitted) & P(bounding)) | P'(ambient)

P'(effective) = F(effective) ? P'(permitted) : P'(ambient)

P'(inheritable) = P(inheritable) [не изменяется]

P'(bounding) = P(bounding) [не изменяется]

где:

P() - привилегии процесса до вызова execve()

P'() - привилегии процесса после вызова execve()

F() - привилегии исполняемого файла

С точки зрения механизма привилегий программы можно разделить на две группы:

- программы, использующие привилегии, которые скомпилированы с библиотекой libcap (можно посмотреть с помощью команды ldd);
- программы, не использующие механизм привилегий (capability-dumb).

Рассмотрим привилегии исполняемого файла на примере команды cat. Пусть обычный пользователь попытаетсяввести на экран содержимое файла /etc/shadow, в котором хранятся хэши паролей пользователей и который доступен только root.

```
$ whereis -b cat
cat: /usr/bin/cat
$ ldd /usr/bin/cat
linux-vdso.so.1 (0x00007ffc474d0000)
libc.so.6 => /lib64/libc.so.6 (0x00007f1534915000)
/lib64/ld-linux-x86-64.so.2 (0x00007f1534ea4000)
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
```

Видно, что в выводе команды ldd отсутствует библиотека libcap. Это означает, что программа cat не умеет управлять своими привилегиями, т.е. разработчик программы cat не использовал API библиотеки libcap для обращения к механизму привилегий.

Создадим копию программы cat и с помощью команды setcap установим для неё привилегию cap_dac_read_search. Так как cat является capability-dumb-программой, то необходимо установить оба набора привилегий (+pe) у исполняемого файла: разрешённый (p) и эффективный (e). Если бы программа cat умела управлять своими привилегиями, то было бы достаточно установить только разрешённый (p) набор привилегий. Команда getcap показывает, что привилегия установлена.

```
$ cp /usr/bin/cat .
$ sudo setcap "cap_dac_read_search=ep" cat
$ getcap ./cat
./cat = cap_dac_read_search+ep
$ ./cat /etc/shadow | head -n 1
root:!:0:99999:7:::
```

Теперь обычный пользователь может выступать владельцем любого файла в системе, независимо от установленных для этого файла дискреционных прав доступа, т.е. может обходить механизм DAC по чтению.

Предыдущий урок Следующий урок

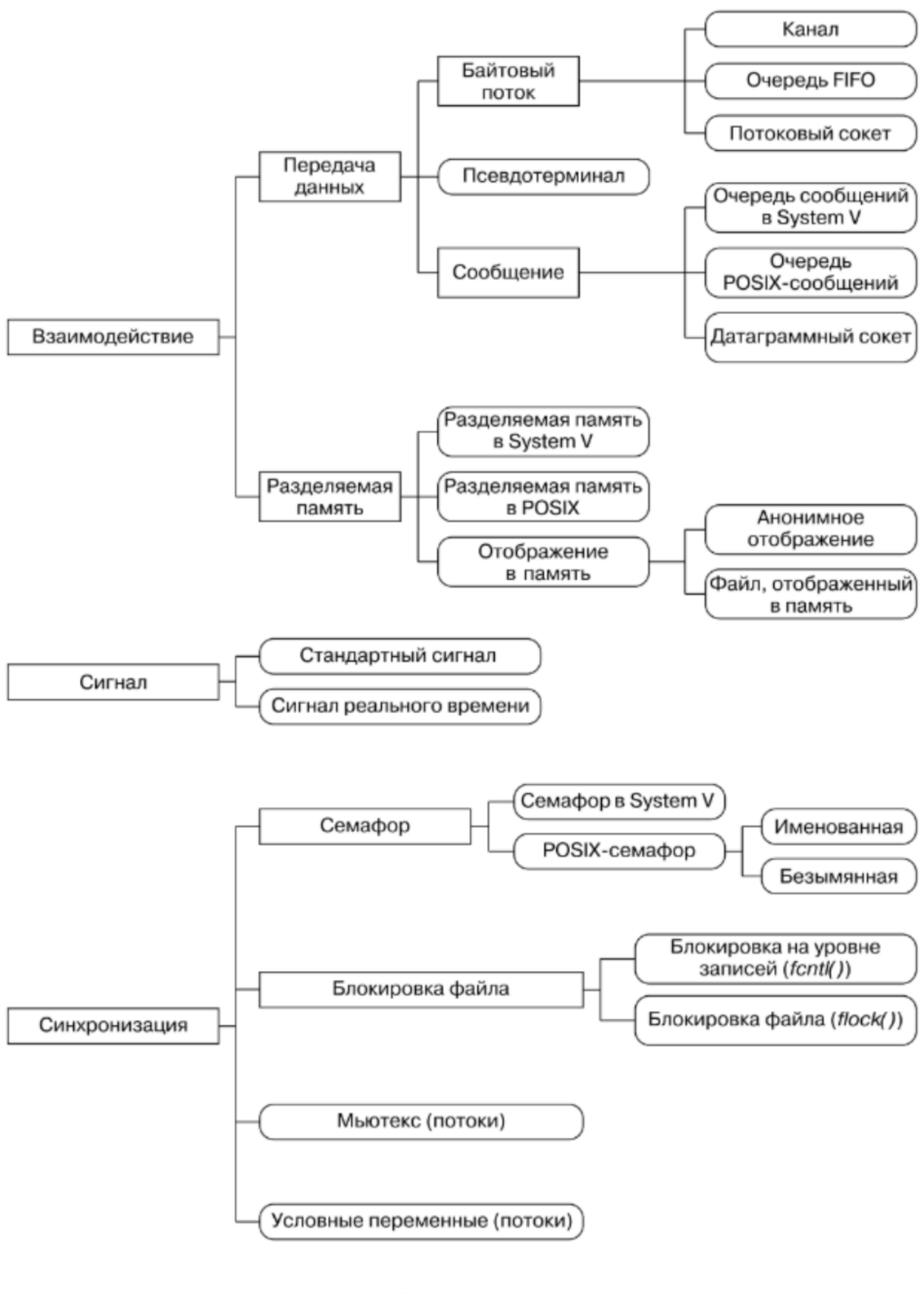
7/13 Урок 6. Средства межпроцессного взаимодействия (IPC)

текущий урок

Выполняя программу, процессы активно взаимодействуют как с файлами, так и с другими процессами. Однако, так как каждый процесс выполняется в своём изолированным от других процессов адресном пространстве, для взаимодействия процессов были разработаны специальные механизмы. В этом уроке проведём обзор средств межпроцессного взаимодействия (Interprocess Communication, IPC), а в следующем подробно рассмотрим механизм сигналов.

На рисунке показаны механизмы IPC, которые можно поделить на три категории:

- Взаимодействие. Эти механизмы отвечают за обмен данными между процессами.
- Синхронизация. Эти механизмы отвечают за синхронизацию работы процессов.
- Сигналы. Изначально они были предназначены для других целей, однако в некоторых ситуациях их можно использовать в качестве средства синхронизации. В редких случаях с их помощью можно организовать взаимодействие: номер сигнала как таковой является своеобразным видом информации, а сигналы реального времени могут нести в себе дополнительные данные (целое число или указатель).



linux-m3-6-1

Как показано на рисунке, разные механизмы часто предоставляют похожие возможности, связанные с IPC. Тому есть несколько причин:

- Похожие механизмы эволюционировали параллельно в разных вариантах UNIX и позже были перенесены на другие системы. Например, очереди FIFO были разработаны для System V, а (потоковые) сокеты впервые появились в BSD.
- Некоторые новые механизмы были разработаны для устранения недостатков более ранних аналогов. Например, механизмы POSIX IPC (очереди сообщений, семафоры и разделяемая память) появились в качестве улучшенной версии более старых средств работы с IPC из System V.

Ряд механизмов на самом деле обладают довольно разными возможностями. Например, потоковые сокеты могут использоваться для взаимодействия по сети, тогда как очереди FIFO позволяют обмениваться данными только между процессами на одном компьютере.

Различные механизмы взаимодействия позволяют процессам обмениваться данными друг с другом (эти же механизмы могут быть использованы для обмена информацией между потоками одного и того же процесса, но такая необходимость возникает не часто — потоки могут взаимодействовать с помощью общих глобальных переменных).

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026
Дизайн MAX Разработка TUNE-IT

8/13 Урок 7. Сигналы

текущий урок

Рассмотрим подробно механизм сигналов, так как системный администратор очень часто обращается именно к этому механизму для завершения “зависших” процессов.

> Сигнал — это оповещение процесса о том, что произошло некое событие.

Иногда сигналы также описываются как программные прерывания. Сигналы аналогичны аппаратным прерываниям в том смысле, что они останавливают нормальное выполнение программы. В большинстве случаев невозможно предсказать, когда именно будет доставлен тот или иной сигнал.

Один процесс может (при наличии необходимых разрешений) отправить сигнал другому процессу. При таком использовании сигналы могут рассматриваться как технология синхронизации.

Процесс также может отправить сигнал самому себе. Однако обычно источником большинства сигналов, отправляемых в процесс, является ядро. Далее перечислены несколько типов событий, которые заставляют ядро сгенерировать сигнал для процесса:

- Произошло аппаратное исключение. Это значит, что аппаратное обеспечение зафиксировало неверное состояние и оповестило об этом ядро, которое, в свою очередь, направило соответствующий сигнал затронутому процессу. Примерами аппаратного исключения могут быть выполнение ошибочного машинного кода, деление на ноль, а также обращение к недоступному участку памяти.
- Пользователь ввел один из специальных символов терминала, генерирующих сигналы. Примерами таких символов могут быть символ прерывания (обычно Ctrl+C) и символ приостановки (обычно Ctrl+Z).
- Произошло программное событие. Например, появился ввод из файлового дескриптора, размер окна терминала был изменен, сработал таймер, для процесса было превышено временное ограничение ЦП либо был завершен дочерний процесс данного процесса.

Каждому сигналу присваивается уникальный идентификатор — (небольшое) целое число, нумерация задается последовательно по возрастанию, начиная с 1. Эти целые числа определены в файле <signal.h> с символьными именами в формате SIGxxxx. Поскольку числа для каждого из сигналов отличаются в зависимости от реализации, в программах всегда применяются символьные имена. Например, когда пользователь вводит символ прерывания, процессу доставляется сигнал SIGINT (с номером 2).

Сигналы можно разделить на две большие категории. Первый набор сигналов состоит из традиционных, или стандартных, сигналов, которые используются ядром для оповещения процессов о свершении событий. В Linux стандартные сигналы пронумерованы от 1 до 31. Второй набор состоит из так называемых сигналов реального времени.

Говорят, что сигнал **генерируется** каким-либо событием. После генерации происходит **доставка** в процесс, который затем выполняет определенные действия для ответа на полученный сигнал. После того, как сигнал был сгенерирован, и до его доставки, говорят, что сигнал находится в состоянии **ожидания**.

Как правило, сигнал в режиме ожидания доставляется в процесс при его следующем запланированном выполнении, либо моментально, если процесс уже запущен (например, если процесс отправил сигнал самому себе). Но иногда мы должны быть уверены в том, что выполнение фрагмента кода не будет прервано доставкой сигнала. Для этого мы можем добавить сигнал в сигнальную маску процесса, то есть набор сигналов, доставка которых временно **заблокирована**. Если сигнал был сгенерирован, в то время как такие сигналы заблокированы, он остается в режиме ожидания до тех пор, пока не будет разблокирован (удален из сигнальной маски). Разные системные вызовы позволяют процессу добавлять и удалять сигналы из сигнальной маски.

По получении сигнала процесс по умолчанию выполняет одно из нижеперечисленных действий, в зависимости от полученного сигнала:

- Сигнал игнорируется, то есть сбрасывается ядром и не влияет на процесс. (Процесс никогда даже не узнает о том, что данный сигнал был отправлен.)
- Процесс завершается. Иногда это называют аварийным завершением процесса, что противопоставляется нормальному завершению — при использовании функции exit().
- Генерируется файл дампа ядра, и процесс завершается. Файл дампа ядра содержит отпечаток виртуальной памяти процесса, который может быть загружен в отладчик для изучения состояния процесса в момент завершения.
- Процесс останавливается — выполнение процесса приостанавливается.
- Выполнение процесса возобновляется после предшествовавшей приостановки.

Для каждого конкретного сигнала программа может изменить действие, выполняемое по его получении. Это называют установкой **диспозиции** сигнала. Программа может установить одну из нижеперечисленных диспозиций сигнала:

- Следует выполнить действие по умолчанию. Эта инструкция применяется для отмены изменения настройки диспозиции сигнала.
- Сигнал игнорируется. Эта инструкция используется для сигналов, действие по умолчанию для которых — завершение процесса.
- Выполняется **обработчик** сигнала.

Обработчик сигнала — функция, написанная программистом и выполняющая нужные действия при получении сигнала. Например, для оболочки установлен обработчик сигнала SIGINT (генерируемого символом прерывания, Ctrl+C), заставляющий оболочку прекратить выполнение текущей задачи и вернуть управление в основной цикл так, что пользователь опять видит на экране приглашение на ввод команды. Оповещение ядра о необходимости активации функции обработчика сигнала называют установкой или настройкой обработчика сигнала. Когда в ответ на получение сигнала программа активирует соответствующий обработчик, мы говорим, что сигнал был обработан или, что синонимично, перехвачен.

Файле /proc/PID/status содержит различные поля битовых масок, которые можно изучить для выяснения того, каким образом процесс обрабатывает сигналы. Битовые маски представлены в виде шестнадцатеричных чисел, при этом наименьший бит представляет сигнал номер 1, а следующий бит влево представляет сигнал номер 2 и т.д. Это поля SigPnd (ожидающие сигналы по потокам), ShdPnd (ожидающие сигналы процесса), SigBlk (заблокированные сигналы), SigIgn (игнорируемые сигналы), SigCgt (перехваченные сигналы).

Аналогичная информация может быть получена с использованием различных параметров команды ps(1).

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026
Дизайн MAX Разработка TUNE-IT

9/13 Урок 8. Управление заданиями

текущий урок

Одной из интересных особенностей bash является механизм управления заданиями (job control), который позволяет, работая в терминале, т.е. когда нет многооконной системы, одновременно запускать несколько программ и переключаться между ними. Управление заданиями поддерживается на уровне ядра с помощью двух понятий: группа процессов и сессия. Группы процессов и сессии составляют двухуровневые иерархические отношения между процессами: первые являются наборами связанных между собой процессов, а вторые - групп. Понятие «задание» часто используется в качестве синонима понятия группы процессов.

> Группа процессов — это набор из одного или нескольких процессов с одним и тем же идентификатором PGID (process group identifier - идентификатор группы процессов).

В каждой группе есть свой лидер - процесс, который создал эту группу и чей идентификатор становится ее PGID. Новый процесс наследует PGID своего родителя.

Группа процессов имеет жизненный цикл - период времени, который начинается с создания группы лидером и заканчивается, когда группу покидает ее последний участник. Чтобы покинуть группу, процесс должен либо завершиться, либо присоединиться к другой группе. Лидер группы процессов может не быть ее последним участником.

> Сессия — это набор групп процессов.

Членство процесса в сессии определяется идентификатором SID (session identifier — идентификатор сессии). Лидером сессии является процесс, который её создал, и чей идентификатор используется в качестве SID. Новый процесс наследует идентификатор SID своего родителя.

Все процессы в сессии разделяют один и тот же контролирующий терминал, который устанавливается при первом открытии устройства терминала лидером сессии. Любой контролирующий терминал может быть связан не более чем с одной сессией.

В любой момент времени одна из групп процессов в сессии является активной, а остальные — фоновыми. Только процессы из активной группы могут считывать ввод из контролирующего терминала. Сигнал, сгенерированный пользователем в терминале с помощью комбинации клавиш, передается всем участникам активной группы процессов. В число этих комбинаций входят прерывание (обычно Ctrl+C), которое генерирует сигнал SIGINT, выход (обычно Ctrl+\), генерирующий сигнал SIGQUIT, и приостановка (обычно Ctrl+Z), передающая сигнал SIGTSTP.

В результате открытия контролирующего терминала лидер сессии становится контролирующим процессом этого терминала. Главной особенностью такого процесса является то, что при утрате соединения с терминалом ядро отправляет ему сигнал SIGHUP.

Идентификаторы PGID и SID процесса можно посмотреть в файле /proc/PID/stat. Мы также можем определить идентификатор устройства контролирующего терминала (имеющий вид десятичного числа, состоящего из основного и дополнительного идентификаторов) и идентификатор его процесса.

Основное назначение групп процессов и сессий связано с управлением заданиями в командной оболочке. Рассмотрим пример. Когда пользователь входит в систему в интерактивном режиме, он задействует контролирующий терминал. Командная оболочка, задействованная при этом, становится лидером сессии, контролирующим процессом для терминала, а также единственным участником своей собственной группы процессов. Любые команды или конвейеры команд, запущенных из этой оболочки, становятся новыми процессами и помещаются в новую группу (изначально они являются единственными ее участниками, однако в нее также попадают потомки этих процессов). Команды или конвейеры, после которых указывается амперсанд (&), попадают в фоновую группу процессов. В противном случае их группа будет активной. Все процессы, созданные во время терминальной сессии, являются ее частью.

Когда мы работаем в графическом интерфейсе, контролирующим является псевдотерминал, и каждое его окно имеет отдельную сессию, лидером и контролирующим процессом которой становится его командная оболочка.

Группы процессов иногда можно применять не только для управления заданиями. Они имеют два полезных свойства: родительский процесс может ожидать любого из своих потомков в определенной группе, и сигнал может быть отправлен сразу всем участникам группы.

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX Разработка TUNE-IT

10/13 Урок 9. Виртуальная файловая система и каналы

текущий урок

В прошлом модуле мы говорили, что в ядре есть слой виртуальной файловой системы, которая скрывает для процессов детали реализации конкретных файловых систем. В этом уроке мы рассмотрим, как же процесс получает доступ к файлу.

Ядро поддерживает три структуры данных (можно называть их таблицами) для эффективной работы процессов с файлами:

- Таблицу дескрипторов файлов для каждого процесса.
- Общесистемную таблицу открытых файлов.
- Таблицу индексных дескрипторов файловой системы.

Для каждого процесса ядро поддерживает таблицу дескрипторов открытых файлов. Каждая запись в этой таблице содержит информацию об одном файловом дескрипторе, включая:

- Флаг закрытия при выполнении close-on-exec.
- Ссылку на дескриптор открытого файла.

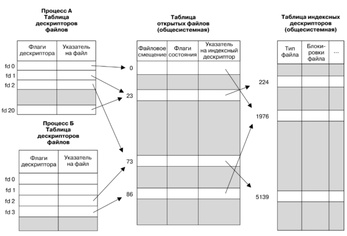
Ядро обслуживает общесистемную таблицу всех дескрипторов открытых файлов, также называемую таблицей открытых файлов. В дескрипторе открытого файла хранится вся информация, относящаяся к открытому файлу, включая:

- указатель чтения-записи, также называемое текущем файловым смещением (обновляемое системными вызовами read() и write() или явно изменяемое с помощью системного вызова lseek());
- флаги открытия файла (то есть аргумент flags системного вызова open());
- режим доступа к файлу (только для чтения, только для записи или для чтения и записи, согласно установкам для системного вызова open());
- установки, относящиеся к вводу-выводу, управляемому сигналами;
- ссылку на индексный дескриптор для этого файла.

Когда происходит открытие файла, копия его индексного дескриптора помещается в таблицу индексных дескрипторов открытых файлов. Для каждого файла там содержится следующая информация:

- тип файла (например, обычный файл, сокет или FIFO-устройства) и права доступа;
- указатель на список блокировок, удерживаемых на этом файле;
- разные свойства файла, включая его размер и метки времени, связанные с различными типами файловых операций.

Связь между таблицами показана на рисунке. На этой схеме у двух процессов имеется несколько дескрипторов открытых файлов.



linux-m3-9-1

В процессе А два дескриптора (1 и 20) ссылаются на один и тот же дескриптор открытого файла (номер 23). Такая ситуация может возникать в результате вызова dup(), dup2() или fcntl(). Дескриптор 2 процесса А и дескриптор 2 процесса Б ссылаются на один и тот же файловый дескриптор (73). Этот сценарий может сложиться после вызова fork() (то есть процесс А является родительским по отношению к процессу Б или наоборот) либо при условии, что один процесс передал открытый дескриптор другому процессу, используя доменный сокет UNIX. Дескриптор 0 процесса А и дескриптор 3 процесса Б ссылаются на различные дескрипторы открытых файлов, но эти дескрипторы ссылаются на одну и ту же запись в таблице индексных дескрипторов (1976), то есть на один и тот же файл. Дело в том, что каждый процесс независимо вызвал open() для одного и того же файла. Похожая ситуация может возникнуть, если один и тот же процесс дважды откроет один и тот же файл.

Каналы являются наиболее старым средством межпроцессного взаимодействия в UNIX. Этот механизм предоставляет элегантное решение распространенной задачи: имея два процесса, выполняющие две разные программы (команды), нужно сделать так, чтобы командная оболочка направила вывод одного из них на ввод другого.

Каналы могут использоваться для передачи данных только между родственными процессами.

Рассмотрим пример:

```
$ ls | wc -l
```

Для выполнения данной команды командная оболочка создает два процесса и выполняет в них соответственно программы ls и wc (это делается с помощью вызовов fork() и exec()). На рисунке показано, как оба процесса используют канал.

Также на рисунке видно, почему каналы (от англ. pipe — «труба») получили такое название. Мы можем рассматривать их как фрагмент трубы, которая позволяет пропускать через себя потоки данных, направляя их от одного процесса к другому.

linux-m3-9-2

Обратите внимание на способ подключения процессов к каналу: тот, что записывает данные (ls), соединяет свой стандартный вывод (файловый дескриптор 1 - fd1) с записывающим концом канала, а считывающий процесс (wc) подключает свой стандартный ввод (файловый дескриптор 0 - fd0) к его другому концу, предназначенному для чтения. Как следствие, ни один из процессов не знает о существовании канала - они просто используют стандартные файловые дескрипторы для чтения и записи. Создаёт такую конструкцию командная оболочка.

Канал - это поток байтов.

Канал представляет собой поток байтов. Другими словами, при передачи данных через канал не существует понятия сообщений или их границ. Процесс, считывающий данные из канала, может прочитать блок любого размера, независимо от того, насколько большой блок был туда записан другим процессом. Кроме того, данные проходят через канал последовательно, т.е. байты считываются из него в том же порядке, в котором они были записаны. Произвольный доступ к содержимому канала с помощью вызова lseek() невозможен.

С помощью файловой системы *proc* можно увидеть таблицу дескрипторов файлов процесса (директория /proc/<pid>/fd), а также соответствующие элементы общесистемной таблицы открытых файлов (директория /proc/<pid>/fdinfo/). Рассмотрим пример.

В данном примере программа ls запускается с аргументом '`/proc/self/fd`', а стандартные потоки вывода и сообщений об ошибках перенаправляются в файлы `/tmp/ls.out` и `/tmp/ls.err` соответственно.

```
$ ls -l /proc/self/fd > /tmp/ls.out 2> /tmp/ls.err
$ cat /tmp/ls.out
  total 0
  lrwx------ 1 user1 user1 64 Feb 14 00:09 0 -> /dev/pts/0
  l-wx------ 1 user1 user1 64 Feb 14 00:09 1 -> /tmp/ls.out
  l-wx------ 1 user1 user1 64 Feb 14 00:09 2 -> /tmp/ls.err
  lr-x------ 1 user1 user1 64 Feb 14 00:09 3 -> /proc/12915/fd
```

Как видно, в таблице дескрипторов файлов процесса ls будет четыре элемента: дескриптор 0 - это стандартный ввод, направленный из терминала, дескриптор 1 - это стандартный вывод, перенаправленный в файл /tmp/ls.out, а дескриптор 2 - это стандартный поток сообщений об ошибках, перенаправленный в файл /tmp/ls.err. Кроме того, у процесса ls есть ещё дескриптор 3, который соответствует открытой директории `/proc/self/fd`, переданной в качестве аргумента команде `ls`.

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026.

Дизайн MAX Разработка TUNE-IT

11/13 **Тестирование. Процессы**

текущий урок

Тестирование — до 13 апр. 2026 г. 23:59 MSK Завершено

Задание 1.1

0.5/0.5 points (graded)

Какие области памяти процесс-родитель и процесс-потомок используют совместно?

- (•) Сегмент текста
- ( ) Сегмент инициализированных данных
- ( ) Куча
- ( ) Стек
- ( ) Все

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.2

0.5/0.5 points (graded)

Что при создании наследует процесс-потомок от процесса-родителя?

- [x] Открытые файлы
- [x] Идентификатор пользователя (UID)
- [ ] Таймеры
- [ ] Идентификатор процесса (PID)

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.3

0.5/0.5 points (graded)

Где хранится значение umask?

- (•) В дескрипторе процесса
- ( ) В правах доступа к файлу
- ( ) В индексном дескрипторе файла

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.4

0.5/0.5 points (graded)

Где хранится настройка по умолчанию для срока действия аккаунта?

- ( ) /etc/login.defs
- (•) /etc/default/useradd
- ( ) /etc/passwd

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.5

0.5/0.5 points (graded)

В каком пространстве расширенных атрибутов хранятся привилегии файлов?

- ( ) system
- ( ) user
- ( ) trusted
- (•) security

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.6

0.5/0.5 points (graded)

Для чего нужны эффективные привилегии файла?

- (•) Для программ, не использующих механизм привилегий (capability-dumb)
- ( ) Для ограничения разрешённых привилегий процесса после выполнения execve()
- ( ) Для совместимости с механизмом set-user-id
- ( ) Нет правильного варианта

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.7

0.0/0.5 points (graded)

Какие привилегии позволяют преодолевать дискреционный контроль доступа?

- [ ] CAP_AUDIT_CONTROL
- [x] CAP_CHOWN
- [ ] CAP_SETUID
- [ ] CAP_NET_RAW

✖

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.8

0.5/0.5 points (graded)

Сколько будет открытых файлов у процесса: `ls -l /proc/self/fd > /tmp/ls.out` ?

- ( ) 1
- ( ) 2
- ( ) 3
- (•) 4

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.9

0.5/0.5 points (graded)

В какой таблице ядра хранится указатель чтения-записи открытого файла?

- ( ) В таблице дескрипторов файлов процесса
- (•) В общесистемной таблице открытых файлов
- ( ) В таблице индексных дескрипторов

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.10

0.0/0.5 points (graded)

К каким средствам IPC относятся каналы?

- ( ) средства взаимодействия
- ( ) средства синхронизации
- (•) средства и взаимодействия и синхронизации

✖

Show answer

Отправить You have used 1 of 1 attempt

‹ Предыдущий урок | Следующий урок ›

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026.

Дизайн MAX. Разработка TUNE-IT

12/13 Заключение

текущий урок

В данном модуле мы познакомились с концепцией процесса, которые наряду с файлами представляют собой вторую основную системную сущность. Таким образом, за первые три модуля мы изучили такие базовые механизмы операционной системы, как хранение данных в файлах и запуск программ на исполнение, а также посмотрели на работу многопользовательской ОС с точки зрения информационной безопасности - а именно изучили механизм дискреционного управления доступом.

Начиная со следующего модуля мы начнём рассматривать вопросы администрирования ОС GNU/Linux. Изучение задач, которые выполняет системный администратор, позволит вам самостоятельно настраивать и поддерживать в актуальном состоянии рабочее место на основе дистрибутива GNU/Linux.

Резюме

В этом модуле мы изучили следующие команды:

ps - вывод информации о процессах

kill - посылка сигнала процессу

getcap - вывести привилегии файла

setcap - установить привилегии файла

getpcaps - вывести привилегии процесса

В этом модуле мы изучили следующие системные файлы:

/proc/<pid>/status - содержимое дескриптора процесса

/proc/sys/kernel/pid_max - максимально возможное количество идентификаторов процессов

/proc/sys/kernel/cap_last_cap - количество поддерживаемых привилегий

/proc/<pid>/fd/ - таблица дескрипторов файлов процесса

/proc/<pid>/fdinfo/ - элементы общесистемной таблицы открытых файлов

Источники

- Ефанов, Д. В. Базовые механизмы защиты ядра Linux : учебное пособие / Д. В. Ефанов. — Москва : НИЯУ МИФИ, 2022. — 192 с. — ISBN 978-5-7262-2924-9. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/355520 (дата обращения: 22.01.2026). — Режим доступа: для авториз. пользователей.
- Керриск Майкл. Linux API. Исчерпывающее руководство. — СПб.: Питер, 2019.
- Стивенс Р., Раго С. UNIX. Профессиональное программирование, 2е издание. – СПб.: Символ Плюс, 2007. – 1040 с.
- Бовет Д., Чезати М. Ядро Linux, 3-е изд. — СПб.: БХВ-Петербург, 2007. — 1104 с.
- ГОСТ Р 50922-96. Защита информации. Основные термины и определения. М.: Издательство стандартов, 1996.
- ГОСТ 34.11-2018. Информационная технология. Криптографическая защита информации. Функция хэширования. М.: Стандартинформ, 2018.
- Pro Bash Programming: Scripting the GNU/Linux Shell. Chris Johnson. Apress, 2009.

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX Разработка TUNE-IT

1/8 Введение

текущий урок

Модуль посвящён рассмотрению вопросов, связанных с настройкой начальной загрузки системы, а также с обеспечением её нормального функционирования при добавлении новых устройств хранения и нового ПО.

Как полученные знания пригодятся в профессиональном развитии

В процессе работы пользователь обязательно сталкивается с задачами добавления новых дисков в компьютер или подключения внешних устройств хранения (например, USB-дисков), а также с подключением дополнительных репозиториев пакетов, установкой нового ПО или обновлением существующего ПО. Кроме того, современные дистрибутивы GNU/Linux для повышения безопасности используют систему SELinux, которую в некоторых случаях также нужно настраивать, например, при создании нового сайта и запуске web-сервера. Данные задачи рассматриваются в данном модуле.

Цели и результат модуля

Студент, освоивший программу четвёртого модуля, должен:

1. Понимать этапы загрузки системы.
2. Понимать назначение блочных устройств и уметь создавать новые файловые системы.
3. Уметь подключать и отключать файловые системы.
4. Уметь подключать дополнительные репозитории пакетов и искать необходимое для работы ПО.
5. Уметь устанавливать новое ПО, а также обновлять существующее ПО.
6. Понимать принцип работы SELinux.
7. Уметь настраивать режимы работы SELinux, как при начальной загрузке, так и в процессе нормальной работы (на лету).
8. Уметь настраивать контексты безопасности SELinux при создании нового содержимого сайта и запуске web-сервера Apache.

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX Разработка TUNE-IT

Блочные устройства

Блочные устройства — это устройства, работа с данными в которых осуществляется порциями фиксированного размера, называемыми блоками. Характерным примером таких устройств являются жесткие диски, твердотельные накопители и флеш-память.

В новом основном уроке узнать, на каких устройствах основана концепция виртуальной памяти, как работает подкачка страниц и как ускорить работу с диском.

- Научиться определять, насколько активно используется оперативная память, на которую приходится диск
- В зависимости от ситуации подключать жесткие диски, твердотельные накопители или флеш-память

Теперь вы можете с уверенностью сказать, что знаете, как работает виртуальная память, и понимаете, как ее можно ускорить.

Все блочные устройства работают по общему принципу: они предоставляют доступ к данным, которые разбиты на блоки фиксированного размера.

Обычно весь жесткий диск делится на одну или несколько областей, называемых разделами. Разделение диска на разделы позволяет, например, создавать отдельные разделы для операционной системы и для пользовательских данных. Это позволяет переустанавливать операционную систему, не затрагивая пользовательские данные.

Каждый раздел имеет свою файловую систему. Файловая система — это способ организации данных на диске. Она определяет, как данные будут храниться и как к ним будет осуществляться доступ.

Для работы с файловой системой, любой ее операционной системе необходимо, чтобы раздел был смонтирован. Монтирование — это процесс, при котором файловая система раздела становится доступной для операционной системы. Обычно монтирование происходит автоматически при загрузке системы.

Монтирование можно выполнить с помощью команды `mount`. Например, чтобы смонтировать раздел `/dev/sda1` в каталог `/mnt`, нужно выполнить следующую команду:

```
# mount /dev/sda1 /mnt
```

Для отсоединения файловой системы используется команда `umount`. Например, чтобы отсоединить раздел, смонтированный в каталог `/mnt`, нужно выполнить следующую команду:

```
# umount /mnt
```

Отсоединение файловой системы может потребоваться, например, при обновлении операционной системы или при замене жесткого диска. Также отсоединение может быть полезно, если вы хотите предотвратить доступ к данным на разделе, пока он не смонтирован.

Команда `mount` без аргументов выводит список всех смонтированных файловых систем. Например:

```
$ mount
/dev/sda1 on / type ext4 (rw,relatime)
/dev/sda2 on /home type ext4 (rw,relatime)
/dev/sda3 on /boot type ext4 (rw,relatime)
```

Здесь видно, что раздел `/dev/sda1` смонтирован в каталог `/`, раздел `/dev/sda2` — в каталог `/home`, а раздел `/dev/sda3` — в каталог `/boot`.

Монтирование файловых систем можно автоматизировать с помощью файла `/etc/fstab`. Этот файл содержит информацию о том, какие разделы и куда должны быть смонтированы при загрузке системы. Например, чтобы автоматически смонтировать раздел `/dev/sda1` в каталог `/mnt`, нужно добавить в файл `/etc/fstab` следующую строку:

```
/dev/sda1 /mnt ext4 defaults 0 0
```

Здесь `/dev/sda1` — это раздел, который нужно смонтировать, `/mnt` — каталог, в который нужно смонтировать раздел, `ext4` — тип файловой системы, `defaults` — параметры монтирования по умолчанию, `0` — флаг, который указывает, что раздел не нужно проверять при загрузке системы, и `0` — флаг, который указывает, что раздел не нужно монтировать при загрузке системы.

Файловая система — это способ организации данных на диске. Она определяет, как данные будут храниться и как к ним будет осуществляться доступ. Существует множество различных файловых систем, каждая из которых имеет свои преимущества и недостатки.

Некоторые из наиболее популярных файловых систем:

- **ext4** — это стандартная файловая система для многих дистрибутивов Linux. Она обеспечивает хорошую производительность и надежность.
- **XFS** — это файловая система, которая хорошо подходит для работы с большими файлами и большими объемами данных.
- **Btrfs** — это современная файловая система, которая предлагает такие функции, как снимки состояния файловой системы и встроенное сжатие данных.
- **NTFS** — это файловая система, которая используется в операционных системах Windows.
- **FAT32** — это старая файловая система, которая используется в основном для флеш-накопителей и других небольших устройств.

Выбор файловой системы зависит от ваших потребностей. Если вам нужна хорошая производительность и надежность, то ext4 — это хороший выбор. Если вам нужно работать с большими файлами, то XFS — это хороший выбор. Если вам нужны такие функции, как снимки состояния файловой системы и встроенное сжатие данных, то Btrfs — это хороший выбор.

Для создания файловой системы используется команда `mkfs`. Например, чтобы создать файловую систему ext4 на разделе `/dev/sda1`, нужно выполнить следующую команду:

```
# mkfs -t ext4 /dev/sda1
```

Здесь `-t ext4` — это параметр, который указывает, что нужно создать файловую систему ext4, а `/dev/sda1` — это раздел, на котором нужно создать файловую систему.

Команда `mkfs` может принимать и другие параметры, которые позволяют настроить файловую систему. Например, параметр `-L` позволяет задать метку тома для файловой системы. Метка тома — это имя, которое будет отображаться при монтировании файловой системы.

Например, чтобы создать файловую систему ext4 на разделе `/dev/sda1` с меткой тома `data`, нужно выполнить следующую команду:

```
# mkfs -t ext4 -L data /dev/sda1
```

Для монтирования файловой системы используется команда `mount`. Например, чтобы смонтировать раздел `/dev/sda1` в каталог `/mnt`, нужно выполнить следующую команду:

```
# mount /dev/sda1 /mnt
```

Для монтирования файловой системы с меткой тома можно использовать параметр `-L`. Например, чтобы смонтировать раздел с меткой тома `data` в каталог `/mnt`, нужно выполнить следующую команду:

```
# mount -L data /mnt
```

Для автоматического монтирования файловой системы при загрузке системы нужно добавить соответствующую строку в файл `/etc/fstab`.

Файл `/etc/fstab` содержит статическую информацию о файловых системах. Эта информация используется командой `mount`. Когда вы запускаете `mount` без аргументов, она просматривает `/etc/fstab` в поисках доступных для монтирования систем. Параметры, указанные в `/etc/fstab`, используются, если вы не предоставляете другие параметры при вызове `mount`.

Каждая файловая система описывается в отдельной строке, состоящей из пяти полей, разделенных пробелами или табуляцией. Строки, начинающиеся с символа `#`, рассматриваются как комментарии. Вот описание, что представляет собой каждое из полей, в порядке их появления в строке:

1. Первое поле (`fs_spec`) описывает специальное блочное устройство, файловую систему, каталог или удаленную файловую систему, которые должны быть смонтированы. Обычно это имя блочного специального устройства, которое необходимо форматировать.
2. Второе поле (`fs_file`) описывает точку монтирования для данной файловой системы. Для файла подкачки это поле должно быть установлено в `swap`.
3. Третье поле (`fs_vfstype`) описывает тип файловой системы. Linux может распознавать многие типы файловых систем, такие как `ext4` или `btrfs`. Также существует специальный тип `auto`, который предполагает автоматическое определение типа файловой системы. Это может быть полезно для съемных носителей, которые могут использоваться в различных системах.
4. Четвертое поле (`fs_mntops`) описывает параметры монтирования. Они могут включать `defaults`, который включает набор параметров по умолчанию, таких как `rw`, `suid`, `dev`, `exec`, `auto`, `nouser` и `async`. Обратитесь к странице руководства `mount(8)` для получения информации о других параметрах, специфичных для каждой файловой системы.
5. Пятое поле (`fs_freq`) используется командой `dump` для определения того, какие файловые системы нуждаются в резервном копировании. Это может быть `0`, чтобы `dump` игнорировал файловую систему, или `1`, чтобы `dump` выполнял резервное копирование файловой системы.
6. Шестое поле (`fs_passno`) используется программой `fsck` для определения порядка, в котором файловые системы должны быть проверены. Если шестое поле не указано или установлено в `0`, `fsck` не будет выполнять проверку файловой системы. Корневая файловая система должна иметь наивысший приоритет `1`, а другие файловые системы — более низкие приоритеты (`2` или больше).

Вот пример файла `/etc/fstab`:

```
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# <file system> <mount point>   <type>  <options>       <dump>  <pass>
proc            /proc           proc    nodev,noexec,nosuid 0       0
# / was on /dev/sda1 during installation
UUID=1234-5678  /               ext4    errors=remount-ro 0       1
# /boot was on /dev/sda2 during installation
UUID=abcd-efgh  /boot           ext4    defaults        0       2
# swap was on /dev/sda3 during installation
UUID=ijkl-mnop  none            swap    sw              0       0
/dev/fd0        /media/floppy0  auto    rw,user,noauto,exec,utf8 0       0
```

Здесь:

- `proc` монтируется в `/proc` с типом `proc` и опциями `nodev,noexec,nosuid`.
- `/dev/sda1` монтируется в `/` с типом `ext4` и опцией `errors=remount-ro`.
- `/dev/sda2` монтируется в `/boot` с типом `ext4` и опцией `defaults`.
- `/dev/sda3` используется как `swap`.
- `/dev/fd0` монтируется в `/media/floppy0` с типом `auto` и опциями `rw,user,noauto,exec,utf8`.

Файл `/etc/fstab` позволяет настроить автоматическое монтирование файловых систем при загрузке системы. Это может быть полезно для серверов, которые должны быть всегда доступны, или для настольных компьютеров, которые должны автоматически монтировать съемные носители.

Для монтирования файловой системы с меткой тома можно использовать параметр `-L`. Например, чтобы смонтировать раздел с меткой тома `data` в каталог `/mnt`, нужно выполнить следующую команду:

```
# mount -L data /mnt
```

Для монтирования файловой системы с UUID можно использовать параметр `-U`. Например, чтобы смонтировать раздел с UUID `1234-5678` в каталог `/mnt`, нужно выполнить следующую команду:

```
# mount -U 1234-5678 /mnt
```

Для автоматического монтирования файловой системы с меткой тома или UUID

Пакеты RPM

Как мы говорили в первом модуле, простые программы могут представлять собой один исполняемый файл, однако сложные программные продукты включают в себя множество файлов, например:

- исполняемые файлы
- разделяемые библиотеки
- конфигурационные файлы
- файлы с документацией, например для справочной системы man

Кроме того, установка программ обычно требует выполнения ряда административных действий, например создания директорий или пользователей, в случае сервера, удаление программ требует проверки системы в первоначальное исходное состояние. Самым "затратным" обычно является их временная зависимость, о таких учёт разных версий ещё в ней не программах.

Чтобы программами было легче управлять, их файлы вместе с инструкциями по сборке, установке и удалению упаковали в специальный архив, называемые пакетом.

В данном уроке мы рассмотрим широко используемый менеджер пакетов RPM.

Менеджер пакетов RPM, первоначально разработанный компанией Red Hat, предоставляет стандартный способ для распространения ПО. Управление программным обеспечением в виде пакетов RPM намного проще, чем работа с программами обособленно, просто извлечением из архива. Он позволяет системному администратору отслеживать, какие файлы для установленных пакетов, а какие необходимо удалять при его деинсталляции, и проверять наличие необходимых пакетов при его установке, предотвращая проблемы, которые возникают в локальной базе данных RPM.

Именно файлов пакетов RPM состоит из четырех элементов [плюс суффикс .rpm] в формате: имя-версия-релиз.архитектура

Например:

coreutils-9.5-11.fc41.x86_64, где

- имя — одно или несколько слов, описывающих содержимое (coreutils)
- версия — номер версии исходного программного обеспечения (9.5)
- релиз — номер выпуска пакета, основанного на данной версии, - устанавливается мейнтейнером пакета, который может не являться разработчиком исходного программного обеспечения (11.fc41)
- архитектура — архитектура процессора, для которого предназначен данный скомпилирован пакет. пакетов (архитектура этого пакета не зависит от архитектуры (в отличие от x86_64 для 64-битных систем, каталог для 64-битных ARM и т.д.)

Для установки пакетов из репозиториев обычно используется команда. Если существует несколько версий, установливается пакет с более высоким номером версии. Если версии одинаковые релизов одной и той же версии, установливается пакет с более высоким номером релиза.

Каждый RPM-пакет представляет собой специальный архив, состоящий из трех компонентов:

- Файлы, устанавливаемые пакетом.
- Информация о пакете (метаданные), такая как название, версия, релиз и архитектура, список изменений пакета, требуется на установку других пакетов, лицензия/права, журнал изменений пакета, и другие подробности.
- Скрипты, которые могут запускаться при установке, обновлении или удалении данного пакета, или дополнительные данные, необходимые для работы самого пакета.

Как правило, поставщики программного обеспечения подписывают пакеты RPM цифровой подписью с использованием ключей GPG. Система RPM затем проверяет целостность пакета, подтверждая, что ключ был подписан соответствующим ключом GPG. Система RPM отказывается устанавливать пакет, если подпись GPG не совпадает.

В большинстве случаев одновременно может быть установлена только одна версия или релиз пакета. Однако, если пакет собран таким образом, что нет конфигурации имен файлов, то можно установить несколько версий. Небольшим ярким примером такого пакета является само ядро ОС. Поскольку новое ядро можно протестировать только путем загрузки с этим ядром, пакет специально разработан таким образом, чтобы можно было установить несколько версий одновременно. Если новое ядро не загружается, старое ядро по-прежнему остаётся доступным и загружаемым.

Для работы с RPM-пакетами используется утилита rpm.

Утилита rpm — это мощный инструмент, который может получить информацию о содержимом пакетов и установленных пакетов. По умолчанию она получает информацию из локальной базы данных установленных пакетов. Однако вы можете использовать параметр -p, чтобы указать, что вместо этого вы хотите информацию о загруженном файле пакета. Это может понадобиться, чтобы проверить содержимое файла пакета перед его установкой.

Общая форма запроса:

```
rpm -q [опции] [имя пакета или файла]
```

Общая информация об установленных пакетах:

rpm -qa: Список всех установленных пакетов

rpm -qf имя_файла: Узнать, какой пакет предоставляет имя_файла

```
$ rpm -qf /etc/yum.repos.d
fedora-repos-41.1-noarch
```

Информация о конкретном пакете:

rpm -q: Список версий пакета, установленных в данный момент

```
$ rpm -q dnf5
dnf5-5.2.13.1-1.fc41.x86_64
```

rpm -qi: Показать подробную информацию о пакете

rpm -ql: Список файлов, установленных пакетом.

```
$ rpm -ql openssh-clients
/etc/ssh/ssh_config
/etc/ssh/ssh_config.d/50-redhat.conf
```

rpm -qd: Вывести список только документации, установленные пакетом.

rpm -q --scripts: Вывести список содержимое оболочки, которые выполняются до или после установки или удаления пакета.

rpm -q --changelog: Вывести информацию об изменениях в пакете.

Если нужно выполнить запрос о пакете, который не установлен, а доступен локально (скачан) в виде rpm, то нужно добавить опцию -p.

Команда rpm также может использоваться для установки пакета RPM, который вы загрузили в систему - для этого нужно использовать опцию -ivh. Однако, на практике применяется более мощный инструмент для управления установкой и обновлением RPM-пакетов из командной строки — утилита dnf.

Можно извлечь файлы из RPM-пакета без его установки. Утилита cpio2rpm может передать содержимое RPM-пакета специальному инструменту архивирования, называемому cpio, который может извлечь весь пакет или отдельные файлы. Для этого перенаправьте вывод команды cpio2rpm имя_пакета.rpm в команду cpio -id, чтобы извлечь все файлы, хранящиеся в RPM-пакете.

Репозитории пакетов

Низкоуровневая команда rpm может использоваться для установки пакетов, но она не предназначена для работы с разными пакетами или автоматического разрешения зависимостей из нескольких источников. Более совершенной системой для управления установкой и обновлением программного обеспечения на основе RPM является утилита dnf.

Команда dnf позволяет устанавливать, обновлять, удалять и получать информацию о пакетах и их зависимостях. Вы можете получить полную информацию, используя команду dnf --help.

Поиск программного обеспечения с помощью dnf

dnf help отображает информацию об использовании.

dnf list отображает установленные и доступные пакеты. Пример:

```
$ dnf list *http*
Installed packages
httpd.x86_64                          2.4.62-5.fc41                 @updates
httpd-core.x86_64                     2.4.62-5.fc41                 @updates
httpd-filesystem.noarch               2.4.62-5.fc41                 @updates
httpd-tools.x86_64                    2.4.62-5.fc41                 @updates

Available packages
http-parser.i686                      2.9.4-12.fc41                 fedora
http-parser.x86_64                    2.9.4-12.fc41                 fedora
http-parser-devel.i686                2.9.4-12.fc41                 fedora
```

dnf search ключевое_слово перечисляет пакеты по ключевым словам, найденным только в поле имени и описания. Для поиска пакетов, в имени, описании и краткое описание которых содержится слово имеет использовать опцию --all. Однако, на практике применяется более мощный инструмент для управления установкой и обновлением RPM-пакетов из командной строки — утилита dnf.

```
$ dnf search all 'web server'
Matched fields: summary
mod_log_sql.x86_64 : Module for the Apache web server to log all HTTP POST messages
php.x86_64 : A small web server that hosts ASP.NET
```

Команда dnf info имя_пакета возвращает подробную информацию о пакете, включая дисковое пространство, необходимое для установки. Чтобы получить информацию о сервере Apache HTTP выполните:

```
$ dnf info httpd
Updating and loading repositories:
Repositories loaded.
Installed packages
Name         : httpd
Epoch        : 0
Version      : 2.4.63
Release      : 1.fc41
Architecture : x86_64
Install size : 64.8 KiB
Source       : httpd-2.4.63-1.fc41.src.rpm
User-space repositories
Summary      : Apache HTTP Server
URL          : https://httpd.apache.org/
License      : Apache-2.0 AND (BSD-3-Clause AND metanail AND MPAD-uuil-variant AND
Spancer-94
             : )
Description  : The Apache HTTP Server is a powerful, efficient, and extensible
             : web server.
Vendor       : Fedora Project
```

команда dnf provides путь_к_файлу отображают пакеты, соответствующие указанному пути (которой часто содержит символы подстановки). Чтобы найти пакеты, предоставляющие доступ к каталогу /usr/share/html, используйте:

```
$ dnf provides /usr/share/html
httpd-filesystem-2.4.57-7.noarch : The basic directory layout for the Apache HTTP server
```

Установка и удаление пакетов

команда dnf install имя_пакета получает и устанавливает пакет программного обеспечения, включая все зависимости.

команда dnf update имя_пакета получает и устанавливает более новую версию указанного пакета, включая все зависимости. Как правило, процесс пытается сохранить файлы конфигурации на месте, но иногда они могут быть перемещены, если разработчик пакета считает, что старый файл не будет работать после обновления. Если имя_пакета не указан, устанавливаются все соответствующие обновления.

Поскольку новое ядро можно протестировать только загрузившись с этого ядра, это пакет специально разработан таким образом, чтобы можно было установить несколько версий одновременно. Если новое ядро не загружается, старое ядро по-прежнему доступно. Использование команды dnf update вместо dnf гарантирует установку только последнего ядра. Конфигурация содержит список ядер, которые всегда должны быть установлены, даже если одновременно загружаемый только одно ядро.

Используйте команду dnf list kernel, чтобы вывести список всех установленных и доступных ядер. Чтобы просмотреть текущие загруженное ядро, используйте команду uname. Опция -r отображает только версию и релиз ядра, а опция -a — релиз ядра и дополнительную информацию.

```
$ uname -r
6.11.4-301.fc41.x86_64
$ uname -a
Linux localhost.localdomain 6.11.4-301.fc41.x86_64 #1 SMP PREEMPT_DYNAMIC Sun Oct 20 15:27:34 UTC 2024 x86_64 GNU/Linux
```

Команда dnf remove имя_пакета удаляет установленный пакет, включая все зависящие от него пакеты.

Команда dnf remove удалит все перечисленные пакеты, а также все пакеты, которые зависят от пакетов, которые удаляются (т.е. перечислены в команде), а также все пакеты, которые зависят от этих пакетов и т.д. Это может привести к неожиданному удалению пакетов, о которых вы даже не думали, поэтому внимательно проверяйте список удаляемых пакетов!

Установка и удаление группы пакетов

Команда dnf также поддерживает концепцию группы пакетов, которая представляет собой наборы связанного программного обеспечения, устанавливаемого вместе в виде группы.

Подобно dnf list, команда dnf group list отображает доступных групп.

```
$ dnf group list
ID                                    Name                                    Installed
libreoffice                           libreoffice                             yes
development-tools                     Development Tools                       yes
c-development                         C Development Tools and Libraries       no
container-management                  Container Management                    yes
```

Команда dnf group info отображает информацию о группе.

```
$ dnf group info "RPM Development Tools"
Id                  : rpm-development-tools
Name                : RPM Development Tools
Description         : These tools include core development tools such rpmbuild.
Installed           : no
Order               :
Langonly            :
Translated          : yes
Repositories        : fedora, updates
Mandatory packages  : dnf5dnf-plugin-config
                    : rpmdevtools
Default packages    : bzr
                    : mock
                    : rpmlint
Optional packages   : createrepo_c
                    : pykgl
                    : zyp5.int
```

Команда dnf group install устанавливает группу.

Просмотр истории транзакций

Все операции установки и удаления регистрируются в файле /var/log/dnf.rpm.log.

Команда dnf history list выведет историю операций установки и удаления пакетов.

```
$ sudo dnf history list
ID     Command line                                  Date and time    Action(s)
Altered
22 dnf install nevim                                 2024-02-13 21:38:34
17
```

Команда dnf history list выведет детальную информацию по данной операции.

```
$ sudo dnf history info 22
Transaction ID : 22
Begin time     : 2024-02-13 21:38:34
Begin rpmdb    : 3226649445:f9f991a5eab5:f95c584e5d525da513a5f857c71da6d7ae8e4d827f
End time       : 2024-02-13 21:38:34
End rpmdb      : 0b40a8a4bdaa5ea6b9985e1ee7f3051282c414859fa24c3e3dc7b9c41ab5b422
User           : DNF5 API <defaults>
Status         : Ok
Releasever     : 41
Command Line   : dnf install nevim
Comment        :
Packages altered:
  Install  gpm-libs-1.20.7-40.fc41.x86_64                  @System
  Install  nevim-0:0.10.4-1.fc41.x86_64                    @updates
  Install  libluv-0:1.48.0-2.fc41.x86_64                   fedora
  Install  libtermkey-0:0.22-20.fc41.x86_64                fedora
  Install  libvterm-0:0.3.3-4.fc41.x86_64                  fedora
  Install  lua5.1-lpeg-0:1.1.0-2.fc41.x86_64               fedora
```

Подкоманда history undo отменяет все действия от указанной транзакции.

← Предыдущий урок Следующий урок →

Этапы загрузки

Не существует единого определения начальной загрузки, так как для разных аппаратных платформ и информационных систем, построенных на их основе, критерии достижения системы работоспособного состояния будут разными. Поэтому рассмотрим этапы начальной загрузки на примере платформы x86-64 с поддержкой UEFI и дистрибутива GNU/Linux с использованием systemd для запуска сервисов.

Можно выделить пять последовательных этапов в начальной загрузке:

1. Работа уровня аппаратной платформы.
2. Работа загрузчика операционной системы.
3. Работа ядра.
4. Работа корневого процесса (systemd).
5. Работа скриптов загрузки сервисов.

Этап 1. Аппаратная часть

После включения питания или полной перезагрузки управление передается программе, хранящейся в постоянной памяти (обычно ПЗУ). Эта программа обычно выполняет базовую самодиагностику машины и обращается к энергонезависимой памяти для чтения дополнительных параметров. Эта память в ПК представляет собой КМОП-память с резервным питанием от батарейки, поэтому часто называют ее «CMOS»; за пределами мира ПК ее обычно называют «NVRAM» (nonvolatile RAM).

Параметры, хранящиеся в NVRAM, различаются в разных системах, но как минимум, они должны указывать, какое устройство содержит загрузчик ОС. Такое устройство известно как «загрузочное устройство» (boot device). Этап аппаратной загрузки загружает загрузчик ОС из фиксированной позиции на загрузочном устройстве, а затем передает ему управление.

Сразу после включения питания системная прошивка выполнит минимальную инициализацию оборудования и передаст управление загрузчику ОС (например, systemd-boot или GRUB), хранящемуся на диске.

Этап 2. Загрузчик ОС

Основная задача загрузчика ОС — найти ядро на каком-либо устройстве, загрузить его в ОП и передать ему управление. Большинство загрузчиков ОС допускают интерактивное использование, чтобы обеспечить возможность указания альтернативного ядра (возможно, резервного на случай, если последнее скомпилированное ядро не работает) и для передачи необязательных параметров ядру.

В давние времена загрузчик ОС был расположен в начальном блоке размером 512 байт загрузочного устройства. Этот блок известен как «MBR» (Master Boot Record).

В большинстве систем загрузчик ОС очень ограничен из-за различных ограничений. Даже в системах, отличных от ПК, существуют некоторые ограничения на размер и сложность загрузчика, но ограничение размера MBR (512 байт, включая таблицу разделов) делает практически невозможным размещение в нем большого количества функциональности.

Поэтому в большинстве систем роль загрузки ОС разделена между основным загрузчиком ОС и вторичным загрузчиком ОС. Этот вторичный загрузчик ОС может располагаться на разделе диска. В GNU/Linux стандартным загрузчиком ОС является GRUB (GRand Unified Bootloader) второй версии.

Затем загрузчик находит ядро ОС на диске и загружает его в ОП. В системах, использующих EFI или другие типы микропрограммного обеспечения, это микропрограммное обеспечение может также загрузить ядро напрямую.

Этап 3. Ядро

После загрузки ядро выполняет следующие действия:

1. Инициализирует аппаратные устройства с помощью соответствующих драйверов.
2. Запускает “свопинг” - раздел на диске для расширения (ещё говорят - подкачки) виртуальной памяти.
3. Монтирует корневую файловую систему и создаёт корневую директорию с именем “/”.
4. Создает начальный процесс пользовательского пространства, которому присваивается число 1 в качестве PID (идентификатора процесса). Во многих современных дистрибутивах GNU/Linux этот процесс выполняет программу systemd.

Через загрузчик ОС ядру могут быть переданы параметры, влияющие на конфигурацию загружаемой системы (например, корневая файловая система по умолчанию может быть переопределена).

Сначала ядро монтирует специальную файловую систему в оперативной памяти, и уже затем ищет корневую файловую систему. В настоящее время это реализовано в виде “initramfs” — сжатый архив CPIO, который ядро извлекает в tmpfs. Раньше использовались обычные файловые системы, использующие блочное устройство в оперативной памяти (ramdisk), поэтому название “initrd” до сих пор используется для описания обеих концепций. Именно загрузчик ОС загружает как ядро, так и образы initrd/initramfs в память. В initramfs также для управления сервисами может использоваться systemd, аналогично реальной системе.

Этап 4. systemd

После обнаружения и монтирования корневой файловой системы initrd передает управление системному менеджеру (например, systemd), хранящемуся в корневой файловой системе, который затем выполняет проверку всего оставшегося оборудования, монтирование всех необходимых файловых систем и запуск всех настроенных сервисов.

При завершении работы системный менеджер останавливает все службы, отмонтирует все незанятые файловые системы, а затем (опционально) переходит в exitrd. exitrd использует tmpfs и отмонтирует/отключит оставшиеся файловые системы, включая корневую директорию. В качестве последнего шага система выключается.

Более подробно настройку и управление systemd мы рассмотрим в пятом модуле.

Этап 5. Сервисы

Сервис - это программа, предоставляющая услугу, также часто называемая «демоном» (daemon). Обычно демон может получать различные параметры командной строки и параметры. Более подробно загрузку сервисов и их настройку мы рассмотрим в пятом модуле.

‹ Предыдущий урок | Следующий урок ›

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX Разработка TUNE-IT

Во втором модуле мы подробно рассмотрели систему управления DAC. С точки зрения DAC каждый процесс выполняется от имени определённого пользователя и имеет доступ ко всем файлам, к которым у этого пользователя есть права. Но такая система прав не всегда удобна. Иногда нужно, чтобы процесс имел доступ к файлам, к которым пользователь, от имени которого он запущен, доступа не имеет. Например, процесс nginx должен иметь доступ к файлам сайтов, но пользователь nginx не должен иметь доступа к файлам пользователей.

SELinux предназначен для более гибкого управления доступом, чем DAC, и также для защиты ОС от атак с повышением привилегий.

Классы объектов

SELinux предоставляет набор общих универсальных мандатных управляемых доступом правил. Эти правила можно применять к любому объекту на системе, чтобы определить, какой доступ разрешён, а какой — нет. Для этого SELinux создаёт для каждого объекта специальную метку безопасности. Эта метка называется security context.

Для каждого субъекта и каждого объекта на системе SELinux создаёт security context, в котором содержится информация о том, какой доступ к этому объекту или субъекту разрешён. Такие правила доступа, определяющие, что разрешено, называются политикой безопасности.

Перед разработкой политики безопасности стоит ряд очевидных вопросов. Сколько разных уровней доступа нужно объектам? Сколько нужно субъектов? Какие права доступа должны быть у этих объектов?

Например, когда говорится о процессе, работающем от имени root, обычно подразумевается, что он может делать что угодно. Но в определённых ролях, таких как веб-сервер, процесс, работающий от имени root, может не нуждаться в таком широком доступе. Вместо этого ему может потребоваться доступ только к определённым файлам, каталогам и портам, необходимым для работы веб-сервера.

SELinux предоставляет для более гибкого управления доступом. Он позволяет определять не просто права доступа, а создавать политику безопасности, которая будет определять, какие процессы могут получать доступ к каким файлам, каталогам и портам. Для этого SELinux создаёт для каждого объекта специальную метку безопасности. Эта метка называется security context.

Security context — это набор меток, которые определяют, какой доступ разрешён, а какой — нет. Для этого SELinux создаёт для каждого объекта специальную метку безопасности. Эта метка называется security context.

Security context состоит из пяти меток:

```
$ sestat --help | grep -E "Security|Mode|Policy|Allow|Max"
Security Context: system_u:system_r:sestat_t:s0-s0:c0.c1023
Mode:             enforcing
Policy version:   33
Policy from config file:  targeted
Allow unconfined:         off
Allow runaway:            off
Allow ptrace:             off
Allow execstack:          off
Allow execmem:            off
Allow execmod:            off
Allow mmap_zero:          off
Allow mmap_min_addr:      off
Allow kernel_load:        off
Allow memprotect:         on
Allow console_login:      off
Allow mls_equal_policy:   on
Allow secure_mode:        off
Allow secure_mode_policyload: off
Allow deny_unknown:       off
Allow mls_range_policy:   on
Allow selinuxuser_execheap: off
Allow selinuxuser_execmod: off
Allow selinuxuser_execstack: off
Allow selinuxuser_ping:   off
Allow sysadm_exec_content: on
Allow user_direct_mouse:  off
Allow user_ping:          off
Allow user_rw_noexattrfile: off
Allow unlabeled_t_unconfined: off
Allow xguest_exec_content: off
Max kernel policy version:  33
```

Security context выглядит примерно так:

```
$ ls -Z /etc/shadow
system_u:object_r:shadow_t:s0 /etc/shadow
```

Если файл, содержащий базовую политику не задан, то команда sestat выведет информацию о текущей политике безопасности. Рассмотрим пример:

```
$ sestat
SELinux status:                 enabled
SELinuxfs mount:                /sys/fs/selinux
SELinux root directory:         /etc/selinux
Loaded policy name:             targeted
Current mode:                   enforcing
Mode from config file:          enforcing
Policy MLS status:              enabled
Policy deny_unknown status:     allowed
Memory protection checking:     actual (secure)
Max kernel policy version:      33
```

В состоянии текущей сессии ОС подразделяется от этого объекта на роли и об определённых ролях. Роль — это набор правил, которые определяют, какой доступ разрешён субъектам. Такие правила доступа определяют, какие процессы системы и к каким объектам они могут получать доступ. Например, процесс, работающий от имени root, может иметь широкий набор доступа. Такие объекты, как процессы, пользователи и программы, могут быть частью определённой роли.

```
$ sestat --role
SELinux role:
```

Роль — это набор правил, которые определяют, какой доступ разрешён субъектам. Такие правила доступа определяют, какие процессы системы и к каким объектам они могут получать доступ. Например, процесс, работающий от имени root, может иметь широкий набор доступа. Такие объекты, как процессы, пользователи и программы, могут быть частью определённой роли.

```
$ sestat --role -Z
system_u:system_r:sestat_t:s0-s0:c0.c1023
```

Роль — это набор правил, которые определяют, какой доступ разрешён субъектам. Такие правила доступа определяют, какие процессы системы и к каким объектам они могут получать доступ. Например, процесс, работающий от имени root, может иметь широкий набор доступа. Такие объекты, как процессы, пользователи и программы, могут быть частью определённой роли.

Рассмотрим общий набор ролей. Мы можем отличать операции, выполняемые от имени root, от операций, работающих от имени пользователя

6/8 Тестирование. Управление начальной загрузкой

текущий урок

Тестирование — до 13 апр. 2026 г. 23:59 MSK Завершено

Задание 1.1

0.5/0.5 points (graded)

Какое имя файла блочного устройства соответствует третьему разделу на втором жёстком диске с интерфейсом SATA?

- /dev/vda2
- /dev/sda3
- /dev/sdb2
- /dev/sdb3 ✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.2

0.5/0.5 points (graded)

Какая команда выведет точки монтирования файловой системы и объём свободного пространства в единицах СИ?

- df
- du -h
- df -h
- df -H ✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.3

0.5/0.5 points (graded)

В каком пространстве расширенных атрибутов хранится контекст безопасности файла?

- system
- user
- trusted
- security ✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.4

0.5/0.5 points (graded)

С помощью какой команды можно узнать к какому пакету относится файл /etc/selinux/config?

- rpm -ql
- rpm -qf ✔
- dnf info
- seinfo

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.5

0.5/0.5 points (graded)

На каком этапе начальной загрузки можно передать параметры ядру?

- 1 (UEFI)
- 2 (GRUB) ✔
- 3 (ядро)

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.6

0.5/0.5 points (graded)

Что произойдёт после обновления ядра при следующей загрузке системы?

- Будет загружено новое ядро. Старое ядро останется в системе ✔
- Будет загружено новое ядро. Старое ядро будет удалено
- Будет загружено старое ядро. Новое ядро будет установлено, но для его загрузки нужно будет дополнительно настроить загрузчик ОС

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.7

0.5/0.5 points (graded)

В каком файле настраивается автоматическое монтирование файловой системы во время начальной загрузки?

- devices
- fstab ✔
- filesystems

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.8

0.5/0.5 points (graded)

Какое значение директивы SELINUX заставляет SELinux НЕ форсировать решения о доступе, а только отмечать в журнале попытки доступа?

- enforcing
- permissive ✔
- disabled

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.9

0.5/0.5 points (graded)

Какому праву в DAC соответствует право getattr к файлу в SELinux?

- праву на поиск в директории, содержащей файл ✔
- праву на чтение файла
- праву на доступ к расширенным атрибутам файла

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.10

0.5/0.5 points (graded)

В чём опасность команды "dnf remove"?

- данная команда удалит указанный пакет, а также все пакеты, которые зависят от данного пакета ✔
- данная команда не освобождает место в файловой системе при удалении пакета
- данная команда удаляет систему управления пакетами dnf

Show answer

Отправить You have used 1 of 1 attempt

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026.
Дизайн MAX Разработка TUNE-IT

7/8 Заключение

текущий урок

В данном модуле мы рассмотрели этапы начальной загрузки и познакомились с базовыми задачами, которые необходимо выполнить, чтобы система могла успешно загрузиться. Мы более подробно взглянули на блочные устройства, научились создавать и подключать новые файловые системы, искать и устанавливать необходимое ПО, настраивать SELinux.

В следующем модуле мы более подробно рассмотрим этап начальной загрузки, за который отвечает менеджер сервисов systemd, а затем рассмотрим несколько базовых сервисов, без которых система не может нормально и безопасно функционировать.

Резюме

В этом модуле мы изучили следующие команды:

mkfs - создать файловую систему

e2label - изменить метку файловой системы

df - вывести информацию об объёме пространства файловой системы

du - посчитать размер файлов в директории

lsblk - вывести информацию о блочных устройствах

mount - смонтировать файловую систему

umount - отмонтировать файловую систему

lsof - вывести информацию об открытых файлах

rpm - управление программными пакетами

rpmkeys - проверка цифровой подписи пакета

dnf - управление репозиториями пакетов

seinfo - вывести информацию о политике SELinux

sesearch - поиск правил в политике SELinux

`semanage` - управление политикой SELinux

В этом модуле мы изучили следующие системные файлы:

/proc/devices - список групп устройств, зарегистрированных в ядре

/proc/filesystems - типы файловых систем, зарегистрированные в ядре

/etc/yum.repos.d/ - директория с конфигурационными файлами репозиториев

/etc/selinux/config - конфигурационный файл SELinux

Источники

- Ефанов, Д. В. Базовые механизмы защиты ядра Linux : учебное пособие / Д. В. Ефанов. — Москва : НИЯУ МИФИ, 2022. — 192 с. — ISBN 978-5-7262-2924-9. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/355520 (дата обращения: 22.01.2026). — Режим доступа: для авториз. пользователей.
- Mayer F., MacMillan K., Caplan D. SELinux by example: understanding security enhanced Linux. Upper Saddle River, NJ: Prentice Hall, 2007. - 425 с.
- Vermeulen Sven. SELinux System Administration. Packt Publishing, 2013. 103 с.

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX Разработка TUNE-IT

1/8 Введение

текущий урок

Модуль продолжает рассмотрение вопросов, связанных с начальной загрузкой системы. Менеджер системы и сервисов systemd является первым процессом в пространстве пользователя, и по значимости для функционирования системы он занимает второе место после ядра Linux. В пятом модуле рассматривается как настройка самого менеджера systemd, так и двух наиболее важных сервисов, за запуск которых он отвечает - сервиса журналирования journald, к которому обращаются другие системные службы и сетевые сервисы для информирования о своей работу, и сервиса входа пользователей в систему logind, благодаря которому функционирует многопользовательский режим работы. При входе в систему пользователь обязательно проходит процедуру аутентификации, которая реализована в виде системы PAM, настройка которой также рассмотрена в пятом модуле.

Как полученные знания пригодятся в профессиональном развитии

Как мы говорили в первом модуле - операционная система предоставляет приложениям набор услуг. Данные услуги настраиваются и управляются с помощью менеджера systemd. В повседневной деятельности пользователю необходимо запускать различные сервисы для своей работы или для тестирования своих проектов. Это могут быть web-сервер или сервер печати.

Кроме того, в случае каких-то неполадок, пользователю необходимо уметь находить в системном журнале нужные события, чтобы обнаружить источник проблем.

Мы также говорили в первом модуле, что операционная система является многопользовательской, а значит пользователю необходимо понимание того, как осуществляется вход пользователя в систему, и как например настроить виртуальные терминалы.

Каждый пользователь постоянно сталкивается с аутентификацией при доступе в систему или выполнении привилегированных команд. Поэтому настройка системы аутентификации является важным навыком любого пользователя GNU/Linux.

Цели и результат модуля

Студент, освоивший программу пятого модуля, должен:

1. Уметь запускать и мониторить системные службы и сетевые сервисы с помощью systemctl.
2. Уметь загружаться в различных режимах (таргетах).
3. Уметь загружаться в специальных режимах, предназначенных для восстановления разрушенной системы.
4. Уметь читать системный журнал с помощью journalctl.
5. Понимать как настроить нужное количество виртуальных терминалов для входа пользователя в систему.
6. Уметь подключать дополнительные факторы аутентификации для повышения гибкости и/или безопасности системы.

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX Разработка TUNE-IT

Конспект урока

Основные понятия

Linux было написано в основном на языке программирования Си, поэтому многие системные службы, которые работают в фоновом режиме, называются демонами. Они выполняют свои задачи в отсутствие пользователя и ожидают запросов служб и клиентов сервисов. Затем, в работающем системе, их называют процессами. Демоны могут быть запущены несколькими способами, в зависимости от того, как часто они используются.

Демоны — это процессы, которые работают в фоновом режиме и не связаны с каким-либо терминалом. Как правило, демоны запускаются автоматически при загрузке системы, но также могут быть запущены пользователем. Имена демонов обычно заканчиваются на букву d. Например, sshd — это демон, который обеспечивает вход в сеть через SSH.

В современных дистрибутивах Linux для управления загружаемыми процессами (и, которого PID равен 1) является systemd.

Ключевые возможности systemd:

- Параллельный запуск системных служб во время загрузки, что повышает скорость загрузки системы.
- Запуск демонов по запросу без необходимости запускать отдельный службы.
- Отслеживание процессов с использованием групп Linux (cgroups), а не будет пытаться отслеживать процессы по PID-файлам.
- Отслеживание зависимых процессов со службами, основанными на D-Bus и триггерах уровня.

Основной составляющих, которые управляет systemd, является юнит (unit). Представляющий собой ресурс, который может быть создан, уничтожен, включен или выключен в процессе работы системы. Действия юнитов управляются файлами юнитов.

Файлы конфигурации systemd — директория, в которой хранятся файлы конфигурации.

/usr/lib/systemd/system/ — файлы юнитов, распространяемых с установленными пакетами RPM.

/etc/systemd/system/ — файлы юнитов, созданных системным администратором.

/run/systemd/system/ — файлы юнитов, созданных во время выполнения. Этот каталог имеет приоритет выше, чем каталог, в котором установлены файлы юнитов пакетов RPM.

Unit — Общие настройки юнита.

Install — Настройки, которые используются при установке юнита.

Service — Настройки юнитов, которые управляются systemd.

Mount — Настройки точек монтирования файловых систем.

Path — Используется для задания путей, которые отслеживаются systemd.

Socket — Представляет сокеты межпроцессного взаимодействия (IPC), которые systemd должен отслеживать. Когда сокет получает запрос, systemd запускает службу, которая будет принимать запрос или слушать. Для каждого сокета должен быть создан соответствующий файл службы. Например, сокеты используются для X11, Syslog, Journald, Avahi, CUPS, D-Bus, Docker и SSH.

Timer — Представляет таймер, который управляется systemd. Например, вместо использования cron или at можно создать файл таймера, который будет управлять другим юнитом. Это можно использовать для периодических заданий, которые могут выполняться автоматически.

Swap — Представляет файлы подкачки или разделы, которые используются для виртуальной памяти.

Target — Используется для группировки юнитов systemd. Они предоставляют стандартную точку синхронизации, которая может использоваться программами для запуска в определенном состоянии. Многие юнитов target соответствуют уровням выполнения SysV, которые они заменяют.

Automount — Настраивает точки автоматического монтирования файловой системы. Каждый автоматически монтируемый юнит должен иметь соответствующий юнит монтирования, который будет определять, какая файловая система будет монтироваться.

Slice — Управляет ресурсами для группы процессов. Юниты slice не содержат процессов, но используются для группировки других юнитов в иерархическое дерево. Затем ресурсы могут быть выделены для юнитов в этом дереве. Это, по сути, то же самое, что и cgroups, но с интеграцией с systemd.

Scope — Управляет внешними системными процессами. Scope-юниты, в отличие от служебных, создаются программно с использованием API-интерфейсов systemd. Например, это может использоваться для управления пользовательскими сеансами, службами, которые были запущены вручную, и контейнерами.

Snapshot — Сохраняет состояние менеджера systemd. Полезно для временного возврата к предыдущему состоянию.

Device — Настраивает устройства, которые существуют в дереве устройств. Это может использоваться для создания зависимостей, основанных на устройствах, так что определенные службы запускаются только при наличии определенного оборудования.

Управление сервисами

Для управления сервисами (и, на самом деле, юнитами) systemctl использует службы с расширением .service. Если вы не предоставляете расширение при управлении службами, systemctl предполагает, что вы имеете в виду службу с расширением .service.

Команды systemctl без аргументов выведут список загруженных в активных юнитов.

Вывод списка сервисов

Во многих вариантах использования вам не нужно знать полный список доступных юнитов. Для вывода списка, который вам, вероятно, нужен, можно использовать несколько вариантов команды.

Вывод списка всех загруженных в данный момент сервисов можно сделать:

```
$ systemctl list-units --type=service
$ systemctl list-units --type service
$ systemctl list-units service
$ systemctl list-units
```

Для каждого юнита его описание содержит:

UNIT file — Полное имя юнита.

LOAD — Состояние загрузки файла конфигурации.

ACTIVE и SUB — Текущее состояние активации файла юнита верхнего и нижнего уровней.

DESCRIPTION — Краткое описание назначения юнитов.

По умолчанию команда systemctl list-units ограничивает вывод списка только активными юнитами. Чтобы увидеть все загруженные юнитов, даже если они неактивны, используйте опцию --all:

```
$ systemctl list-units --all
```

Вывод списка всех сервисов

Вы можете проверить любой сервисный юнит, который служба systemd загрузила в память, даже если он не активен. Это полезно, если вы хотите настроить службу, но она не запущена. Чтобы увидеть все установленные юнитов, независимо от того, загружены они или нет, используйте опцию list-unit-files:

```
$ systemctl list-unit-files
$ systemctl list-unit-files --type service
$ systemctl list-unit-files --type=service
```

Это отобразит список доступных сценариев инициализации, которые systemd может попытаться проанализировать и включить или отключить. Третий столбец показывает, включен или отключен юнит:

```
UNIT FILE                                  STATE
proc-sys-fs-binfmt_misc.automount          static
-.mount                                    generated
dbus.service                               enabled
dm-event.service                           disabled
dnf-makecache.service                      static
dnf-system-upgrade.service                 disabled
```

Отображение всех доступных юнитов:

```
$ systemctl list-unit-files --all
```

Вывод состояния сервисов

Вы можете проверить любой сервисный юнит, чтобы получить подробную информацию о его состоянии. Это включает в себя, загружен ли он, активен ли он, и недавние записи журнала. Чтобы проверить состояние службы, используйте команду status:

```
$ systemctl status httpd.service
```

Отображение подробной информации о сервисном юните, состоит следующее системный службы:

```
$ systemctl status httpd.service
```

Замените httpd на имя сервисного юнита, который вы хотите проверить (например, httpd). Пример:

```
$ systemctl status httpd
● httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
    Drop-In: /usr/lib/systemd/system/service.d
             └─10-timeout-abort.conf
     Active: active (running) since Sat 2024-05-25 21:41:46 MSK; 1min ago
       Docs: man:httpd.service(8)
   Main PID: 7888 (httpd)
     Status: "Started, listening on: port 80"
      Tasks: 213 (limit: 23004)
     Memory: 25.4M (peak: 27.9M swap: 8.0M swap peak: 8.0M)
        CPU: 1.000s
     CGroup: /system.slice/httpd.service
             ├─7888 /usr/sbin/httpd -DFOREGROUND
             ├─7889 /usr/sbin/httpd -DFOREGROUND
             ├─7890 /usr/sbin/httpd -DFOREGROUND
             ├─7891 /usr/sbin/httpd -DFOREGROUND
             └─7892 /usr/sbin/httpd -DFOREGROUND

May 25 21:41:46 localhost systemd[1]: Starting httpd.service - The Apache HTTP Server...
```

Это отобразит множество следующей информации:

- Имя юнита, описанного вместе с краткими описаниями.
- Loaded — Информация о том, загружен ли сервисный юнит абсолютный путь к файлу юнита и указывается ли юнит для запуска при загрузке системы (enabled или disabled).
- Active — Информация о том, активен ли сервисный юнит, с отметкой времени.
- Main PID — Идентификатор процесса и имя соответствующего системного службы.
- Status — Дополнительная информация о сервисном процессе.
- Cgroup — Дополнительная информация о связанных процессах.
- А также последние записи в журнале, связанные с работой данного сервиса.

Проверка работоспособности и конкретного сервисного юнита:

```
$ systemctl is-active name.service
```

Определение, управляем ли сервис конкретного сервисного юнита при загрузке системы:

```
$ systemctl is-enabled name.service
```

Команды systemctl по шагам и принятой терминологии для использования в скриптах, и потому возвращают код завершения, а вместо различных сообщений. Если вы желаете состояние службы, вы можете использовать опцию --quiet.

```
$ systemctl is-active apache2 --quiet && echo "Apache is running"
```

Чтобы вывести список юнитов, которые не работают, можно использовать команду systemctl — только с параметром.

Сервисы связаны функциональными зависимостями друг от друга. Чтобы использовать службу, которая зависит от другой службы, systemd должен также использовать службу. Чтобы

```
$ systemctl list-dependencies --after httpd.service
```

Например, чтобы проконтролировать список служб, которые запустились до httpd, введите:

```
$ systemctl list-dependencies --after httpd
httpd.service
● ├─dbus-broker.service
● ├─dm-event.service
● ├─system.slice
● ├─systemd-journald.service
● ├─systemd-tmpfiles-setup.service
● ├─basic.target
● │ ├─paths.target
● │ ├─slices.target
● │ ├─sockets.target
● │ ├─sysinit.target
```

Проверьте, какие службы зависят от конкретного сервиса:

```
$ systemctl list-dependencies --before name.service
```

Например, чтобы проконтролировать список служб, которые зависят от httpd, введите:

```
$ systemctl list-dependencies --before httpd
httpd.service
● ├─basic.target
● │ ├─sockets.target
● │ │ ├─dbus.socket
● │ │ ├─dm-event.socket
● │ │ ├─systemd-journald-dev-log.socket
● │ │ ├─systemd-journald.socket
● │ │ ├─systemd-udevd-control.socket
● │ │ └─systemd-udevd-kernel.socket
● │ ├─sysinit.target
● │ │ ├─dev-hugepages.mount
● │ │ ├─dev-mqueue.mount
● │ │ ├─kmod-static-nodes.service
● │ │ ├─proc-sys-fs-binfmt_misc.automount
● │ │ ├─sys-fs-fuse-connections.mount
● │ │ ├─sys-kernel-config.mount
● │ │ ├─sys-kernel-debug.mount
● │ │ ├─sys-kernel-tracing.mount
● │ │ ├─systemd-ask-password-console.path
● │ │ ├─systemd-binfmt.service
● │ │ ├─systemd-boot-random-seed.service
● │ │ ├─systemd-firstboot.service
● │ │ ├─systemd-hwdb-update.service
● │ │ ├─systemd-journal-catalog-update.service
● │ │ ├─systemd-journal-flush.service
● │ │ ├─systemd-machine-id-commit.service
● │ │ ├─systemd-modules-load.service
● │ │ ├─systemd-pcrmachine.service
● │ │ ├─systemd-pcrphase-sysinit.service
● │ │ ├─systemd-random-seed.service
● │ │ ├─systemd-repart.service
● │ │ ├─systemd-sysctl.service
● │ │ ├─systemd-sysusers.service
● │ │ ├─systemd-tmpfiles-setup-dev-early.service
● │ │ ├─systemd-tmpfiles-setup-dev.service
● │ │ ├─systemd-tmpfiles-setup.service
● │ │ ├─systemd-udev-trigger.service
● │ │ ├─systemd-udevd.service
● │ │ ├─systemd-update-done.service
● │ │ ├─cryptsetup.target
● │ │ ├─integritysetup.target
● │ │ ├─local-fs.target
● │ │ ├─swap.target
● │ │ ├─veritysetup.target
● │ │ └─systemd-pstore.service
```

Запуск и остановка сервисов

Для запуска сервиса используется подкоманда start. Пример запуска веб-сервера Apache:

```
$ systemctl start httpd.service
$ systemctl start httpd
● httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
    Drop-In: /usr/lib/systemd/system/service.d
             └─10-timeout-abort.conf
     Active: active (running) since Sat 2024-05-25 21:41:46 MSK; 1min ago
       Docs: man:httpd.service(8)
   Main PID: 7888 (httpd)
     Status: "Started, listening on: port 80"
      Tasks: 213 (limit: 23004)
     Memory: 25.4M (peak: 27.9M swap: 8.0M swap peak: 8.0M)
        CPU: 1.000s
     CGroup: /system.slice/httpd.service
             ├─7888 /usr/sbin/httpd -DFOREGROUND
             ├─7889 /usr/sbin/httpd -DFOREGROUND
             ├─7890 /usr/sbin/httpd -DFOREGROUND
             ├─7891 /usr/sbin/httpd -DFOREGROUND
             └─7892 /usr/sbin/httpd -DFOREGROUND

May 25 21:41:46 localhost systemd[1]: Starting httpd.service - The Apache HTTP Server...
```

В приведенном выше, что команда запускает веб-сервер Apache. Потом он был запущен. Чтобы остановить службу, используйте подкоманду stop. Для перезапуска службы, что, что запустит сама по процессам и создаст перезапуск.

Для остановки сервиса используется подкоманда stop аналогичным образом.

> Учитывайте существующие зависимости и отслеживайте зависимость между службами. Запуск конкретной службы может потребовать запуска другой или остановки другой службы. Например, вы не можете запустить две службы, которые используют один и тот же порт.

При попытке запуска новой службы systemd автоматически управляет всеми зависимостями. Все сервисы, от которых зависит ваша служба, также запускаются, если они еще не запущены. Если какая-либо из зависимостей не может запустить, ваша служба также не запустится.

Перезапуск и перезагрузка сервиса

Во многих перезапустить службу, чтобы изменения вступили в силу. Чтобы это было сделано, вам нужно:

- Остановить выбранный юнит и затем запустить его командами start и stop.
- Перезапустить юнит только в том случае, если соответствующая служба уже запущена.
- Перезагрузить конфигурацию системной службы без прерывания ее выполнения.

Перезапуск системных служб:

```
$ systemctl restart name.service
```

Замените httpd на имя службы, которую вы хотите перезапустить (например, httpd). Чтобы выполнялся службы не запущена, это не повлияет на неё.

Перезапуск службы только в том случае, если соответствующая служба уже работает:

```
$ systemctl try-restart name.service
```

Перезагрузка конфигурации без прерывания выполнения службы:

```
$ systemctl reload name.service
```

Загрузка в целевое состояние системы

Понятие целевых юнитов

Все системные единицы работают, вы можете управлять процессом загрузки можно управлять, какие службы запускаются при загрузке системы. Вы можете выбрать, чтобы работает графический интерфейс или только интерфейс командной строки. Вы можете даже создать специальные конфигурации, в которых запускаются только службы, которые вам нужны для решения конкретной задачи. Это может быть полезно для устранения неполадок.

> Цель (target) - это набор юнитов systemd, которые запускаются для достижения определенного состояния системы функционального состояния.

Цели в systemd - это группы сервисных юнитов, которые действуют как точки синхронизации во время загрузки системы. Вместо целевых юнитов, которые используются для запуска определенных служб для конкретного состояния, systemd предоставляет несколько целевых функциональных состояний.

Наиболее известные цели представлены в таблице.

basic.target	Начальное дно
rescue.target	Традиционная цель для однопользовательского обслуживания, похожая на режим однопользовательского пользователя 1
multi-user	Многопользовательский режим
graphical	Графический режим с графическим экраном входа пользователя
emergency	Аварийная оболочка

Изменение целевого юнита по умолчанию

Основной юнит-файл, который systemd использует для загрузки системы, называется default.target. Он используется для создания системы. При загрузке цели systemd разрешает default.target, определяя, какой юнит-файл он связан. Чтобы увидеть, какой целевой юнит используется по умолчанию, введите команду get-default:

```
$ systemctl get-default
graphical.target
```

Вывод будет похож на:

```
$ systemctl list-units --type target
UNIT                   LOAD   ACTIVE SUB    DESCRIPTION
basic.target           loaded active active Basic System
cryptsetup.target      loaded active active Local Encrypted Volumes
getty.target           loaded active active Login Prompts
graphical.target       loaded active active Graphical Interface
local-fs-pre.target    loaded active active Local File Systems (Pre)
local-fs.target        loaded active active Local File Systems
multi-user.target      loaded active active Multi-User System
network-online.target  loaded active active Network is Online
network.target         loaded active active Network
nfs-client.target      loaded active active NFS client services
paths.target           loaded active active Path Units
remote-fs.target       loaded active active Remote File Systems
slices.target          loaded active active Slice Units
sockets.target         loaded active active Socket Units
swap.target            loaded active active Swaps
sysinit.target         loaded active active System Initialization
time-set.target        loaded active active System Time Set
timers.target          loaded active active Timer Units

LOAD   = Reflects whether the unit definition was properly loaded.
ACTIVE = The high-level unit activation state, i.e. generalization of SUB.
SUB    = The low-level unit activation state, values depend on unit type.
20 loaded units listed.
To show all installed unit files use 'systemctl list-unit-files'.
```

Все юниты, установлены. Из перечня, однако не все из них являются "конечными состояниями". Используя multi-user.target — это промежуточное состояние, которое достигается, в том числе другими юнитами.

Посмотрите состояние, использованное другого перехода по умолчанию.

```
$ systemctl set-default name.target
```

Замените name.target на имя нужного вам целевого юнита (например, graphical.target).

Или в качестве альтернативы перезагрузите систему:

```
$ systemctl isolate name.target
```

Обслуживание и восстановление системы

Вы можете перевести операционную систему в аварийную и режим восстановления, используя systemctl. Это однопользовательская среда, для устранения неполадок при загрузке. В данном случае systemctl позволяет вам взаимодействовать с системой, прежде чем будут запущены многие службы. Это полезно, если служба мешает нормальной загрузке. Для этого выполните команду:

```
$ systemctl rescue
```

Это отправит вам сообщение, похожее на то, что вы видите при загрузке системы. Затем вам будет предложено ввести пароль root для входа в систему. Чтобы перейти в режим восстановления, вы должны иметь доступ к root. Если вы не можете получить доступ к root, вы можете использовать режим восстановления. Это позволяет вам загрузить систему в специальный режим, который не требует пароля root. Чтобы использовать режим восстановления, добавьте rd.break в конец строки linux в меню загрузчика GRUB. Затем нажмите Ctrl-x, чтобы загрузиться в этом режиме.

Перезагрузка системы, чтобы вернуться в нормальный режим:

```
$ systemctl reboot
```

Переключение в аварийный режим:

```
$ systemctl emergency
```

Аварийный режим похож на режим восстановления, но он еще более минимальный. Он не монтирует никаких файловых систем, кроме корневой. Он также не запускает никакие службы, кроме оболочки аварийного режима. Чтобы загрузиться в аварийном режиме, добавьте systemd.unit=emergency.target в конец строки linux в меню загрузчика GRUB. Затем нажмите Ctrl-x, чтобы загрузиться в этом режиме.

Перезагрузка системы, чтобы вернуться в нормальный режим:

```
$ systemctl reboot
```

Или в качестве альтернативы перезагрузите систему:

```
$ systemctl isolate multi-user.target
```

Перезагрузка системы, чтобы вернуться в нормальный режим:

```
$ systemctl reboot
```

Или в качестве альтернативы перезагрузите систему:

```
$ systemctl isolate multi-user.target
```


Политика конфиденциальности
Пользовательское соглашение
Возможности сотрудничества
Обратная связь и поддержка
Отзывы на SkillFactory
© SkillFactory, 2013 — 2024

Основные понятия

Сервис журналирования journald (полное имя systemd-journald) - это системная служба, которая собирает и хранит данные журналов. Она создает и поддерживает структурированные, индексированные журналы на основе информации журналов, получаемой из различных источников:

- Сообщения журналов ядра, через kmsg
- Простые сообщения системного журнала, через вызов библиотечных функции семейства syslog
- Структурированные сообщения системного журнала через собственный API
- Стандартный вывод и стандартный вывод сообщений об ошибках сервисных юнитов.
- Записи аудита, поступающие из подсистемы аудита ядра

Для каждого события сервис будет собирать многочисленные поля метаданных, которые сохраняются в журнале таким образом, что их нельзя подделать.

Собираемые журналом данные в основном текстовые, но могут также включать двоичные данные, если это необходимо. Отдельные поля, составляющие запись журнала, могут иметь размер до 2**-1 байт.

Сервис хранит данные журнала либо постоянно в каталоге /var/log/journal, либо в оперативной памяти в каталоге /run/log/journal/ (в последнем случае они теряются при перезагрузке). По умолчанию данные журнала хранятся постоянно, если каталог /var/log/journal/ существует во время загрузки, в противном случае происходит неявный переход к энергозависимому хранилищу.

Обязательно определите место хранения журнала с помощью директивы Storage= в конфигурационном файле journald.conf, чтобы данные сохранялись на диске независимо от существования каталога /var/log/journal/.

Обратите внимание, что journald изначально будет использовать ОП, пока вызов journalctl --flush (или отправка SIGUSR1 в journald) не приведёт к переключению на постоянное ведение журнала.

В системах, где каталог /var/log/journal/ еще не существует, но где требуется постоянное хранение журнала (и используется файл journald.conf по умолчанию), достаточно создать каталог и убедиться, что он имеет правильные режимы доступа и права владения.

Работа с журналом

Для поиска событий в журнале предназначена команда journalctl. Формат команды следующий:

journalctl [опции...] [условия...]

Если вызов осуществляется без параметров, отобразится содержимое журнала, доступного вызывающему пользователю, начиная с самой старой записи.

Если передан один или несколько условий (match), вывод фильтруется соответствующим образом. Соответствие имеет формат "FIELD=VALUE", например, "_SYSTEMD_UNIT=httpd.service", ссылаясь на поля структурированной записи журнала. Если указано несколько совпадений, соответствующих разным полям, записи журнала фильтруются по обоим, т.е. в результирующем выводе будут отображаться только записи, соответствующие всем указанным совпадениям этого типа. Если два совпадения относятся к одному и тому же полю, то они автоматически сопоставляются как альтернативы, т.е. в результирующем выводе будут отображаться записи, соответствующие любому из указанных совпадений для одного и того же поля. Наконец, символ "+" может отображаться как отдельное слово между другими терминами в командной строке. Это приводит к тому, что все совпадения до и после объединяются в дизъюнкцию (логическое ИЛИ).

Также можно фильтровать записи, указав абсолютный путь к файлу в качестве аргумента. Путь к файлу может быть файлом или символической ссылкой, и файл должен существовать на момент запроса. Если путь к файлу ссылается на исполняемый двоичный файл, к запросу добавляется совпадение "_EXE=" для канонизированного пути к двоичному файлу. Если путь к файлу ссылается на исполняемый скрипт, к запросу добавляется совпадение "_COMM=" для имени скрипта. Если путь к файлу ссылается на файл устройства, к запросу добавляются совпадения "_KERNEL_DEVICE=" для имени ядра устройства и для каждого из его родительских устройств. Символические ссылки разыменовываются, имена ядер синтезируются, и родительские устройства идентифицируются из среды на момент запроса. В общем случае, файл устройства является наилучшим аналогом реального устройства, поскольку записи журнала обычно не содержат полей, идентифицирующих реальное устройство. Для того чтобы результирующие записи журнала были корректными для реального устройства, соответствующие части среды на момент записи, в частности, реальное устройство, соответствующее файлу устройства, должны были быть такими же, как и на момент запроса. Поскольку файлы устройств могут меняться при перезагрузках, указание пути к файлу устройства приводит к тому, что результирующие записи ограничиваются записями текущей загрузки.

Дополнительные ограничения могут быть добавлены с помощью параметров --boot, --unit=, и т. д., чтобы еще больше ограничить отображаемые записи (логическое И).

Вывод осуществляется попеременно из всех доступных файлов журнала, независимо от того, находятся ли они в процессе ротации или записи, и независимо от того, принадлежат ли они самой системе или являются доступными пользовательскими журналами. Параметр --header можно использовать для идентификации отображаемых файлов.

Набор используемых файлов журнала можно изменить с помощью параметров --user, --system, --directory= и --file=.

Всем пользователям предоставляется доступ к их личным журналам. Однако по умолчанию доступ к системному журналу и журналам других пользователей предоставляется только пользователям root и пользователям, входящим в несколько специальных групп. Члены групп "systemd-journal", "adm" и "wheel" могут читать все файлы журнала. Обратите внимание, что две последние группы традиционно имеют дополнительные привилегии, зависящие от конкретного дистрибутива. Например в rpm-дистрибутивах, члены группы "wheel" могут выполнять административные задачи.

По умолчанию вывод осуществляется постранично через less, а длинные строки "обрезаются" до ширины экрана. Скрытую часть можно просмотреть, используя клавиши со стрелками влево и вправо. Постраничная навигация может быть отключена.

При выводе на терминал строки окрашиваются в соответствии с приоритетом:

- строки уровня ERROR и выше окрашиваются в красный цвет;
- строки уровня WARNING окрашиваются в желтый цвет;
- строки уровня NOTICE подсвечиваются;
- строки уровня INFO отображаются обычным образом;
- строки уровня DEBUG окрашиваются в серый цвет.

Для записи записей в журнал можно использовать несколько методов. Обычно, вывод из модулей systemd автоматически подключается к журналу. Кроме того, команду systemd-cat можно использовать для отправки сообщений в журнал напрямую.

В таблице представлены наиболее часто используемые опции команды journalctl.

Опция	Назначение
-S, --since=, -U, --until=	Начать отображение записей с указанной даты, или до указанной даты, соответственно. Указания даты должны иметь формат "2026-03-04 02:15:00". Если часть времени опущена, предполагается "00:00:00". Если опущена только часть секунд, предполагается ":00". Если часть даты опущена, предполагается текущий день. В качестве альтернативы, понимаются строки "yesterday", "today", "tomorrow", которые относятся к 00:00:00 дня, предшествующего текущему дню, текущего дня или дня после текущего дня, соответственно. "now" относится к текущему времени. Также можно указать относительное время, с префиксом "-" или "+", которое относится ко времени до или после текущего времени, соответственно.
-b [ID][±offset][:id], --boot[=ID][±offset][:id]	Показать сообщения с конкретной загрузки. Это добавит совпадение для "_BOOT_ID=". Аргумент может быть пустым, в этом случае будут показаны журналы для текущей загрузки. Если идентификатор загрузки опущен, положительное смещение будет искать загрузки, начиная с начала журнала, а смещение, равное или меньшее нуля, будет искать загрузки, начиная с конца журнала. Таким образом, 1 означает первую загрузку, найденную в журнале в хронологическом порядке, 2 — вторую и так далее; в то время как -0 — последнюю загрузку, -1 — предпоследнюю и так далее. Пустое смещение эквивалентно -0.
-u, --unit=UNIT|PATTERN	Отображает сообщения для указанного юнита UNIT или для любого из юнитов, соответствующих PATTERN. Для каждого имени юнита добавляется совпадение "_SYSTEMD_UNIT=UNIT". Также добавляются совпадения для сообщений от systemd и сообщений о дампах памяти для указанного юнита. Добавляется совпадение "_SYSTEMD_SLICE=UNIT" для unit systemd.slice, чтобы показать журналы дочерних элементов среза. С параметром --user все аргументы --unit= будут преобразованы для соответствия сообщениям пользователя, как если бы они были указаны с параметром --user-unit=. Этот параметр может быть указан несколько раз.
-p, --priority=	Фильтрация вывода по приоритетам сообщений или диапазонам приоритетов: один уровень (0/"emerg" … 7/"debug") или диапазон OT..ДО. Если указан один уровень, отображаются все сообщения с этим уровнем или более важным (численно меньшим). Если указан диапазон, отображаются все сообщения в пределах диапазона, включая границы. Это добавит совпадения "PRIORITY=" для указанных приоритетов.
-f, --follow	Отображать только самые последние записи в журнале и непрерывно выводить новые записи по мере их добавления, пока не будет нажата клавиша Ctrl-C.

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026.
Дизайн MAX Разработка TUNE-IT

4/8 Урок 3. Сервис входа в систему logind

текущий урок

Основные понятия

Сервис входа в систему logind - это сервис, который управляет входом пользователей в систему и, таким образом, обеспечивает многопользовательский режим работы системы. Полное имя сервиса - systemd-logind.

Сервис logind отвечает за следующие задачи:

- Отслеживание пользователей и сессий, их процессов и их состояния простоя. Это реализуется путем выделения slice-юнита systemd для каждого пользователя внутри user.slice и scope-юнита внутри него для каждой одновременной сессии пользователя. Кроме того, для каждого вошедшего в систему пользователя запускается экземпляр системной службы user@.service.
- Генерацию и управление идентификаторами сессий. Если доступен аудит, и для сессии уже установлен идентификатор сессии аудита, то этот идентификатор повторно используется в качестве идентификатора сессии. В противном случае используется независимый счетчик сессий.
- Предоставление пользователям доступа на основе Polkit для таких операций, как выключение системы или переход в спящий режим.
- Реализация логики блокировки выключения/перехода в спящий режим для приложений.
- Обработка аппаратных клавиш включения питания/сна.
- Управление несколькими рабочими местами.
- Управление переключением сессий.
- Управление доступом к устройствам для пользователей.
- Автоматическое создание текстовых логинов (agetty) на виртуальной консоли.
- Активация и управление каталогами пользователя времени выполнения.
- Планирование выгрузки системы.
- Отправка сообщений "wall".

В своей работе logind оперирует следующими понятиями:

Рабочее место (seat) состоит из всех аппаратных устройств, назначенных определенному рабочему месту. Обычно оно включает в себя как минимум одно графическое устройство, клавиатуру и мышь. Оно также может включать видеокамеры, звуковые карты и многое другое. Рабочие места идентифицируются именами рабочих мест, которые представляют собой строки (<= 255 символов), начинающиеся с четырех символов "seat", за которыми следует как минимум один символ из диапазона [a-zA-Z0-9], "_" и "-". Они подходят для использования в качестве имён файлов. Имена рабочих мест могут быть или не быть стабильными и могут быть повторно использованы, если рабочее место снова станет доступным.

Сессия (session) определяется временем от момента входа пользователя в систему до момента выхода из неё. Сессия привязана к одному рабочему месту или не привязана совсем (например, для "виртуальных" SSH-подключений). К одному рабочему месту может быть привязано несколько сессий, но активной может быть только одна из них, остальные работают в фоновом режиме. Сессия идентифицируется короткой строкой.

systemd гарантирует, что сеансы аудита идентичны сеансам systemd, и использует идентификатор сеанса аудита в качестве идентификатора сеанса в systemd (если аудит включен). В общем случае идентификатор сеанса представляет собой короткую строку, состоящую только из [a-zA-Z0-9], "_" и "-", подходящую для использования в качестве имени файла. Идентификаторы сеансов уникальны на локальном компьютере и никогда не используются повторно, пока компьютер находится в сети. Пользователь соответствует человеку, использующему компьютер. Один пользователь может иметь несколько открытых сеансов одновременно. Пользователь идентифицируется числовым идентификатором пользователя (UID) или именем пользователя (строка). Многосессионная система позволяет использовать несколько пользовательских сеансов на одном и том же рабочем месте одновременно. Многоместная система позволяет использовать несколько независимых рабочих мест, которые могут использоваться индивидуально и одновременно разными пользователями.

Все аппаратные устройства, которые могут быть назначены на рабочее место, назначаются на одно из них. Устройство может быть назначено только одному месту одновременно. Если устройство не назначено ни одному конкретному месту, оно автоматически назначается специальному месту по умолчанию, называемому "seat0".

Обратите внимание, что оборудование, такое как принтеры, жесткие диски или сетевые карты, как правило, не назначается конкретному месту. Они доступны всем местам в равной степени. При этом, USB-накопители могут быть назначены месту. "seat0" существует всегда.

Управление сервисом

Для управления сервисом входа предназначена команда loginctl. Формат команды следующий:

```
loginctl [опции] {подкоманды} [имена]
```

У команды loginctl большой список подкоманд, которые подробно описаны в документации. Все подкоманды можно разделить на сессионные, пользовательские и рабочего места. В таблице представлены основные подкоманды.

Название подкоманды	Назначение подкоманды
list-sessions	Вывести текущие сессии
session-status [ID...]	Отображает краткую информацию о состоянии одной или нескольких сессий во время выполнения, за которой следуют самые последние записи журнала. Принимает один или несколько идентификаторов сессий в качестве параметров. Если идентификаторы сессий не переданы, отображается состояние сессии вызывающего процесса (пользователя). Эта функция предназначена для генерации удобочитаемого вывода. Если нужен вывод, пригодный для анализа, используйте вместо этого show-session.
show-session [ID...]	Отображение свойств одной или нескольких сессий или самого менеджера. Если аргумент не указан, будут показаны свойства менеджера. Если указан идентификатор сессии, будут показаны свойства сессии. В частности, если указанный идентификатор - "self", используется сессия, к которой принадлежит процесс loginctl. Если "auto", используется текущая сессия, как и в случае с "self". По умолчанию пустые свойства не выводятся, их можно вывести с помощью --all. Чтобы выбрать определенные свойства для отображения, используйте --property=.
list-users	Список текущих авторизованных пользователей
user-status [USER...]	Отображает краткую информацию о состоянии во время выполнения для одного или нескольких авторизованных пользователей, за которой следуют самые последние записи журнала. Принимает в качестве параметров одно или несколько имен пользователей или числовых идентификаторов пользователей. Если параметры не переданы, отображается состояние для пользователя сессии вызывающего процесса. Эта функция предназначена для генерации удобочитаемого вывода. Если нужен вывод, пригодный для машинного анализа, используйте вместо этого show-user.
show-user [USER...]	Отображает свойства одного или нескольких пользователей или самого менеджера. Если аргумент не указан, будут показаны свойства менеджера. Если указан пользователь, отображаются свойства пользователя. По умолчанию пустые свойства не выводятся. Используйте --all, чтобы показать и их. Чтобы выбрать определенные свойства для отображения, используйте --property=.
list-seats	Список доступных в данный момент мест

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026.
Дизайн MAX Разработка TUNE-IT

Основные понятия

PAM - это система библиотек, которые решают задачи аутентификации приложений (сервисов) в системе. Библиотека предоставляет программный интерфейс (Application Programming Interface - API), который позволяет программам, предоставляющим привилегии (такими как login и su), делегировать выполнение стандартных задач аутентификации.

> Модуль PAM - это разделяемая библиотека, предназначенная для выполнения определённой задачи аутентификации.

Обычно библиотеки располагаются в директории /usr/lib64/security/. Пример:

```
$ find /usr/lib64/security/ -type f | wc
     61      61    2137
$ file /usr/lib64/security/pam_unix.so
/usr/lib64/security/pam_unix.so: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV),
dynamically linked, BuildID[sha1]=54bd85c2807e774cddbc1bc63c1539b42bbealaf, stripped
```

Видно, что в системе установлен как минимум 61 модуль PAM, а также видно, что сам модуль представляет собой разделяемую библиотеку в формате ELF.

Главная особенность подхода PAM заключается в том, что характер аутентификации динамически настраивается. Другими словами, системный администратор может свободно выбирать, как отдельные приложения, предоставляющие сервисы, будут аутентифицировать пользователей. Эта динамическая конфигурация задается содержимым конфигурационного файла /etc/pam.conf. В качестве альтернативы и предпочтительнее, конфигурация может быть задана отдельными конфигурационными файлами, расположенными в каталоге pam.d. Наличие этого каталога приведет к тому, что PAM будет игнорировать /etc/pam.conf.

Файлы конфигурации определяют связь между приложениями (службами) и подключаемыми модулями аутентификации (PAM), которые выполняют фактические задачи аутентификации.

Для работы с PAM, приложение должно использовать функции библиотеки libpam, т.е. быть скомпилированным с этой библиотекой. Проверить, работает ли приложение с библиотекой libpam можно с помощью команды ldd. Например:

$ ldd /usr/bin/su

linux-vdso.so.1 (0x00007fe6a1a8f000)

libpam.so.0 => /lib64/libpam.so.0 (0x00007fe6a1a53000)

libpam_misc.so.0 => /lib64/libpam_misc.so.0 (0x00007fe6a1a4c000)

libc.so.6 => /lib64/libc.so.6 (0x00007fe6a1859000)

Видно, что утилита su использует библиотеки libpam.so и libpam_misc.so.

Задачи аутентификации делятся на четыре независимые группы управления (management groups):

- управление учетными записями (account)
- управление аутентификацией (auth)
- управление паролями (password)
- управление сессиями (session)

Данные группы отвечают за различные аспекты типичного запроса пользователя на доступ к ограниченному сервису. Назначение групп представлено в таблице.

Название группы	Назначение группы
account	Группа выполняет проверку учётной записи. Обычно используется для ограничения/разрешения доступа к сервису на основе времени суток, текущих доступных системных ресурсов (максимальное количество пользователей), способа входа пользователя в систему, или истек ли срок действия пароля пользователя?
auth	Группа обеспечивает два аспекта аутентификации пользователя. Во-первых, она устанавливает, что пользователь является тем, за кого себя выдает, путем указания приложению запросить у пользователя пароль или другие средства идентификации. Во-вторых, модуль может предоставлять членство в группе или другие привилегии. Не все виды аутентификации относятся к этому типу: существуют аппаратные схемы аутентификации (например, использование смарт-карт и биометрических устройств), с соответствующими модулями, которые могут быть легко заменены более стандартными подходами к аутентификации - в этом проявляется гибкость PAM.
password	Группа отвечает за обновление механизмов (токенов) аутентификации, связанного с пользователем. Как правило, такие сервисы тесно связаны с сервисами группы аутентификации, т.е. существует один модуль для каждого типа аутентификации (auth) на основе «запрос/ответ».
session	Группа задач охватывает действия, которые должны быть выполнены до предоставления услуги и после её отказа. Такие задачи включают ведение журналов аудита и монтирование домашнего каталога пользователя. Группа управления сессиями также регистрирует информацию об открытии/закрытии обмена данными с пользователем.

Конфигурационный файл /etc/pam.conf

При запуске приложения, предоставляющего привилегии с поддержкой PAM, оно активирует подключение к API. В ходе активации выполняется чтение конфигурационного файла /etc/pam.conf или файлов, расположенных в каталоге pam.d.

В этих файлах перечислены модули, которые будут выполнять задачи аутентификации, необходимые для данной службы, и соответствующее поведение PAM в случае неудачи отдельных модулей.

Синтаксис конфигурационного файла /etc/pam.conf следующий.

Файл состоит из списка правил, каждое правило обычно размещается на одной строке, но может быть расширено экранированным символом конца строки: '\'. Комментарии предваряются символами # и продолжаются до следующего конца строки.

Формат каждого правила представляет собой набор токенов, разделённых пробелами, причём первые три нечувствительны к регистру:

```
имя_сервиса тип флаг путь_к_модулю аргументы_модуля
```

Синтаксис файлов, содержащихся в каталоге /etc/pam.d/, идентичен, за исключением отсутствия поля имя_сервиса. В этом случае именем сервиса является имя самого файла в каталоге /etc/pam.d/. Это имя файла должно быть написано строчными буквами.

Важной особенностью PAM является то, что модули можно стекировать, создавая сложную конфигурацию для данного сервиса. Имя сервиса обычно представляет собой имя соответствующего приложения, например login и su. Специальное имя сервиса other зарезервировано для задания правил по умолчанию.

тип - это группа управления, которой соответствует правило. Он используется для указания того, с какой из групп управления должен быть связан соответствующий модуль.

флаг указывает на поведение PAM, если модуль не сможет успешно выполнить задачу аутентификации. Для флагов существует два типа синтаксиса: простой, который содержит одно простое ключевое слово и более сложный, который включает в себя выбор пар значение=действие в квадратных скобках. Простой синтаксис представлен в таблице.

Название флага	Назначение флага
required	неудача модуля в конечном итоге приведет к тому, что PAM вернёт ошибку, но только после того, как будут вызваны оставшиеся модули стека
requisite	аналогично required, однако, в случае, если такой модуль возвращает ошибку, управление сразу возвращается приложению или вышестоящему стеку PAM. Возвращаемое значение - это значение, связанное с первым required или requisite модулем, который вернул ошибку. Обратите внимание, этот флаг можно использовать для защиты от возможности ввода паролей пользователем через небезопасный канал связи. И наоборот, использование этого флага может сообщить злоумышленнику о прохождении того, или иного модуля.
sufficient	если такой модуль успешно выполняется и ни один из ранее необходимых модулей не дал сбоя, платформа PAM немедленно возвращает успешный результат приложению или вышестоящему стеку PAM, не вызывая никаких дополнительных модулей в стеке. Сбой sufficient модуля игнорируется, и обработка стека модулей PAM продолжается без изменений.
optional	успех или сбой этого модуля важен только в том случае, если он является единственным модулем в стеке, связанным с этим типом службы
include	включить все строки заданного типа из файла конфигурации, указанного в качестве аргумента этого элемента управления
substack	включить все строки заданного типа из файла конфигурации, указанного в качестве аргумента этого элемента управления. Отличие от include в том, что весь substack рассматривается как отдельный модуль.

путь_к_модулю - это либо полное имя модуля PAM, используемого приложением (начинается с '/'), либо относительный путь от расположения модулей по умолчанию: /lib/security/ или /lib64/security/, в зависимости от архитектуры.

аргументы_модуля - это список токенов, разделённых пробелами, которые можно использовать для изменения конкретного поведения данного модуля. Такие аргументы_модуля будут зависят от конкретного модуля. Обратите внимание, что если вы хотите включить пробелы в аргумент, вам следует заключить этот аргумент в квадратные скобки.

Более гибким вариантом, чем использование одного конфигурационного файла, является настройка libpam через содержимое каталогов pam.d. В этом случае каталоги заполняются файлами, каждый из которых имеет имя файла, равное имени службы (в нижнем регистре), т.е. это личный конфигурационный файл для указанной службы.

Синтаксис каждого файла в каталоге pam.d аналогичен синтаксису файла /etc/pam.conf и состоит из строк следующего вида:

```
тип флаг путь_к_модулю аргументы_модуля
```

Единственное отличие заключается в отсутствии имени службы (service-name). Имя службы - это имя соответствующего конфигурационного файла. Например, /etc/pam.d/login содержит конфигурацию для службы входа в систему login.

‹ Предыдущий урок | Следующий урок ›

6/8 Тестирование. Управление начальной загрузкой

текущий урок

Тестирование — до 13 апр. 2026 г., 23:59 MSK Завершено

Задание 1.1

0.5/0.5 points (graded)

К какому фактору аутентификации относится ввод пароля пользователем?

- (•) user knows
- ( ) user has
- ( ) user is
- ( ) user does

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.2

0.5/0.5 points (graded)

Какая управляющая группа PAM отвечает за монтирование (подключение) домашних директорий пользователей по сети?

- ( ) account
- ( ) auth
- ( ) password
- (•) session

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.3

0.5/0.5 points (graded)

Какой флаг PAM в случае НЕуспешного прохождения модуля сразу возвращает управление программе?

- ( ) required
- ( ) sufficient
- ( ) optional
- (•) requisite

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.4

0.0/0.5 points (graded)

Чем характеризуются следующие режимы работы системы: rescue и emergency?

- [x] В rescue-режиме сетевые средства не подняты
- [x] В emergency-режиме сетевые средства не подняты
- [x] В rescue-режиме смонтированы локальные файловые системы
- [x] В emergency-режиме смонтированы локальные файловые системы
- [ ] В rescue-режиме обычные пользователи могут входить в систему
- [ ] В emergency-режиме обычные пользователи могут входить в систему

✖

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.5

0.5/0.5 points (graded)

На каком этапе начальной загрузки можно указать режим emergency?

- ( ) 1 (UEFI)
- (•) 2 (GRUB)
- ( ) 3 (ядро)
- ( ) 4 (systemd)

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.6

0.5/0.5 points (graded)

В какую директорию помещаются файлы юнитов при установке нового ПО?

- (•) /usr/lib/systemd/system/
- ( ) /run/systemd/system/
- ( ) /etc/systemd/system/

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.7

0.5/0.5 points (graded)

Какая команда покажет список юнитов, от загрузки которых зависит успешность запуска web-сервера?

- ( ) systemctl list-dependencies --before httpd
- (•) systemctl list-dependencies --after httpd
- ( ) systemctl status httpd

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.8

0.5/0.5 points (graded)

Какой подкомандой systemctl вывести target-юнит, в который система перейдёт автоматически во время начальной загрузки?

- ( ) isolate
- (•) get-default
- ( ) list-units

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.9

0.5/0.5 points (graded)

Выберите верное утверждение:

- (•) сессия (session) может быть привязана только к одному рабочему месту (seat)
- ( ) рабочее место (seat) может быть привязано только к одной сессии (session)
- ( ) рабочее место (seat) и сессия (session) не связаны друг с другом

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.10

0.5/0.5 points (graded)

Какую программу запускает менеджер logind для входа пользователя в систему через виртуальный терминал?

- (•) agetty
- ( ) login
- ( ) bash

✔

Show answer

Отправить You have used 1 of 1 attempt

‹ Предыдущий урок Следующий урок ›

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026.

Дизайн MAX Разработка TUNE-IT

7/8 Заключение

текущий урок

В данном модуле мы рассмотрели назначение и настройку менеджера системы и сервисов systemd, который также является первым процессом в пространстве пользователя. Также были рассмотрены сервисы журналирования и входа пользователей в систему, которые управляются менеджером systemd. Затем мы познакомились с системой аутентификации PAM, которая используется при входе пользователя в систему.

Таким образом, в четвёртом и пятом модулях были рассмотрены все этапы начальной загрузки от момента включения питания компьютера до входа пользователя в систему и запуска его командной оболочки.

В следующем модуле мы рассмотрим задачи, которые выполняет системный администратор для включения системы в локальную сеть и запуск контейнеров.

Резюме

В этом модуле мы изучили следующие команды:

systemctl - управление демоном systemd.

systemd-journald.service - демон журналирования.

journalctl - чтение системного журнала.

systemd-cat - отправка сообщений в журнал напрямую.

systemd-logind.service - демон входа в систему.

loginctl - управление демоном logind.

В этом модуле мы изучили следующие системные файлы:

/etc/pam.conf - файл конфигурации PAM

/etc/pam.d - директория конфигурации PAM

Источники

- James M. Kearns. Linux systemd: Mastering Service Management: A practical guide to unit files, timers, journald, and reliable operations across Debian, Ubuntu, RHEL, and Fedora. Packt Publishing, 2025.
- Donald A. Tevault. Linux Service Management Made Easy with systemd: Advanced techniques to effectively manage, control, and monitor Linux systems and services. Packt Publishing, 2022.
- Kenneth Geisshirt. Pluggable Authentication Modules: The Definitive Guide to PAM for Linux SysAdmins and C Developers: A comprehensive and practical guide to PAM for Linux: how modules work and how to implement them. Packt Publishing, 2007.

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX Разработка TUNE-IT

1/7 Введение

текущий урок

Модуль является завершающим в данном курсе и объединяющим все предыдущие модули. В модуле рассматриваются вопросы подключения рабочего места к сети, а также основы работы с платформой контейнеризации Docker.

Как полученные знания пригодятся в профессиональном развитии

Подключение компьютера к локальной сети или Интернет является необходимым условием для решения профессиональных задач, а также для поддержания рабочего места в актуальном и безопасном состоянии.

Контейнеры Docker широко используются как для организации собственной среды разработки, так и для публикации разработанного ПО со всем окружением (библиотеками, зависимостями, настройками) в виде стандартного образа (image), что обеспечивает стабильную работу ПО на любой системе (как на рабочей станции разработчика, так и на сервере), ускоряет развертывание, упрощает масштабирование и эффективнее использует ресурсы сервера по сравнению с технологией виртуальных машин.

Цели и результат модуля

Студент, освоивший программу шестого модуля, должен:

1. Уметь настраивать сетевые интерфейсы.
2. Уметь настраивать подключение к внешнему DNS-серверу.
3. Уметь выполнять мониторинг сетевого трафика.
4. Уметь устанавливать платформу контейнеризации Docker.
5. Уметь находить и скачивать образы, содержащие необходимое для работы ПО.
6. Уметь запускать контейнеры, взаимодействовать с ними, а также осуществлять их мониторинг.

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX Разработка TUNE-IT

Основные понятия TCP/IP

Сетевая модель TCP/IP представляет собой упрощенный четырехуровневый набор абстракций, описывающих взаимодействие различных протоколов для передачи пакетов в сетях и через интернет. Каждый уровень использует данные и протоколы с нижележащего и передает выше — данные сервису.

Четыре уровня:

- Приложение. Каждое приложение имеет специфичную для сервиса, позволяющую клиенту и серверу обмениваться данными между собой протокол (например, HTTP, используется клиентом и сервером веб-браузера, или SSH (защищенный), HTTPS (защищенная версия протокола HTTP), FTP (протокол для передачи файлов) и SMTP (протокол почты).
- Транспортный. Транспортные протоколы – это TCP и UDP. TCP – это надежный протокол, который гарантирует доставку пакетов в нужной последовательности. UDP – ненадежный протокол, который не гарантирует доставку пакетов и их последовательность. Он используется для передачи потоков данных, где допустимо некоторое количество потерь, например при передаче голоса или видеосигнала. Каждый пакет имеет исходный порт и целевой порт. Эти порты используются для идентификации приложения, которое отправляет или принимает данные.
- Сетевой. Сетевой уровень отвечает за маршрутизацию пакетов. Для IP-адресации используется IPv4 (версия 4) или IPv6 (версия 6). Для определения пути к целевому IP-адресу используются протоколы маршрутизации, такие как OSPF, BGP и RIP.
- Канальный уровень. Канальный уровень отвечает за доступ к среде передачи данных. Примеры технологий канального уровня: Ethernet (802.3), Wi-Fi (802.11), Bluetooth, PPP (Point-to-Point Protocol) и другие.

Последовательно инкапсулируем сообщение, что, когда оно достигнет интерфейса в системе, мы можем его отправить по сети или извлечь оттуда.

Имена сетевых интерфейсов

Каждое сетевое adapter в системе имеет имя, которое используется для его настройки и идентификации. Имена сетевых интерфейсов присваиваются ядром Linux, и существуют различные правила для их именования. Вот некоторые из них:

- Интерфейсы Ethernet начинаются с eth:
- Интерфейсы WLAN начинаются с wl
- Интерфейсы WWAN начинаются с ww

Остальная часть имени интерфейса после типа будет основана на информации, предоставленной системой. Например, системой может быть определена информация о топологии PCI.

eth указывает, что это встроенное устройство, а number представляет порядковое номер. В имя этого устройства. Также обратим, wlx0 — это встроенное устройство Ethernet с MAC-адресом 0x0123456789ab, называемым wlx0123456789ab.

eth указывает, что это устройство на основе сигнала в PCI для гостевой связи. Таким образом, wl0 — это карта Ethernet в слоте 0 PCI для гостевой связи.

Имена сетевых интерфейсов начинаются с типа интерфейса:

- Интерфейсы Ethernet начинаются с en
- Интерфейсы WLAN начинаются с wl
- Интерфейсы WWAN начинаются с ww

Остальная часть имени интерфейса после типа будет основана на информации, предоставленной системой. Например, системой может быть определена информация о топологии PCI.

eth указывает, что это встроенное устройство, а number представляет порядковое номер. В имя этого устройства. Также обратим, wlx0 — это встроенное устройство Ethernet с MAC-адресом 0x0123456789ab, называемым wlx0123456789ab.

eth указывает, что это устройство на основе сигнала в PCI для гостевой связи. Таким образом, wl0 — это карта Ethernet в слоте 0 PCI для гостевой связи.

Сетевые интерфейсы имеют имена, которые используются для настройки и идентификации. Имена сетевых интерфейсов присваиваются ядром Linux, и существуют различные правила для их именования. Вот некоторые из них:

- Интерфейсы Ethernet начинаются с eth:
- Интерфейсы WLAN начинаются с wl
- Интерфейсы WWAN начинаются с ww

Остальная часть имени интерфейса после типа будет основана на информации, предоставленной системой. Например, системой может быть определена информация о топологии PCI.

eth указывает, что это встроенное устройство, а number представляет порядковое номер. В имя этого устройства. Также обратим, wlx0 — это встроенное устройство Ethernet с MAC-адресом 0x0123456789ab, называемым wlx0123456789ab.

eth указывает, что это устройство на основе сигнала в PCI для гостевой связи. Таким образом, wl0 — это карта Ethernet в слоте 0 PCI для гостевой связи.

Протокол IPv4

IPv4 – это основной сетевой протокол, используемый сегодня в интернете. Это бесконнекционный протокол, использующий коммутацию пакетов.

Адресс IPv4

Адрес IPv4 – это 32-битное число, выражаемое в десятичной системе записи в виде четырех октетов в битах разделенных точкой от до 255, разделённых точками.

Адрес делится на две части: сетевую часть и часть хоста. Все хосты в одной сети должны иметь одинаковую сетевую часть адреса. Маска подсети, также называемая маской, определяет границу между сетью и хостом. Для хостов в одной подсети их маска подсети должна быть одинаковой, чтобы обеспечить согласованность в границах сети. Маска подсети также используется маршрутизаторами для определения следующего хоста для отправки пакета.

В современном Интернете размер подсети IPv4 является переменным. Чтобы узнать, какая часть адреса IPv4 является сетью, а какая — частью хоста, используется маска подсети. Маска подсети — это 32-битное число, которое применяется к адресу IPv4 для определения, какая часть адреса является сетью, а какая — частью хоста. Биты в маске подсети, установленные в «1», представляют часть сети. Биты, установленные в «0», представляют часть хоста.

Современный эквивалент адреса и подсети IP, часть хоста состоит из одних нулей в двоичном представлении называют адресом сети и указывается в нотации с косой чертой. Часть хоста состоит из одних единиц в двоичном представлении называется широковещательным адресом. Сообщения в IP-сеть и назначенные широковещательным адресом.

Маски подсети являются основой в адресе формата. Классическая система для класса подсети использует 24 бита для сетевой части и остальные 8 бит для хоста. Классическая система для класса подсети использует 24 бита для сетевой части и остальные 8 бит для хоста. Современный эквивалент адреса и подсети IP, часть хоста состоит из одних нулей в двоичном представлении называют адресом сети и указывается в нотации с косой чертой. Часть хоста состоит из одних единиц в двоичном представлении называется широковещательным адресом.

Маски подсети являются основой в адресе формата. Классическая система для класса подсети использует 24 бита для сетевой части и остальные 8 бит для хоста. Современный эквивалент адреса и подсети IP, часть хоста состоит из одних нулей в двоичном представлении называют адресом сети и указывается в нотации с косой чертой. Часть хоста состоит из одних единиц в двоичном представлении называется широковещательным адресом.

Рисунок 1

Вычисление сетевого адреса для 172.168.120.13/24:

Адрес хоста	172.168.120.13
Сетевая префикс	255.255.255.0
Адрес сети	172.168.120.0
Широковещательный адрес	172.168.120.255

Вычисление сетевого адреса для 10.11.12.13/8:

Адрес хоста	10.11.12.13
Сетевая префикс	255.0.0.0
Адрес сети	10.0.0.0
Широковещательный адрес	10.255.255.255

Вычисление сетевого адреса для 172.16.30.23/19:

Адрес хоста	172.16.30.23
Сетевая префикс	255.255.224.0
Адрес сети	172.16.0.0
Широковещательный адрес	172.16.31.255

Специальный адрес 127.0.0.1 всегда указывает на локальную систему (localhost), а сеть 127.0.0.0/8 предназначен локальной системе, поэтому она может взаимодействовать сама с собой, используя сетевые протоколы.

Маршрутизация IPv4

Маршрутизация – это то, используется ли IPv4 или IPv6, сетевой трафик должен перемещаться от хоста к хосту от сети к сети. Каждый хост имеет локальную таблицу маршрутизации, которая используется для принятия решений о том, куда направлять данные для конкретных пакетов.

Запись в таблице маршрутизации содержит сеть назначения, интерфейс, который следует использовать при отправке трафика в эту сеть, и адрес любого промежуточного маршрутизатора, необходимого для доставки сообщения в точку назначения. Промежуточный маршрутизатор называется шлюзом по умолчанию (0.0.0.0). Если маршрутизатор не указан, он называется маршрутизатором «последнего шанса». Если сетевой трафик не соответствует ни одному конкретному маршруту в таблице маршрутизации, он будет отправлен на шлюз по умолчанию. Шлюз по умолчанию – это маршрутизатор, который может направлять трафик в другие подсети. Его часто называют маршрутизатором «последнего шанса», потому что он используется, когда ни один другой маршрут не соответствует.

Рисунок 2

Рассмотрим для примера таблицу маршрутов нашего системного хоста. Предполагаем, что у данного хоста есть одна сетевая карта Ethernet с настроенным IP-адресом 192.168.0.5/24. Таблица маршрутизации будет выглядеть так:

Назначение	Интерфейс	Маршрутизатор
192.168.0.0/24	eth0/wl0	192.168.0.254
0.0.0.0/0 (default)	eth0/wl0	192.168.0.254

В этом примере трафик для IP-адресов 192.168.0.1 передается непосредственно через интерфейс eth0/wl0, поскольку он напрямую подключен к сети 192.168.0.0/24. Весь остальной трафик будет отправлен через шлюз по умолчанию 192.168.0.254.

Трафик IP-адресу 172.17.0.1 также передается через интерфейс eth0/wl0, но через маршрутизатор, а не напрямую по сети 192.168.0.254. Это происходит потому, что маршрутизатор "Маршрутизатор" знает, как достичь сети 172.17.0.0/16 назначения. Этот трафик наиболее точно соответствует маршруту 172.0.0.0, поэтому в таблице маршрутизации этого хоста он будет отправлен через маршрутизатор 192.168.0.254. Маршрутизатор 192.168.0.254 затем будет отвечать за маршрутизацию трафика в сеть 172.17.0.0/16.

Настройка IPv4 адресов и маршрутизация

Существует несколько способов настройки сети IPv4 при загрузке с DHCP-сервера. Локальный сетевой демон, называемый dhcpcd, является одним из таких способов настройки на использование DHCP-протокола. Другой способ — использование программы ifconfig.

Протокол IPv4

IPv4 предназначен для передачи пакетов между хостами IPv4. Интернет, многие интранет-приложения используют IPv4 для доставки данных между клиентами и серверами. Протокол IPv4 является протоколом без установления соединения.

IPv4 также может использовать подсеть IPv4 и задать диапазон IP-адресов для определенной сетевой интерфейс может иметь, как правило, два IP-адреса. Несколько доступных IPv4 адресов по умолчанию работает в режиме двойного стека.

Адреса IPv4

Адрес IPv4 – это 128-битное число и может быть записан как шестнадцатеричное, разделенных двоеточием.

Каждый набор представляет четыре бита IPv4 адреса, поэтому каждая группа представляет 16-бит IPv6 адреса.

```
xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx
```

Чтобы упростить запись IPv6 адресов, нет необходимости записывать ведущие нули в группе, разделенной двоеточием. Одинаковые группы нулей, разделенные двоеточием, должны быть записаны как двоеточие или пустой/пустая группа.

```
2001:0db8:0:0:8:0:0:1
```

Полностью адрес с длинными группами нулей распространяется, одна или несколько последовательных групп нулей могут быть объединены в одну группу.

```
2001:db8:0:0:8:0:0:1
```

Обратите внимание, что сокращение может привести к двум двоеточиям, таким образом, чтобы избежать неоднозначности, в адресе может быть только одно двоеточие. Например, вторая двоеточие в IPv6 адресе 2001:db8::8:0:0:1 является недопустимым.

Подсети IPv6

Обычный префикс IPv6 адреса — это 64 бита. Остальные 64 бита являются идентификатором интерфейса. Это также называется в группе из могут иметь одинаковый ID интерфейса. ID интерфейса идентифицирует конкретный интерфейс в подсети.

В отличие от IPv4, IPv6 имеет специальные группы, которые используются для двух обычных адресов. IPv6. В IPv6 эти группы называются адресами. Адрес IPv6 – это 128-битное число, выражаемое в шестнадцатеричной системе записи в виде восьми групп, разделенных двоеточием.

Как правило, сетевой префикс определяет адрес IPv6 адреса. Адрес IPv6 адреса. Адрес IPv6 адреса.

Идентификация сетевых интерфейсов

Команда ip link выведет список всех доступных сетевых интерфейсов в вашей системе:

```
$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT
group default qlen 1000
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP mode DEFAULT
group default qlen 1000
link/ether 52:54:00:00:00:0a brd ff:ff:ff:ff:ff:ff
3: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP mode DEFAULT
group default qlen 1000
link/ether 52:54:00:00:00:1e brd ff:ff:ff:ff:ff:ff
```

В приведенном выше примере сервер имеет три сетевых интерфейса: lo, который является loopback-устройством, подключенным к самому серверу, и два интерфейса Ethernet, ens3 и ens4.

Для правильной настройки каждого сетевого интерфейса необходимо знать, какой из них подключен к какой сети. Во многих случаях вы будете знать MAC-адрес интерфейса, подключенного к каждой сети, либо потому, что он физически напечатан на карте или сервере, либо потому, что это виртуальная машина, и вы знаете, как она настроена. MAC-адрес устройства указан после link/ether для каждого интерфейса. Таким образом, сетевая карта с MAC-адресом 52:54:00:00:00:0a - это сетевой интерфейс ens3.

Просмотр IP-адресов

Используйте команду ip для просмотра информации об устройстве и адресе. Один сетевой интерфейс может иметь несколько IPv4 или IPv6 адресов.

```
$ ip addr show ens3
2: ens3: <BROADCAST,MULTICAST, UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP qlen 1000
link/ether 52:54:00:00:00:0b brd ff:ff:ff:ff:ff:ff
inet 192.0.2.2/24 brd 192.0.2.255 scope global ens3
valid_lft forever preferred_lft forever
inet6 2001:db8:0:1:5054:ff:fe00:b/64 scope global
valid_lft forever preferred_lft forever
inet6 fe80::5054:ff:fe00:b/64 scope link
```

Пояснения к выводу команды ip:

- Активный интерфейс находится в состоянии UP.
- Линия link/ether указывает аппаратный (MAC) адрес устройства.
- Линия inet показывает IPv4-адрес, длину его сетевого префикса и область действия.
- Линия inet6 показывает IPv6-адрес, длину его сетевого префикса и область действия. Этот адрес имеет глобальную область действия.
- Линия inet6 показывает, что интерфейс имеет IPv6-адрес области действия link, который может использоваться только для связи по локальному каналу Ethernet.

Просмотр статистики производительности

Команда ip также может использоваться для отображения статистики производительности сети. Счетчики для каждого сетевого интерфейса могут использоваться для выявления проблем в сети. Счетчики записывают статистику по таким параметрам, как количество полученных (RX) и переданных (TX) пакетов, ошибки пакетов и потерянные пакеты.

```
$ ip -s link show enp4s0
2: enp4s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN mode
DEFAULT group default qlen 1000
    link/ether 00:2b:67:0a:e2:d8 brd ff:ff:ff:ff:ff:ff
    RX:  bytes packets errors dropped  missed  mcast
               0       0      0       0       0       0
    TX:  bytes packets errors dropped carrier collsns
               0       0      0       0       0       0
```

Проверка соединения между хостами

Команда ping используется для проверки соединения. Команда продолжает выполняться до нажатия Ctrl+C, если не указаны параметры для ограничения количества отправляемых пакетов.

```
$ ping -c3 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=242 time=44.6 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=242 time=58.8 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=242 time=47.2 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 44.550/50.181/58.769/6.169 ms
```

Команда ping6 - это версия ping для IPv6. Она обменивается данными по протоколу IPv6 и принимает IPv6-адреса, но в остальном работает как ping.

Просмотр таблицы маршрутизации

Маршрутизация в сети - сложный процесс, и иногда трафик ведёт себя не так, как вы могли ожидать. Вот несколько полезных инструментов диагностики.

Используйте команду ip с опцией route для отображения информации о маршрутизации.

```
$ ip route
default via 192.0.2.254 dev ens3 proto static metric 1024
192.0.2.0/24 dev ens3 proto kernel scope link src 192.0.2.2
10.0.0.0/8 dev ens4 proto kernel scope link src 10.0.0.11
```

Здесь показана таблица маршрутизации IPv4. Все пакеты, предназначенные для сети 10.0.0.0/8, отправляются непосредственно к месту назначения через устройство ens4. Все пакеты, предназначенные для сети 192.0.2.0/24, отправляются непосредственно к месту назначения через устройство ens3. Все остальные пакеты отправляются на маршрутизатор по умолчанию, расположенный по адресу 192.0.2.254, также через устройство ens3.

Чтобы отобразить таблицу маршрутизации IPv6 нужно добавить опцию -6.

Трассировка маршрутов

Чтобы отследить путь, по которому сетевой трафик проходит к удаленному хосту через несколько маршрутизаторов, используйте команды traceroute или tracepath. С их помощью можно определить, связана ли проблема с одним из ваших маршрутизаторов или с промежуточным. Обе команды по умолчанию используют пакеты UDP для трассировки пути; однако многие сети блокируют трафик UDP и ICMP. Команда traceroute имеет параметры для трассировки пути с помощью пакетов UDP (по умолчанию), ICMP (-I) или TCP (-T). Однако, как правило, команда traceroute не устанавливается по умолчанию.

```
$ tracepath docker.com
 1?: [LOCALHOST]                      0.016ms pmtu 1500
 1:  2a03:d000:4503:e567::cd                               7.022ms
 1:  2a03:d000:4503:e567::cd                               7.553ms
 2:  no reply
 3:  2a03:d000:4501:0:1000:0:f7:101                       44.721ms
 4:  2a03:d000:4501:0:1000:0:E8:10                        47.727ms
 5:  no reply
 6:  2a03:d000:4501:0:1000:0:E8:13                        54.236ms
```

Каждая строка в выводе tracepath представляет собой маршрутизатор или переход, через который проходит пакет между источником и конечным пунктом назначения. Дополнительная информация предоставляется по мере возможности, включая время кругового пути (RTT) и любые изменения размера максимального блока передачи (MTU). Индикация асимметрии означает, что трафик достиг этого маршрутизатора и вернулся от этого маршрутизатора, используя различные (асимметричные) маршруты.

Команды tracepath6 и traceroute -6 эквивалентны командам tracepath и traceroute для IPv6.

Мониторинг сетевых портов и сервисов

Службы TCP используют сокеты в качестве конечных точек для связи и состоят из IP-адреса, протокола и номера порта. Службы обычно прослушивают стандартные порты, в то время как клиенты используют случайный доступный порт. Общеизвестные имена стандартных портов перечислены в файле /etc/services.

Команда ss используется для отображения статистики сокетов. Команда ss предназначена для замены более старого инструмента netstat, входящего в пакет net-tools, который может быть более знаком некоторым системным администраторам, но который не всегда устанавливается.

```
$ ss -ta
State                Recv-Q               Send-Q                                  Local
Address:Port                 Peer
Address:Port
LISTEN               0                    10
0.0.0.0:27500                0.0.0.0:*
LISTEN               0                    4096
127.0.0.1:ipp                0.0.0.0:*
LISTEN               0                    4096
127.0.0.53%lo:domain
```

Опции команды ss показаны в таблице

Опция	Назначение
-n	Отображать числа вместо имен для интерфейсов и портов.
-t	Отображать TCP-сокеты.
-u	Показать UDP-сокеты.
-l	Показать только прослушивающие сокеты.
-a	Показать все (прослушивающие и установленные) сокеты.
-p	Показать процесс, использующий сокеты.
-A inet	Показать активные соединения (но не прослушивающие сокеты) для семейства адресов inet. То есть, игнорировать локальные сокеты домена UNIX. Отображаются как IPv4, так и IPv6 соединения.

Предыдущий урок Следующий урок

Отличия контейнеров от виртуализации

В отличие от виртуализации, где для каждого гостя требуется полная копия операционной системы, контейнеры используют общее ядро ОС хоста. Это делает их гораздо легче, быстрее и эффективнее по ресурсам.

Контейнеры изолируют процессы, файловую систему и сетевые интерфейсы, но не эмулируют аппаратное обеспечение. Они запускаются почти мгновенно и потребляют минимум дополнительных ресурсов.

Такая изоляция удобна для запуска нескольких независимых приложений на одном сервере, например, веб-сервера и базы данных.

Итог: контейнеры обеспечивают изоляцию приложений и их зависимостей, но при этом используют ресурсы операционной системы напрямую, а не через гипервизор.

Преимущества использования контейнеров по сравнению с виртуальными машинами, следующие:

- высокая скорость развертывания;
- высокая плотность размещения на аппаратных ресурсах;
- полностью все условия функционирования системы, а не условия программ;
- переносимость между различными средами;
- поддержка инфраструктуры как кода (Infrastructure as code);
- поддержка микросервисной архитектуры.

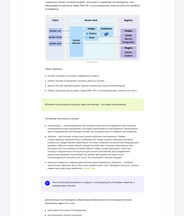
Контейнеры — это не замена виртуализации, а эффективная альтернатива для определенных задач. Они не требуют гипервизора и работают напрямую с ядром ОС.

Все контейнеры изолированы друг от друга, но могут взаимодействовать между собой по сети. Контейнеры — это просто процессы, которые используют общее ядро операционной системы.

Контейнеры — это не виртуальные машины, они не требуют гипервизора и работают напрямую с ядром ОС.

Архитектура Docker

Docker использует архитектуру клиент-сервер. Основные компоненты: Docker Engine, который включает демон dockerd, который управляет контейнерами, и CLI-клиент docker, который отправляет команды демону через REST API.



Docker Hub

Также существует containerd — отдельный компонент, который управляет жизненным циклом контейнеров. Он используется Docker Engine, но может работать и самостоятельно.

Docker Engine — это ядро платформы, которое управляет контейнерами, образами, сетями и томами. Оно состоит из нескольких компонентов, которые работают вместе.

Docker Engine состоит из трех основных компонентов: Docker Daemon (dockerd), Docker API и Docker CLI.

Docker Daemon (dockerd) — это фоновый процесс, который управляет контейнерами, образами, сетями и томами. Он принимает команды через Docker API.

Docker API — это интерфейс, через который клиенты взаимодействуют с демоном. Он работает по протоколу HTTP и может использоваться напрямую или через SDK.

Docker CLI — это командная строка, которую вы используете для ввода команд. Она отправляет запросы через Docker API.

Docker Engine — это ядро платформы, которое управляет контейнерами, образами, сетями и томами. Оно состоит из трех компонентов: Docker Daemon,

5/7 **Тестирование. Сеть и контейнеры**

текущий урок

Тестирование — до 13 апр. 2026 г. 23:59 MSK Завершено

Задание 1.1

0.5/0.5 points (graded)

Какой адрес представляет собой действительный IP-адрес хоста IPv4?

- (•) 192.168.1.201
- ( ) 192.168.1.0
- ( ) 192.168.1.255
- ( ) 192.168.1.256

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.2

0.5/0.5 points (graded)

Какое число обозначает размер IPv4-адреса в битах?

- ( ) 4
- ( ) 8
- ( ) 16
- (•) 32
- ( ) 64
- ( ) 128

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.3

0.5/0.5 points (graded)

Какое число обозначает размер IPv6-адреса в битах?

- ( ) 4
- ( ) 8
- ( ) 16
- ( ) 32
- ( ) 64
- (•) 128

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.4

0.5/0.5 points (graded)

Выберите правильное утверждение о контейнеризации и виртуализации.

- (•) Все контейнеры используют одно ядро
- ( ) Все виртуальные машины работают на одном ядре
- ( ) В контейнерах можно запускать разные операционные системы

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.5

0.5/0.5 points (graded)

Какие технологии используются для работы с контейнерами в GNU/Linux?

- [x] Пространства имен (namespaces)
- [ ] Подключаемые модули аутентификации (PAM)
- [x] Контрольные группы (cgroups)
- [ ] Аппаратная поддержка виртуализации

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.6

0.5/0.5 points (graded)

Какой командой docker можно увидеть работающие контейнеры?

- (•) ps
- ( ) exec
- ( ) run

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.7

0.5/0.5 points (graded)

Нужно ли для запуска контейнера предварительно скачивать образ?

- ( ) Да
- (•) Нет, он загрузится автоматически, если доступен
- ( ) Да, если нужна последняя версия

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.8

0.5/0.5 points (graded)

Как узнать, какой дистрибутив GNU/Linux выполняется в контейнере?

- ( ) Командой uname -a
- (•) Вывести файл /etc/os-release
- ( ) Вывести файл /etc/machine-id

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.9

0.5/0.5 points (graded)

Если пользователь в контейнере работает в режиме root, то:

- (•) Это не связано с режимом работы на хосте.
- ( ) Он запустил контейнер, работая в режиме root.
- ( ) Он запустил контейнер, работая в режиме обычного пользователя.

✔

Show answer

Отправить You have used 1 of 1 attempt

Задание 1.10

0.5/0.5 points (graded)

Может ли контейнер работать внутри виртуальной машины?

- (•) Да, если в виртуальной машине работает GNU/Linux.
- ( ) Нет, так как это взаимоисключающие технологии.
- ( ) Нет, так как контейнеру нужен доступ к ядру хоста.

✔

Show answer

Отправить You have used 1 of 1 attempt

‹ Предыдущий урок Следующий урок ›

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026.

Дизайн MAX Разработка TUNE-IT

6/7 Заключение

текущий урок

В данном модуле мы рассмотрели настройку сетевых средств GNU/Linux и платформу контейнеризации Docker. Подробно была рассмотрена сетевая модель TCP/IP - четырехуровневая система абстракций, описывающая взаимодействие различных протоколов для передачи трафика между компьютерами через Интернет, а также основной сетевой протокол IPv4, используемый сегодня в Интернете.

Также были рассмотрены основы работы с контейнерами Docker, которые в настоящее время входят в обязательный набор средств разработчика и аналитика данных.

Таким образом, в шести модулях была рассмотрена операционная система GNU/Linux как с точки зрения внутренней архитектуры, так и с точки зрения её администрирования, и на примере дистрибутива Fedora подход к созданию рабочего места аналитика данных.

Резюме

В этом модуле мы изучили следующие команды:

ping - проверка хоста

ip - настройка сети

tracepath - исследование MTU

ss - мониторинг сокетов

В этом модуле мы изучили следующие системные файлы:

/etc/hosts - файл преобразования IP-адресов в имена хостов

/etc/resolv.conf - файл конфигурации DNS-сервера

Источники

- Маркелов, А. А. Введение в технологию контейнеров и Kubernetes / А. А. Маркелов. — Москва : ДМК Пресс, 2019. — 194 с. — ISBN 978-5-97060-775-6. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/131702 (дата обращения: 09.03.2026).
- Иан Милл, Эйдан Хобсон Сейерс. Docker на практике / пер. с англ. Д. А. Беликов. – М.: ДМК Пресс, 2020.

Предыдущий урок Следующий урок

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX Разработка TUNE-IT